

Prácticas de Tecnologías de Gestión y Manipulación de Datos

Guillermo López Taboada (guillermo.lopez.taboada@udc.es)

Diego Darriba (diego.darriba@udc.es)

Rubén Fernández Casal (ruben.fcasal@udc.es)

Manuel Oviedo de la Fuente (manuel.oviedo@udc.es)

Edición: Octubre de 2025. Impresión: 2026-09-17

Índice

Prólogo	5
1 Introducción	7
1.1 Contenidos	7
1.2 Planificación (tentativa)	7
1.3 Fuentes de información:	8
2 Manipulación de datos con R	11
2.1 Lectura, importación y exportación de datos	11
2.2 Manipulación de datos	16
2.3 Ejemplo WoS data	30
2.4 Eficiencia en R: preasignación, vectorización y desfases	34
2.5 Resumen: R base	36
2.6 Ejercicios	36
3 Introducción al lenguaje SQL	39
3.1 Bases de Datos Relacionales	39
3.2 Restricciones	41
3.3 Sistemas Gestores de Bases de Datos (SGDB)	43
3.4 Sintaxis SQL	44
3.5 Cláusulas básicas de SQL	45
3.6 Gestión de datos	48
3.7 Gestión de Bases de Datos	50
3.8 Gestión de tablas	50
3.9 Vistas	52
3.10 Ejemplos de consultas SQL	53
3.11 Conexión con bases de datos desde R	53
3.12 Ejemplo Scopus data	56
3.13 Ejercicios SQL con SQLite	56
3.14 Práctica 1: SQL	57
4 Python	59
4.1 Introducción a Python	59
4.2 Python y SQLite	65
4.3 Context Manager	69
4.4 CSV en python	70

5	Manipulación de datos con tidyverse	73
5.1	Introducción al ecosistema tidyverse	73
5.2	Manipulación de datos con dplyr y tidyr	77
5.3	Herramientas tidyr	86
5.4	Operaciones con tablas de datos	86
5.5	Bases de datos con dplyr	87
5.6	Eficiencia y desfases con dplyr	91
5.7	Resumen: dplyr/tidyr y R base	92
5.8	Ejercicios	92
6	Introducción a Tecnologías NoSQL	95
6.1	Conceptos y tipos de bases de datos NoSQL (documental, columnar, clave/valor y de grafos)	95
6.2	Conexión de R a MongoDB	102
6.3	Ejercicios prácticos con MongoDB	102
7	Tecnologías para el Tratamiento de Datos Masivos	103
7.1	Introducción al Aprendizaje Estadístico	103
7.2	Tecnologías Big Data (Hadoop/Spark y Visualización)	125
7.3	Introducción al Análisis de Datos Masivos	131
	Bibliografía	133

Prólogo

Este libro contiene algunas de las prácticas de la asignatura de Tecnologías de Gestión de Datos del Máster interuniversitario en Técnicas Estadísticas).

En este libro se asume que se dispone de conocimientos básicos de R, un lenguaje de programación interpretado y un entorno estadístico desarrollado específicamente para el análisis de datos. Para instalar R se recomienda seguir los pasos descritos en el post <https://rubenfcasal.github.io/post/instalacion-de-r>. Para el desarrollo de código e informes se sugiere emplear *RStudio Desktop*, que se puede instalar y configurar siguiendo las indicaciones proporcionadas en el post <https://rubenfcasal.github.io/post/instalacion-de-rstudio>.

Para una introducción a la programación en R se puede consultar el libro Fernández-Casal et al. (2022): *Introducción al Análisis de Datos con R* (github). Adicionalmente, en el post <https://rubenfcasal.github.io/post/ayuda-y-recursos-para-el-aprendizaje-de-r> se proporcionan enlaces a recursos adicionales, incluyendo bibliografía y cursos. También puede ser de utilidad el libro Fernández-Casal (2023): *Notas de Programación en R* (github).

Este libro ha sido escrito en R-Markdown empleando el paquete `bookdown` y está disponible en el repositorio Github: [gltaboada/tgdbook](https://gltaboada.github.io/tgdbook). Se puede acceder a la versión en línea a través del siguiente enlace:

<https://gltaboada.github.io/tgdbook>.

donde puede descargarse en formato pdf.

Para ejecutar los ejemplos mostrados en el libro será necesario tener instalados los siguientes paquetes: `dplyr` (colección `tidyverse`), `tidyr`, `stringr`, `readxl`, `openxlsx`, `naniar`, `RODBC`, `sqldf`, `RSQLite`, `foreign`, `magrittr`, `knitr`, `GGally`, `caret`, `MASS`, `class`, `e1071`, `mpae` y `microbenchmark`. Por ejemplo mediante los comandos:

```
pkgs <- c('dplyr', 'tidyr', 'stringr', 'readxl', 'openxlsx', 'magrittr',
          'naniar', 'RODBC', 'sqldf', 'RSQLite', 'foreign', 'knitr',
          'GGally', 'caret', 'MASS', 'class', 'e1071', 'mpae', 'microbenchmark')
# install.packages(pkgs, dependencies=TRUE)
install.packages(setdiff(pkgs, installed.packages()[, 'Package']), dependencies = TRUE)
```

Para generar el libro (compilar) se recomendaría consultar el libro de “Escritura de libros con bookdown” en castellano.



Este obra está bajo una licencia de Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional (esperamos poder liberarlo bajo una licencia menos restrictiva más adelante...).

Capítulo 1

Introducción

La información relevante de la materia está disponible en la guía docente y la ficha de la asignatura

En particular, los resultados de aprendizaje son:

- Manejar de forma autónoma y solvente el software necesario para acceder a conjuntos de datos en entornos profesionales y/o en la nube.
- Saber gestionar conjuntos de datos masivos en un entorno multidisciplinar que permita la participación en proyectos profesionales complejos que requieran el uso de técnicas estadísticas.
- Saber relacionar el software de diseño y gestión de bases de datos con el específicamente implementado para el análisis de datos.

1.1 Contenidos

1. Introducción al lenguaje SQL
 - Bases de datos relacionales
 - Sintaxis SQL
 - Conexión con bases de datos desde R
2. Introducción a tecnologías NoSQL
 - Conceptos y tipos de bases de datos NoSQL (documental, columnar, clave/valor y de grafos)
 - Conexión de R a NoSQL
3. Tecnologías para el tratamiento de datos masivos
 - Introducción al Aprendizaje Estadístico
 - Tecnologías Big Data (Hadoop, Spark, Sparklyr)
 - Ejercicios de análisis de datos masivos.

1.2 Planificación (tentativa)

La impartición de los contenidos durante el curso dependerá de los conocimientos de partida y la asimilación de los conceptos. Para completar nuestra visión de los conocimientos previos os requerimos completar este formulario en la primera sesión de clase: <https://forms.gle/D5bhiLLBUFuh6k1n8>

- Semana 1 (20/10) : 21/10 - Presentación
- Semana 2 (27/10) : 28 y 30/10 - Tema 1 - SQL

- Semana 3 (03/11) : 04 y 06/11 - Seminario dplyr
- Semana 4 (10/11) : 11, 13 y 14/11 - Tema 1 - SQL
- Semana 5 (17/11) : 18, 20/11 - Tema 1 - SQL. + 21/11 - Seminario texto proc. (CSV, excel, Json) y open data
- Semana 6 (24/11) : 25/11 - Tema 2 NOSQL + 27 y 28/11 - Tema 3 Big Data
- Semana 7 (1/12) : 2, 4 y 5/12 Tema 3 Big Data.
- Semana 8 (8/12) : 9 y 11/12 - Intro a AE + 12/12 Tutoría práctica ML.
- Semana 9 (15/12) : 16, 18 y 19/12 - Clases de paso / tutorías de cara a examen.

1.2.1 Evaluación

- **Examen** (60%): El examen de la materia evaluará los siguientes aspectos: Conceptos de la materia: Dominio de los conocimientos teóricos y operativos de la materia. Asimilación práctica de materia: Asimilación y comprensión de los conocimientos teóricos y operativos de la materia.
- **Prácticas de laboratorio** (40%): Evaluación de las prácticas de laboratorio desarrolladas por los estudiantes.

1.3 Fuentes de información:

1.3.1 Básica

- Daroczi, G. (2015). Mastering Data Analysis with R. Packt Publishing
- Grolmund, G. y Wickham, H. (2016). R for Data Science O'Reilly
- Silberschatz, A., Korth, H. y Sudarshan, S. (2014). Fundamentos de Bases de Datos. Mc Graw Hill
- Rubén Fernández Casal y Julián Costa Bouza. Apuntes de Aprendizaje Estadístico
- Luraschi, J., Kuo, K., Ruiz, K. Mastering Spark with R O'Reilly
- Rubén Fernández Casal (R Machinery):
 - Introducción al Análisis de Datos con R (con Javier Roca, Julián Costa y Manuel Oviedo)
 - Ayuda y Recursos para el Aprendizaje de R
 - Notas de Programación en R
 - Capítulo introducción a Rmarkdown
 - Escritura de libros con el paquete bookdown (con Tomás Cotos)
 - Pesentación análisis de datos con R

1.3.2 Complementaria:

- Wes McKinney (2017). Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython. O'Reilly (2ª ed.)
- Tom White (2015). Hadoop: The Definitive Guide. O'Reilly (4ª ed.)
- Alex Holmes (2014). Hadoop in practice. Manning (2ª ed.)

- Centro de Supercomputación de Galicia (2020). [Servicio de Big Data del CESGA] (<https://bigdata.cesga.es/>)

Capítulo 2

Manipulación de datos con R

En el proceso de análisis de datos, al margen de su obtención y organización, una de las primeras etapas es el acceso y la manipulación de los datos (ver Figura 2.1). En este capítulo se repasarán brevemente las principales herramientas disponibles en el paquete base de R para la lectura, exploración, filtrado, transformación y combinación de tablas de datos. Posteriormente en el Capítulo 5 se mostrará como alternativa el uso del paquete `dplyr`, resolviendo los mismos ejercicios de la Sección 2.6 con ambos enfoques.

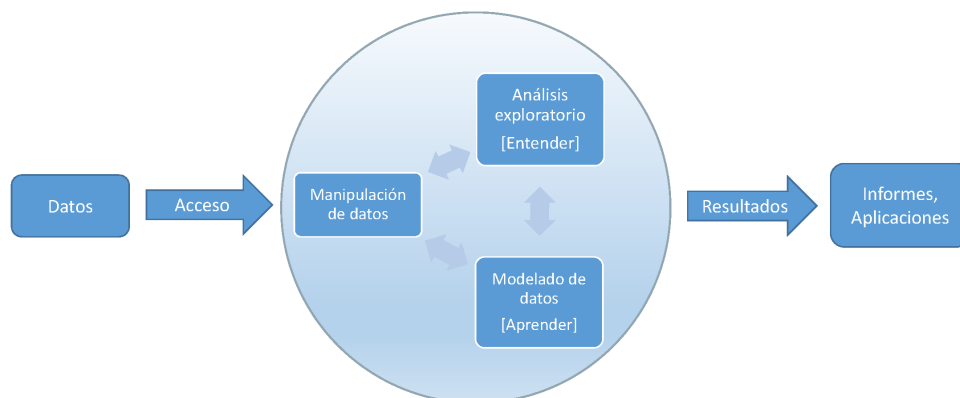


Figura 2.1: Etapas del proceso

2.1 Lectura, importación y exportación de datos

Además de la introducción directa, R es capaz de importar datos externos en múltiples formatos:

- bases de datos disponibles en librerías de R
- archivos de texto en formato ASCII
- archivos en otros formatos: Excel, SPSS, Matlab...
- bases de datos relacionales: MySQL, Oracle...
- formatos web: HTML, XML, JSON...
- otros lenguajes de programación: Python, Julia...

2.1.1 Formato de datos de R

El formato de archivo en el que habitualmente se almacena objetos (datos) R es binario y está comprimido (en formato "gzip" por defecto). Para cargar un fichero de datos se emplea normalmente `load()`. A continuación se utiliza el fichero `empleados.RData` que contiene datos de empleados de un banco.

```
res <- load("data/empleados.RData")
res
```

```
## [1] "empleados"
```

```
ls()
```

```
## [1] "cite_cran" "cite_fig" "cite_fig2" "cite_fun"
## [5] "cite_fun_" "cite_pkg" "cite_pkg_" "citefig"
## [9] "citefig2" "empleados" "fig.path" "inline"
## [13] "inline2" "is_html" "is_latex" "latexfig"
## [17] "latexfig2" "res"
```

y para guardar `save()`:

```
# Guardar
save(empleados, file = "data/empleados_new.RData")
```

Aunque, como indica este comando en la ayuda (`?save`):

For saving single R objects, `saveRDS()` is mostly preferable to `save()`, notably because of the functional nature of `readRDS()`, as opposed to `load()`.

```
saveRDS(empleados, file = "data/empleados_new.rds")
## restore it under a different name
empleados2 <- readRDS("data/empleados_new.rds")
# identical(empleados, empleados2)
```

Normalmente, el objeto empleado en R para almacenar datos en memoria es el `data.frame`.

2.1.2 Acceso a datos en paquetes

R dispone de múltiples conjuntos de datos en distintos paquetes, especialmente en el paquete `datasets` que se carga por defecto al abrir R. Con el comando `data()` podemos obtener un listado de las bases de datos disponibles.

Para cargar una base de datos concreta se utiliza el comando `data(nombre)` (aunque en algunos casos se cargan automáticamente al emplearlos). Por ejemplo, `data(cars)` carga la base de datos llamada `cars` en el entorno de trabajo ("`.GlobalEnv`") y `?cars` muestra la ayuda correspondiente con la descripción de la base de datos.

2.1.3 Lectura de archivos de texto

En R, para leer archivos de texto se suele utilizar la función `read.table()`. Suponiendo, por ejemplo, que en el directorio actual está el fichero `empleados.txt`. La lectura de este fichero vendría dada por el código:

```
# Session > Set Working Directory > To Source...?
datos <- read.table(file = "data/empleados.txt", header = TRUE)
# head(datos)
str(datos)
```

```
## 'data.frame': 474 obs. of 10 variables:
## $ id : int 1 2 3 4 5 6 7 8 9 10 ...
## $ sexo : chr "Hombre" "Hombre" "Mujer" "Mujer" ...
## $ fechnac : chr "2/3/1952" "5/23/1958" "7/26/1929" "4/15/1947" ...
## $ educ : int 15 16 12 8 15 15 15 12 15 12 ...
## $ catlab : chr "Directivo" "Administrativo" "Administrativo" "Administrativo" ...
## $ salario : num 57000 40200 21450 21900 45000 ...
## $ salini : int 27000 18750 12000 13200 21000 13500 18750 9750 12750 13500 ...
## $ tiempemp : int 98 98 98 98 98 98 98 98 98 98 ...
## $ expprev : int 144 36 381 190 138 67 114 0 115 244 ...
## $ minoria : chr "No" "No" "No" "No" ...
```

```
class(datos)
```

```
## [1] "data.frame"
```

Si el fichero estuviese en el directorio *c:\datos* bastaría con especificar `file = "c:/datos/empleados.txt"`. Nótese también que para la lectura del fichero anterior se ha establecido el argumento `header=TRUE` para indicar que la primera línea del fichero contiene los nombres de las variables.

Los argumentos utilizados habitualmente para esta función son:

- **header**: indica si el fichero tiene cabecera (`header=TRUE`) o no (`header=FALSE`). Por defecto toma el valor `header=FALSE`.
- **sep**: carácter separador de columnas que por defecto es un espacio en blanco (`sep=" "`). Otras opciones serían: `sep=","` si el separador es un “,”, `sep="*"` si el separador es un “*”, etc.
- **dec**: carácter utilizado en el fichero para los números decimales. Por defecto se establece `dec = "."`. Si los decimales vienen dados por “,” se utiliza `dec = ","`.

Resumiendo, los (principales) argumentos por defecto de la función `read.table` son los que se muestran en la siguiente línea:

```
read.table(file, header = FALSE, sep = " ", dec = ".")
```

Para más detalles sobre esta función véase `help(read.table)`.

Están disponibles otras funciones con valores por defecto de los parámetros adecuados para otras situaciones. Por ejemplo, para ficheros separados por tabuladores se puede utilizar `read.delim()` o `read.delim2()`:

```
read.delim(file, header = TRUE, sep = "\t", dec = ".")
read.delim2(file, header = TRUE, sep = "\t", dec = ",")
```

2.1.4 Importación desde SPSS

El programa R permite lectura de ficheros de datos en formato SPSS (extensión *.sav*) sin necesidad de tener instalado dicho programa en el ordenador. Para ello se necesita:

- cargar la librería `foreign`
- utilizar la función `read.spss`

Por ejemplo:

```
library(foreign)
datos <- read.spss(file = "data/Employee data.sav",
                  to.data.frame = TRUE)
```

```
# head(datos)
str(datos)

## 'data.frame':    474 obs. of  10 variables:
## $ id      : num  1 2 3 4 5 6 7 8 9 10 ...
## $ sexo    : Factor w/ 2 levels "Hombre","Mujer": 1 1 2 2 1 1 1 2 2 2 ...
## $ fechnac  : num  1.17e+10 1.19e+10 1.09e+10 1.15e+10 1.17e+10 ...
## $ educ     : Factor w/ 10 levels "8","12","14",...: 4 5 2 1 4 4 4 2 4 2 ...
## $ catlab   : Factor w/ 3 levels "Administrativo",...: 3 1 1 1 1 1 1 1 1 1 ...
## $ salario  : Factor w/ 221 levels "15750","15900",...: 179 137 28 31 150 101 121 31 71 45 ...
## $ salini   : Factor w/ 90 levels "9000","9750",...: 60 42 13 21 48 23 42 2 18 23 ...
## $ tiempemp : Factor w/ 36 levels "63","64","65",...: 36 36 36 36 36 36 36 36 36 36 ...
## $ expprev  : Factor w/ 208 levels "Ausente","10",...: 38 131 139 64 34 181 13 1 14 91 ...
## $ minoria  : Factor w/ 2 levels "No","Sí": 1 1 1 1 1 1 1 1 1 1 ...
## - attr(*, "variable.labels")= Named chr [1:10] "Código de empleado" "Sexo" "Fecha de nacimiento" "Nivel educativo" ...
## .. attr(*, "names")= chr [1:10] "id" "sexo" "fechnac" "educ" ...
## - attr(*, "codepage")= int 1252
```

Nota: Si hay fechas, puede ser recomendable emplear la función `spss.get()` del paquete `Hmisc`.

2.1.5 Importación desde Excel

Se pueden leer ficheros de Excel (con extensión `.xlsx`) utilizando, por ejemplo, los paquetes:

- `openxlsx`,

```
library(openxlsx)
datos<-read.xlsx("./data/coches.xlsx")
class(datos)
```

```
## [1] "data.frame"
```

- `RODBC` (este paquete se empleará más adelante para acceder a bases de datos), entre otros.

El siguiente código implementa una función que permite leer todos los archivos en formato `.xlsx` en un directorio:

```
library(openxlsx)
read_xlsx <- function(path = '.') {
  files <- dir(path, pattern = '*.xlsx') # list.files
  # file.list <- lapply(files, readWorkbook)
  file.list <- vector(length(files), mode = 'list')
  for (i in seq_along(files))
    file.list[[i]] <- readWorkbook(files[i])
  file.names <- sub('\\.xlsx$', '', basename(files))
  names(file.list) <- file.names
  file.list
}
```

Para combinar los archivos, suponiendo que tienen las mismas columnas, podríamos ejecutar una llamada a `rbind()` (R base):

```
df <- do.call('rbind', file.list)
```

o emplear la función `bind_rows()` del paquete `dplyr`, donde las columnas se emparejan por nombre, y cualquier columna que falte se rellenará con `NA`:

```
df <- dplyr::bind_rows(file.list)
```

El Capítulo 5 provee de otras utilidades para la manipulación de datos con `dplyr` (Wickham et al., 2026).

Los datos cargados en R (usualmente un `data.frame`) se pueden exportar desde Excel fácilmente a un archivo de texto *separado por comas* (extensión `.csv`), evitando utilizar algunos de los paquetes mencionados anteriormente. Por ejemplo, supongamos que queremos leer el fichero `coches.xls`:

- Desde Excel, se selecciona el menú **Archivo -> Guardar como -> Guardar como**, y en **Tipo**, se escoge la opción de archivo CSV. De esta forma se guardarán los datos en el archivo `coches.csv`.
- El fichero `coches.csv` es un fichero de texto plano (se puede editar con el bloc de notas, *Notepad*), con cabecera, las columnas separadas por “;”, y siendo “,” el carácter decimal.
- Por lo tanto, la lectura de este fichero se puede hacer con:

```
datos <- read.table("coches.csv", header = TRUE,
                    sep = ";", dec = ",")
```

Otra posibilidad, es utilizar la función `read.csv2`. Esta función no es más que una adaptación de la función general `read.table` con las siguientes opciones:

```
read.csv2(file, header = TRUE, sep = ";", dec = ",", ...)
```

Por lo tanto, la lectura del fichero `coches.csv` se puede hacer de modo más directo con:

```
datos <- read.csv2("coches.csv")
```

Esta forma de proceder, exportando a formato CSV, se puede emplear con otras hojas de cálculo o fuentes de datos. Hay que tener en cuenta que si estas fuentes emplean el formato anglosajón, el separador de campos será `sep = ","` y el de decimales `dec = "."`, las opciones por defecto en la función `read.csv()`.

2.1.6 Exportación de datos

Puede ser de interés la exportación de datos para que puedan ser leídos con otros programas. Para ello, se puede emplear la función `write.table()`. Esta función es similar, pero funcionando en sentido inverso, a `read.table()`, ver Sección 2.1.3.

Veamos un ejemplo:

```
tipo <- c("A", "B", "C")
longitud <- c(120.34, 99.45, 115.67)
datos <- data.frame(tipo, longitud)
datos
```

```
##   tipo longitud
## 1    A   120.34
## 2    B    99.45
## 3    C   115.67
```

Para guardar el `data.frame` `datos` en un fichero de texto se puede utilizar:

```
write.table(datos, file = "datos.txt")
```

Otra posibilidad es utilizar la función:

```
write.csv2(datos, file = "datos.csv")
```

que dará lugar al fichero `datos.csv` importable directamente desde Excel. Las opciones anteriores sólo dependen del paquete `utils`, que se instala por defecto con R base.

2.1.7 Python, Julia y otros lenguajes de programación

R es un lenguaje de programación libre (derivado del lenguaje S en los Laboratorios Bell) que se caracteriza por su capacidad para interactuar con otros lenguajes de programación, incluyendo Python (Van Rossum and Drake Jr, 1995) y Julia (Bezanson et al., 2017).

En el ámbito de la Estadística (como en la denominada **Ciencia de Datos**), R destaca por su extensa y detallada documentación (en muchos casos como resultado de aportaciones metodológicas y/o avances científicos). Por ejemplo, después de diez años de la primera edición del libro *An Introduction to Statistical Learning con aplicaciones en R (ISLR)*, James et al. (2013), algunos de los mismos autores publicaron la edición en Python (ISLP), James et al. (2023).

Las últimas ediciones de ambos libros están disponibles en abierto en <https://www.statlearning.com>. Por otro lado, en 2015, se lanzó el paquete **reticulate** disponible en <https://rstudio.github.io/reticulate/>, permitiendo la ejecución de código Python desde R (y en 2020 se completó la integración de Python en la interfaz de RStudio).

```
library(reticulate)
os <- import("os")
os$listdir(".")
```

Si queremos trabajar con Python de forma interactiva, podemos usar `repl_python()`. Los objetos creados en Python se pueden usar en R con `py` de **reticulate**.

Recientemente, *Julia* se presenta también como una alternativa a considerar. El paquete **JuliaConnector** disponible en <https://cran.r-project.org/web/packages/JuliaConnector/> facilita la importación de funciones y paquetes completos de Julia a R, es decir, permite el uso de funciones de Julia directamente en R.

R también permite el uso/comunicación de otros lenguajes de programación como Java, C, C++, Fortran, entre otros.

En julio de 2022, la compañía *RStudio Inc* anunció que cambiaba su nombre a *Posit*, para reflejar su interés en el desarrollo de herramientas para lenguajes adicionales de programación, principalmente Python. En esa misma fecha lanzó Quarto como una nueva alternativa a R Markdown para ofrecer compatibilidad integrada para múltiples lenguajes, como Python o Julia, además de R. Desde julio de 2025 ya están disponibles versiones estables de Positron, un nuevo IDE para el análisis de datos que es compatible de forma nativa con otros lenguajes, como Python o Julia, y que se supone que será un sustituto de RStudio en el futuro.

2.2 Manipulación de datos

Una vez cargada una (o varias) bases de datos hay una serie de operaciones que serán de interés para el tratamiento de datos:

- Operaciones con variables:
 - crear
 - recodificar (e.g. categorizar)
 - ...
- Operaciones con casos:
 - ordenar
 - filtrar
 - ...

- Operaciones con tablas de datos:
 - unir
 - combinar
 - consultar
 - ...

A continuación se tratan algunas operaciones *básicas*.

2.2.1 Operaciones con variables

2.2.1.1 Creación (y eliminación) de variables

Consideremos de nuevo la base de datos `cars` incluida en el paquete `datasets`:

```
data(cars)
# str(cars)
head(cars)
```

```
##   speed dist
## 1     4    2
## 2     4   10
## 3     7    4
## 4     7   22
## 5     8   16
## 6     9   10
```

Utilizando el comando `help(cars)` se obtiene que `cars` es un `data.frame` con 50 observaciones y dos variables:

- `speed`: Velocidad (en millas por hora)
- `dist`: tiempo hasta detenerse (en pies)

Recordemos que, para acceder a la variable `speed` se puede hacer directamente con su nombre o bien utilizando notación “matricial” (se seleccionan las 6 primeras observaciones por comodidad).

```
cars$speed
```

```
## [1]  4  4  7  7  8  9 10 10 10 11 11 12 12 12 12 13 13 13
## [19] 13 14 14 14 14 15 15 15 16 16 17 17 17 18 18 18 18 19
## [37] 19 19 20 20 20 20 20 22 23 24 24 24 24 25
```

```
# cars[, 1]      # Equivalente
# cars[, "speed"] # Equivalente
```

Supongamos ahora que queremos transformar la variable original `speed` (millas por hora) en una nueva variable `velocidad` (kilómetros por hora) y añadir esta nueva variable al `data.frame` `cars`. La transformación que permite pasar millas a kilómetros es $\text{kilómetros} = \text{millas} / 0.62137$ que en R se hace directamente con:

```
(cars$speed / 0.62137)[1:10]
```

Finalmente, incluimos la nueva variable que llamaremos `velocidad` en `cars`:

```
cars$velocidad <- cars$speed / 0.62137
head(cars)
```

```
##   speed dist velocidad
```

```
## 1    4    2  6.437388
## 2    4   10  6.437388
## 3    7    4 11.265430
## 4    7   22 11.265430
## 5    8   16 12.874777
## 6    9   10 14.484124
```

También transformaremos la variable `dist` (en pies) en una nueva variable `distancia` (en metros), por lo que la transformación deseada es `metros=pies/3.2808`:

```
cars$distancia <- cars$dist / 3.2808
head(cars)
```

```
##   speed dist velocidad distancia
## 1     4    2  6.437388  0.6096074
## 2     4   10  6.437388  3.0480371
## 3     7    4 11.265430  1.2192148
## 4     7   22 11.265430  6.7056815
## 5     8   16 12.874777  4.8768593
## 6     9   10 14.484124  3.0480371
```

Ahora, eliminaremos las variables originales `speed` y `dist`, y guardaremos el `data.frame` resultante con el nombre `coches`. En primer lugar, veamos varias formas de acceder a las variables de interés:

```
cars[, c(3, 4)]
cars[, c("velocidad", "distancia")]
cars[, -c(1, 2)]
```

Utilizando alguna de las opciones anteriores se obtiene el `data.frame` deseado:

```
coches <- cars[, c("velocidad", "distancia")]
# head(coches)
str(coches)
```

```
## 'data.frame':   50 obs. of  2 variables:
## $ velocidad: num  6.44 6.44 11.27 11.27 12.87 ...
## $ distancia: num  0.61 3.05 1.22 6.71 4.88 ...
```

Finalmente, los datos anteriores podrían ser guardados en un fichero exportable a Excel con el siguiente comando:

```
write.csv2(coches, file = "coches.csv")
```

2.2.1.2 Recodificación de variables

Con el comando `cut()` podemos crear variables categóricas a partir de variables numéricas. El parámetro `breaks` permite especificar los intervalos para la discretización, puede ser un vector con los extremos de los intervalos o un entero con el número de intervalos. Por ejemplo, para categorizar la variable `cars$speed` en tres intervalos equidistantes podemos emplear¹:

```
fspeed <- cut(cars$speed, 3, labels = c("Baja", "Media", "Alta"))
table(fspeed)
```

```
## fspeed
##  Baja Media  Alta
##    11    24    15
```

¹ Aunque si el objetivo es obtener las frecuencias de cada intervalo puede ser más eficiente emplear `hist()` con `plot = FALSE`.

Para categorizar esta variable en tres niveles con aproximadamente el mismo número de observaciones podríamos combinar esta función con `quantile()`:

```
breaks <- quantile(cars$speed, probs = 0:3/3)
etiquetas3 <- c("Baja", "Media", "Alta")
fspeed <- cut(cars$speed, breaks, labels = etiquetas3)
table(fspeed)
```

```
## fspeed
##  Baja Media  Alta
##    17    16    15
```

Para otro tipo de recodificaciones podríamos emplear la función `ifelse()` vectorial:

```
fspeed <- ifelse(cars$speed < 15, "Baja", "Alta")
etiquetas2 <- c("Baja", "Alta")
fspeed <- factor(fspeed, levels = etiquetas2)
table(fspeed)
```

```
## fspeed
##  Baja Alta
##    23   27
```

Alternativamente, en el caso de dos niveles podríamos emplear directamente la función `factor()`:

```
fspeed <- factor(cars$speed >= 15,
                 labels = etiquetas2) # levels = c("FALSE", "TRUE")
table(fspeed)
```

```
## fspeed
##  Baja Alta
##    23   27
```

En el caso de múltiples niveles, se podría emplear `ifelse()` anidados:

```
fspeed <- ifelse(cars$speed < 10, "Baja",
                 ifelse(cars$speed < 20, "Media", "Alta"))
fspeed <- factor(fspeed, levels = etiquetas3)
table(fspeed)
```

```
## fspeed
##  Baja Media  Alta
##     6    32    12
```

Otra alternativa, sería emplear la función `recode()` del paquete `car`.

```
library(car)
fspeed <- recode(cars$speed, "0:10 = 'Baja';
                          10:20 = 'Media';
                          else='Alta'
                          ")
fspeed <- factor(fspeed, levels = c("Baja", "Media", "Alta"))
```

NOTA: Para acceder directamente a las variables de un `data.frame` podríamos emplear la función `attach()` para añadirlo a la ruta de búsqueda y `detach()` al finalizar. Sin embargo esta forma de proceder puede causar numerosos inconvenientes, especialmente al modificar la base de datos, por lo que la recomendación sería emplear `with()`. Por ejemplo, podríamos calcular el factor anterior empleando:

```
fspeed <- with(cars, ifelse(speed < 10, "Baja",
                           ifelse(speed < 20, "Media", "Alta")))
fspeed <- factor(fspeed, levels = c("Baja", "Media", "Alta"))
table(fspeed)
```

```
## fspeed
##  Baja Media  Alta
##      6    32   12
```

2.2.2 Operaciones con casos

2.2.2.1 Ordenación

Continuemos con el data.frame `cars`. Se puede comprobar que los datos disponibles están ordenados por los valores de `speed`. A continuación haremos la ordenación utilizando los valores de `dist`. Para ello, utilizaremos el conocido como vector de índices de ordenación. Este vector establece el orden en que tienen que ser elegidos los elementos para obtener la ordenación deseada. Veamos primero un ejemplo sencillo:

```
x <- c(2.5, 4.3, 1.2, 3.1, 5.0) # valores originales
ii <- order(x)
ii # vector de ordenación
```

```
## [1] 3 1 4 2 5
```

```
x[ii] # valores ordenados (por defecto, ascendentemente)
```

```
## [1] 1.2 2.5 3.1 4.3 5.0
```

En el caso de vectores, el procedimiento anterior se podría hacer directamente con:

```
sort(x)
```

Sin embargo, para ordenar tablas de datos será necesario la utilización del vector de índices de ordenación. A continuación, se muestran los datos de `cars` ordenados por `dist`:

```
ii <- order(cars$dist) # Vector de índices de ordenación
cars2 <- cars[ii, ]    # Datos ordenados por dist
head(cars2)
```

```
##   speed dist velocidad distancia
## 1     4    2  6.437388  0.6096074
## 3     7    4 11.265430  1.2192148
## 2     4   10  6.437388  3.0480371
## 6     9   10 14.484124  3.0480371
## 12    12   14 19.312165  4.2672519
## 5     8   16 12.874777  4.8768593
```

2.2.2.2 Filtrado

El filtrado de datos consiste en elegir una submuestra que cumpla determinadas condiciones. Para ello, se puede utilizar la función `subset(x, subset, select, drop = FALSE, ...)`, que además permite seleccionar variables con el argumento `select`.

A continuación se muestran un par de ejemplos:

```
# datos con dis>85
subset(cars, dist > 85)
```

```
##      speed dist velocidad distancia
## 47      24   92  38.62433  28.04194
## 48      24   93  38.62433  28.34674
## 49      24  120  38.62433  36.57644
```

```
# datos con speed en (10,15) y dist > 45
subset(cars, speed > 10 & speed < 15 & dist > 45)
```

```
##      speed dist velocidad distancia
## 19      13   46  20.92151  14.02097
## 22      14   60  22.53086  18.28822
## 23      14   80  22.53086  24.38430
```

También se pueden hacer el filtrado empleando directamente los correspondientes vectores de índices:

```
ii <- cars$dist > 85
cars[ii, ] # dis>85
```

```
##      speed dist velocidad distancia
## 47      24   92  38.62433  28.04194
## 48      24   93  38.62433  28.34674
## 49      24  120  38.62433  36.57644
```

```
ii <- cars$speed > 10 & cars$speed < 15 & cars$dist > 45
cars[ii, ] # speed en (10,15) y dist>45
```

```
##      speed dist velocidad distancia
## 19      13   46  20.92151  14.02097
## 22      14   60  22.53086  18.28822
## 23      14   80  22.53086  24.38430
```

En este caso, puede ser de utilidad la función `which()`:

```
it <- which(ii)
str(it)
```

```
## int [1:3] 19 22 23
```

```
cars[it, ]
```

```
##      speed dist velocidad distancia
## 19      13   46  20.92151  14.02097
## 22      14   60  22.53086  18.28822
## 23      14   80  22.53086  24.38430
```

```
# rownames(cars[it, ])
id <- which(!ii)
str(cars[id, ])
```

```
## 'data.frame': 47 obs. of 4 variables:
## $ speed : num 4 4 7 7 8 9 10 10 10 11 ...
## $ dist : num 2 10 4 22 16 10 18 26 34 17 ...
## $ velocidad: num 6.44 6.44 11.27 11.27 12.87 ...
## $ distancia: num 0.61 3.05 1.22 6.71 4.88 ...
```

```
# Equivalentemente:
str(cars[-it, ])
```

```
## 'data.frame': 47 obs. of 4 variables:
## $ speed : num 4 4 7 7 8 9 10 10 10 11 ...
## $ dist : num 2 10 4 22 16 10 18 26 34 17 ...
```

```
## $ velocidad: num  6.44 6.44 11.27 11.27 12.87 ...
## $ distancia: num  0.61 3.05 1.22 6.71 4.88 ...
```

```
# ?which.min
```

Si se realiza una selección de variables como en:

```
cars[ii, "speed"]
```

```
## [1] 13 14 14
```

es posible que se quiera mantener la estructura original de los datos, para ello, bastaría con:

```
cars[ii, "speed", drop=FALSE]
```

```
##      speed
## 19      13
## 22      14
## 23      14
```

```
# subset(cars, ii, "speed") # equivalente
```

A veces puede ser necesario dividir (particionar) el conjunto de datos, uno para cada nivel de un grupo (factor), para ello se puede usar la función `split()`:

```
speed2 <- factor(cars$speed > 20, labels = c("slow", "fast"))
table(speed2)
```

```
## speed2
## slow fast
##   43    7
```

```
cars2 <- split(cars, speed2)
class(cars2) # lista con 2 data.frames
```

```
## [1] "list"
```

```
sapply(cars2, class)
```

```
##           slow           fast
## "data.frame" "data.frame"
```

```
sapply(cars2, dim)
```

```
##           slow fast
## [1,]    43    7
## [2,]     4    4
```

```
cars2$fast
```

```
##      speed dist velocidad distancia
## 44      22   66  35.40564  20.11704
## 45      23   54  37.01498  16.45940
## 46      24   70  38.62433  21.33626
## 47      24   92  38.62433  28.04194
## 48      24   93  38.62433  28.34674
## 49      24  120  38.62433  36.57644
## 50      25   85  40.23368  25.90832
```

De forma inversa, podríamos recuperar el data.frame original con:

```
unsplit(cars2,speed2)
```

2.2.3 Datos faltantes

La problemática originada por los datos faltantes (*missing data*) en cualquier conjunto de datos subyace cuando se desea realizar un análisis estadístico, para más información en R, se puede consultar CRAN Task View: Missing Data

Vamos a ver un ejemplo, empleando el conjunto de datos `airquality` que contiene datos faltantes en sus dos primeras variables:

```
data("airquality")
datos <- airquality[,1:3]
summary(datos)
```

```
##      Ozone      Solar.R      Wind
##  Min.   : 1.00   Min.    : 7.0   Min.    : 1.700
## 1st Qu.: 18.00   1st Qu.:115.8   1st Qu.: 7.400
## Median : 31.50   Median :205.0   Median : 9.700
## Mean   : 42.13   Mean    :185.9   Mean    : 9.958
## 3rd Qu.: 63.25   3rd Qu.:258.8   3rd Qu.:11.500
## Max.   :168.00   Max.    :334.0   Max.    :20.700
## NAs    :37      NAs     :7
```

```
nrow(datos)
```

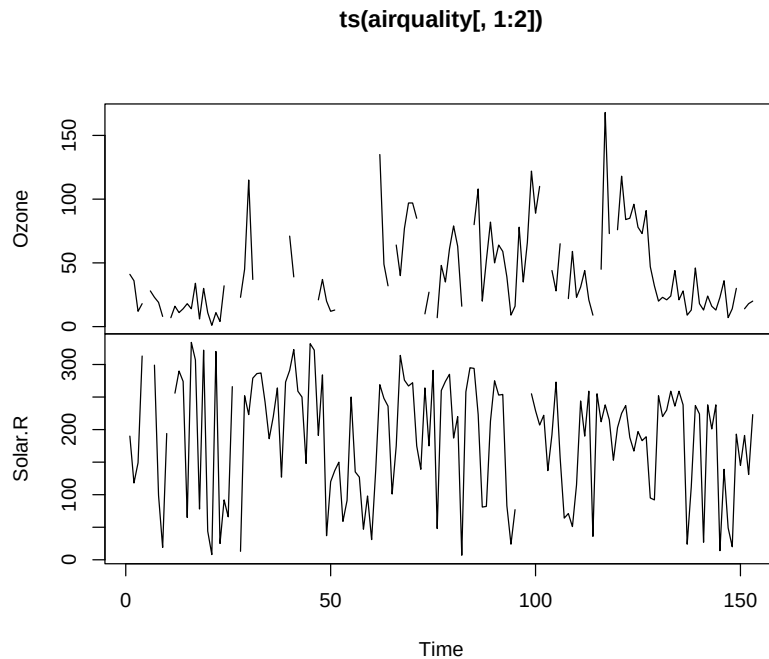
```
## [1] 153
```

```
# Datos faltantes por variable
sapply(datos, function(x) sum(is.na(x)))
```

```
##      Ozone Solar.R      Wind
##       37       7        0
```

A continuación se muestra la distribución de los datos perdidos en el data.frame (a lo largo del tiempo, por mes):

```
plot(ts(airquality[,1:2]))
```



¿Existe un patrón no aleatorio en los datos faltantes del ozono? Esta pregunta puede ser abordada parcialmente utilizando el test de Little (Little, 1988), disponible en la función `mcar_test()` del paquete `nanianr`. Este test permite evaluar si los datos faltantes son generados por un mecanismo completamente aleatorio (MCAR). Si la hipótesis de MCAR es rechazada, esto sugiere que los datos faltantes podrían estar siguiendo un mecanismo MAR (*missing at random*) o MNAR (*non missing at random*).

Sin embargo, en muchos estudios, se omite el paso anterior y se procede directamente con alguno de los siguientes métodos:

- Análisis de casos completos (*complete cases*)
- Análisis de casos disponibles (borrado por parejas *pairwise cases*)
- Imputación de datos faltantes (por la media, mediana, último valor observado, vecino más cercano, valores predichos usando los datos observados....)

Siguiendo con el ejemplo, ante la presencia de datos faltantes, en R inicialmente no podemos conocer cómo se relacionan las tres primeras variables:

```
cor(datos[,1:3])
```

```
##           Ozone Solar.R Wind
## Ozone      1      NA  NA
## Solar.R    NA      1  NA
## Wind       NA     NA   1
```

y requiere indicar cómo tratar los datos perdidos. Por ejemplo, una opción sería realizar un análisis sólo de los casos completos, eliminando todas las observaciones (filas) con algún dato faltante de nuestro conjunto de datos:

```
datosC <- na.omit(datos)
nrow(datosC) # n fija (sólo se utilizan 111 de las 153 de Wind)
```

```
## [1] 111
```

```
cor(datosC[,1:3])
```

```
##           Ozone      Solar.R      Wind
## Ozone      1.0000000  0.3483417 -0.6124966
## Solar.R    0.3483417  1.0000000 -0.1271835
## Wind      -0.6124966 -0.1271835  1.0000000
```

```
# otra forma de hacerlo sería:
# nrow(datos[complete.cases(datos),])
# cor(datos[,1:3], use = "complete.obs")
```

También, se podría usar toda la información disponible. El tamaño muestral n sería variable en función de los NA's de cada par de variables:

```
cor(datos[,1:3], use = "pairwise.complete.obs")
```

```
##           Ozone      Solar.R      Wind
## Ozone      1.0000000  0.34834169 -0.60154653
## Solar.R    0.3483417  1.00000000 -0.05679167
## Wind      -0.6015465 -0.05679167  1.00000000
```

Por ejemplo, ahora la correlación usa los 146 pares de observaciones disponibles para (Solar.R,Wind), en lugar de 111 del primer caso.

Por último, también se podría realizar una imputación (Van Buuren, 2018). A modo de ejemplo, en el siguiente código, se utiliza la media:

```
datosI <- datos
datosI$Ozone[is.na(datos$Ozone)] <- mean(datos$Ozone, na.rm = T)
datosI$Solar.R[is.na(datos$Solar.R)] <- mean(datosI$Solar.R, na.rm = T)
cor(datosI[,1:3])
```

```
##           Ozone      Solar.R      Wind
## Ozone      1.0000000  0.30296951 -0.53093584
## Solar.R    0.3029695  1.00000000 -0.05524488
## Wind      -0.5309358 -0.05524488  1.00000000
```

Notar que para el caso del ozono, se han sustituido los 37 NA's (24% de las observaciones) por un único valor (de ahí que ahora la varianza sea menor a la observada inicialmente, algo que en principio, no sería deseable).

```
var(datos$Ozone, na.rm = T)
```

```
## [1] 1088.201
```

```
var(datosI$Ozone)
```

```
## [1] 823.3096
```

Los datos faltantes son una realidad común en muchos estudios, aunque nadie los desea. Para tratarlos correctamente, es esencial comprender cómo se obtuvieron los datos observados y por qué algunos datos no fueron registrados antes de iniciar cualquier otro análisis. No abordar adecuadamente los datos faltantes puede tener un efecto perjudicial en nuestro estudio, ya que las conclusiones obtenidas podrían ser no representativas o contener sesgos.

2.2.4 Funciones `apply`

2.2.4.1 La función `apply`

Una forma de evitar la utilización de bucles es utilizando la sentencia `apply` que permite evaluar una misma función en todas las filas, columnas, etc. de un array de forma simultánea.

La sintaxis de esta función es:

```
apply(X, MARGIN, FUN, ...)
```

- **X**: matriz (o array).
- **MARGIN**: un vector indicando las dimensiones donde se aplicará la función. 1 indica filas, 2 indica columnas, y `c(1,2)` indica filas y columnas.
- **FUN**: función que será aplicada.
- **...**: argumentos opcionales que serán usados por FUN.

Veamos la utilización de la función `apply` con un ejemplo:

```
x <- matrix(1:9, nrow = 3)
x

##      [,1] [,2] [,3]
## [1,]    1    4    7
## [2,]    2    5    8
## [3,]    3    6    9

apply(x, 1, sum)      # Suma por filas

## [1] 12 15 18

apply(x, 2, sum)      # Suma por columnas

## [1]  6 15 24

apply(x, 2, min)      # Mínimo de las columnas

## [1]  1  4  7

apply(x, 2, range)    # Rango (mínimo y máximo) de las columnas

##      [,1] [,2] [,3]
## [1,]    1    4    7
## [2,]    3    6    9
```

Alternativamente, se puede utilizar opciones más eficientes: `colSums()`, `rowSums()`, `colMeans()` y `rowMeans()`, como se muestra en el siguiente código de ejemplo:

```
x <- matrix(1:1e8, ncol = 10, byrow = FALSE)
t1 <- proc.time()
out<-apply(x, 2, mean)
proc.time() - t1

##      user  system elapsed
##      1.67    0.14    2.00

t2 <- proc.time()
out <- colMeans(x)
proc.time() - t2

##      user  system elapsed
```

```
##      0.12      0.02      0.16
```

2.2.4.2 Variantes de la función apply

- a. La función `lapply(X, FUN, ...)` aplica la función `FUN` a cada elemento de una lista en R y devuelve una lista como resultado (sin necesidad de especificar el argumento `MARGIN`). Notar que todas las estructuras de datos en R pueden convertirse en listas, por lo que `lapply()` puede utilizarse en más casos que `apply()`.

```
# lista con las medianas de las variables
list <- lapply(cars, median)
str(list)
```

```
## List of 4
## $ speed      : num 15
## $ dist       : num 36
## $ velocidad: num 24.1
## $ distancia: num 11
```

- b. La función `sapply(X, FUN, ..., simplify = TRUE, USE.NAMES = TRUE)` permite iterar sobre una lista o vector (alternativa más eficiente a un `for`):

```
# matriz con las medias, medianas y desv. de las variables
res <- sapply(cars,
              function(x) c(mean = mean(x),
                           median = median(x),
                           sd = sd(x)))

# str(res)
res
```

```
##           speed      dist velocidad distancia
## mean    15.400000 42.98000 24.783945 13.100463
## median   15.000000 36.00000 24.140206 10.972933
## sd        5.287644 25.76938  8.509655  7.854602
```

```
cfuns <- function(x, funs = c(mean, median, sd))
  sapply(funs, function(f) f(x))
x <- 1:10
cfuns(x)
```

```
## [1] 5.50000 5.50000 3.02765
```

```
sapply(cars, cfuns)
```

```
##           speed      dist velocidad distancia
## [1,] 15.400000 42.98000 24.783945 13.100463
## [2,] 15.000000 36.00000 24.140206 10.972933
## [3,]  5.287644 25.76938  8.509655  7.854602
```

```
nfuns <- c("mean", "median", "sd")
sapply(nfuns,
      function(f) eval(parse(text = paste0(f, "(x)"))))
```

```
##      mean  median      sd
## 5.50000 5.50000 3.02765
```

- c. La función `tapply()` es similar a la función `apply()` y permite aplicar una función a los datos desagregados, utilizando como criterio los distintos niveles de una variable factor. Es decir, facilita la creación de tablas resumen por grupos. La sintaxis de esta función es como sigue:

```
tapply(X, INDEX, FUN, ...,)
```

- X: matriz (o array).
- INDEX: factor indicando los grupos (niveles).
- FUN: función que será aplicada.
- ...: argumentos opcionales .

Consideremos, por ejemplo, el data.frame `ChickWeight` con datos de un experimento relacionado con la repercusión de varias dietas en el peso de pollos.

```
data(ChickWeight)
# str(ChickWeight)
head(ChickWeight)
```

```
## Grouped Data: weight ~ Time | Chick
##   weight Time Chick Diet
## 1    42    0     1    1
## 2    51    2     1    1
## 3    59    4     1    1
## 4    64    6     1    1
## 5    76    8     1    1
## 6    93   10     1    1
```

```
peso <- ChickWeight$weight
dieta <- ChickWeight$Diet
levels(dieta) <- c("Dieta 1", "Dieta 2", "Dieta 3", "Dieta 4")
tapply(peso, dieta, mean) # Peso medio por dieta
```

```
## Dieta 1 Dieta 2 Dieta 3 Dieta 4
## 102.6455 122.6167 142.9500 135.2627
```

```
tapply(peso, dieta, summary)
```

```
## $`Dieta 1`
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  35.00  57.75   88.00  102.65  136.50  305.00
##
## $`Dieta 2`
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   39.0   65.5   104.5   122.6   163.0   331.0
##
## $`Dieta 3`
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   39.0   67.5   125.5   142.9   198.8   373.0
##
## $`Dieta 4`
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  39.00  71.25  129.50  135.26  184.75  322.00
```

Alternativamente, se podría emplear la función `aggregate()` que tiene las ventajas de admitir fórmulas y disponer de un método para series de tiempo.

```
# help(aggregate)
aggregate(peso, by = list(dieta = dieta), FUN = "mean" )
```

```
##      dieta      x
## 1 Dieta 1 102.6455
## 2 Dieta 2 122.6167
```

```
## 3 Dieta 3 142.9500
## 4 Dieta 4 135.2627
```

```
aggregate(peso ~ dieta, FUN = "summary" ) # con formula
```

```
##      dieta peso.Min. peso.1st Qu. peso.Median peso.Mean
## 1 Dieta 1   35.0000   57.7500    88.0000  102.6455
## 2 Dieta 2   39.0000   65.5000   104.5000  122.6167
## 3 Dieta 3   39.0000   67.5000   125.5000  142.9500
## 4 Dieta 4   39.0000   71.2500   129.5000  135.2627
##      peso.3rd Qu. peso.Max.
## 1    136.5000  305.0000
## 2    163.0000  331.0000
## 3    198.7500  373.0000
## 4    184.7500  322.0000
```

2.2.5 Generación de tablas

Hay muchos paquetes de R que se pueden utilizar para generar tablas en informes RMarkdown o en aplicaciones shiny. Entre las herramientas disponibles podríamos destacar la función `kable()` del paquete `knitr` para generar tablas básicas, y la función `datatable()` del paquete `DT` para generar tablas dinámicas. Otros paquetes son: `kableExtra`, `flextable`, `reactable`, `reactablefmtr`, `formattable`, `gt` y `tinytable`.

2.2.5.1 Tablas con `kable()`

A continuación, se muestra un ejemplo, de tabla resumen, con las medias, medianas y desviación típica de las variables:

```
res <- sapply(cars,
              function(x) c(mean = mean(x),
                           median = median(x),
                           sd = sd(x)))
knitr::kable(t(res), digits = 1,
             col.names = c("Media", "Mediana", "Desv. típica"))
```

	Media	Mediana	Desv. típica
speed	15.4	15.0	5.3
dist	43.0	36.0	25.8
velocidad	24.8	24.1	8.5
distancia	13.1	11.0	7.9

Y en este segundo ejemplo, se muestra el resumen de un modelo de regresión lineal simple (distancia de frenado en función de la velocidad del vehículo):

```
modelo <- lm(dist ~ speed, data = cars)
coefs <- coef(summary(modelo))
knitr::kable(coefs, escape = FALSE, digits = 5)
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-17.57909	6.75844	-2.60106	0.01232
speed	3.93241	0.41551	9.46399	0.00000

2.2.5.2 Tablas interactivas con `datatable()`

```
library(DT)
datatable(iris,options = list(scrollX = TRUE))
```

2.2.6 Operaciones con tablas de datos

2.2.6.1 Unir tablas:

- `rbind()`: combina vectores, matrices, arrays o data.frames por filas.
- `cbind()`: Idem por columnas.
- `merge()`: Fusiona dos data.frame por columnas o nombres de fila comunes. También permite otras operaciones de unión (*join*) de bases de datos, algunas de ellas se verán con más detalle en el Capítulo 5.

2.2.6.2 Combinar tablas:

- `match(x, table)` devuelve un vector (de la misma longitud que `x`) con las (primeras) posiciones de coincidencia de `x` en `table` (o NA, por defecto, si no hay coincidencia).

Para realizar consultas combinando tablas puede ser más cómodo el operador `%in% (?'%in%')`.

- `pmatch(x, table, ...)`: similar al anterior pero con coincidencias parciales de cadenas de texto.

2.3 Ejemplo WoS data

Ejemplo *wosdata.R* en *wosdata.zip*. Ver paquete *scimetr*.

Empleando la función `ImportSources.wos()` se importaron ficheros de texto descargados de WoS (que por defecto tienen una limitación de 500 registros). Posteriormente se creo una base de datos como una lista de tablas con la función `CreateDB.wos()`, que finalmente se almacenó en el fichero *db_udc_2015.rds*.

```
db <- readRDS("data/wosdata/db_udc_2015.rds")
str(db, 1)
```

```
## List of 11
## $ Docs      : 'data.frame':  856 obs. of  26 variables:
## $ Authors   : 'data.frame': 4051 obs. of  4 variables:
## $ AutDoc    : 'data.frame': 5511 obs. of  2 variables:
## $ Categories: 'data.frame':  189 obs. of  2 variables:
## $ CatDoc    : 'data.frame': 1495 obs. of  2 variables:
## $ Areas     : 'data.frame':  121 obs. of  2 variables:
## $ AreaDoc   : 'data.frame': 1364 obs. of  2 variables:
## $ Addresses : 'data.frame': 3655 obs. of  5 variables:
## $ AddAutDoc : 'data.frame': 7751 obs. of  3 variables:
## $ Journals  : 'data.frame':  520 obs. of 12 variables:
## $ label     : chr ""
## - attr(*, "variable.labels")= Named chr [1:62] "Publication type" "Author" "Book authors" "Editor" ...
## ..- attr(*, "names")= chr [1:62] "PT" "AU" "BA" "BE" ...
## - attr(*, "class")= chr "wos.db"
```

Puede ser recomendable añadir a los datos un atributo `variable.labels` que contenga un vector de etiquetas de las variables y empleando como nombres de las componentes las propias variables:

Tabla 2.1: Variable labels

	variable.labels
PT	Publication type
AU	Author
BA	Book authors
BE	Editor
GP	Group author
AF	Author full

```
variable.labels <- attr(db, "variable.labels")
knitr::kable(head(as.data.frame(variable.labels)),
  caption = "Variable labels")
```

Las tablas de datos con este atributo son compatibles con RStudio. Por ejemplo, también se mostrarán las etiquetas al abrirla con `View()`

```
Docs <- db$Docs # No copia los datos (crea otro objeto que apunta a los mismos datos)
attr(Docs, "variable.labels") <- variable.labels[names(Docs)]
# View(Docs)
```

Para combinar tablas podemos emplear `match(x, table)`. Por ejemplo, el siguiente código permite añadir el nombre de la revista a la tabla de documentos, combinándola con la de revistas:

```
str(Docs)
```

```
## 'data.frame': 856 obs. of 26 variables:
## $ idd: int 1 2 3 4 5 6 7 8 9 10 ...
## $ idj: int 260 37 86 272 64 429 14 408 333 214 ...
## $ TI : chr "Fractionation of Miscanthus x giganteus via modification of the Formacell process" "Role of
Membered Macrocyclic Platform" ...
## $ PT : Factor w/ 2 levels "Journal","Series": 1 1 1 1 1 1 1 1 1 1 ...
## $ DT : Factor w/ 9 levels "Article","Book Review",...: 1 1 1 1 5 1 6 9 1 1 ...
## $ NR : int 40 45 35 78 2 59 0 53 20 50 ...
## $ TC : int 1 5 2 2 1 0 0 10 0 1 ...
## $ Z9 : int 1 5 2 2 1 0 0 10 0 1 ...
## $ U1 : int 5 0 10 4 0 1 0 26 0 3 ...
## $ U2 : int 6 0 61 21 2 5 0 66 2 4 ...
## $ PD : chr "DEC 23" "DEC 21" "DEC 15" "DEC 14" ...
## $ PY : int 2015 2015 2015 2015 2015 2015 2015 2015 2015 2015 ...
## $ VL : chr "77" "54" "538" "21" ...
## $ IS : chr "" "24" "" "51" ...
## $ PN : chr "" "" "" "" ...
## $ SU : chr "" "" "" "" ...
## $ SI : chr "" "" "" "" ...
## $ MA : chr "" "" "" "" ...
## $ BP : chr "275" "11680" "824" "18662" ...
## $ EP : chr "281" "11687" "833" "18670" ...
## $ AR : chr "" "" "" "" ...
## $ DI : chr "10.1016/j.indcrop.2015.08.066" "10.1021/acs.inorgchem.5b01652" "10.1016/j.scitotenv.2015.08.066" ...
## $ D2 : chr "" "" "" "" ...
## $ PG : int 7 8 10 9 2 28 3 16 4 25 ...
## $ UT : num 3.66e+11 3.67e+11 3.63e+11 3.68e+11 3.66e+11 ...
## $ an : num 3 9 4 8 3 5 8 5 3 3 ...
## - attr(*, "variable.labels")= Named chr [1:26] NA NA "Title" "Publication type" ...
## ..- attr(*, "names")= chr [1:26] NA NA "TI" "PT" ...
```

```

str(db$Journals) # View(db$Journals)

## 'data.frame':   520 obs. of  12 variables:
## $ idj: int  1 2 3 4 5 6 7 8 9 10 ...
## $ SO : chr  "TISSUE ANTIGENS" "ACTA NEUROLOGICA SCANDINAVICA" "ACTA PSYCHOLOGICA" "AFINIDAD" ...
## $ SE : chr  "" "" "" "" ...
## $ BS : chr  "" "" "" "" ...
## $ LA : chr  "English" "English" "English" "English" ...
## $ PU : chr  "WILEY-BLACKWELL" "WILEY-BLACKWELL" "ELSEVIER SCIENCE BV" "ASOC QUIMICOS" ...
## $ PI : chr  "HOBOKEN" "HOBOKEN" "AMSTERDAM" "BARCELONA" ...
## $ PA : chr  "111 RIVER ST, HOBOKEN 07030-5774, NJ USA" "111 RIVER ST, HOBOKEN 07030-5774, NJ USA" "PO BOX 211, 1000 AE AMSTERDAM, NETHERLANDS" "INST QUIMICO SARRIA, VIA AUGUSTA, 390, 08017 B
## $ SN : chr  "0001-2815" "0001-6314" "0001-6918" "0001-9704" ...
## $ EI : chr  "1399-0039" "1600-0404" "1873-6297" "" ...
## $ J9 : chr  "TISSUE ANTIGENS" "ACTA NEUROL SCAND" "ACTA PSYCHOL" "AFINIDAD" ...
## $ JI : chr  "Tissue Antigens" "Acta Neurol. Scand." "Acta Psychol." "Afinidad" ...

ii <- match(Docs$idj, db$Journals$idj)
docs2 <- Docs[, c("PY", "TI")]
docs2$Journal <- db$Journals$SO[ii]
head(docs2)

##      PY
## 1 2015
## 2 2015
## 3 2015
## 4 2015
## 5 2015
## 6 2015
##
##                                     TI
## 1                               Fractionation of Miscanthus x giganteus via modification of
## 2 Role of Temperature and Pressure on the Multisensitive Multiferroic Dicyanamide Framework [TPrA] [Mn
like Structure
## 3                               Methane and carbon dioxide emissions from constructed wetlands receiving ana
## 4                               Exceptionally Inert Lanthanide(III) PARACEST MRI Contrast Agents Based on an 18
Membered Macrocyclic Platform
## 5                               Community-
Acquired Pneumonia Requiring Hospitalization
## 6                               Low-
latency Java communication devices on RDMA-enabled networks
##                               Journal
## 1                               INDUSTRIAL CROPS AND PRODUCTS
## 2                               INORGANIC CHEMISTRY
## 3                               SCIENCE OF THE TOTAL ENVIRONMENT
## 4                               CHEMISTRY-A EUROPEAN JOURNAL
## 5                               NEW ENGLAND JOURNAL OF MEDICINE
## 6 CONCURRENCY AND COMPUTATION-PRACTICE & EXPERIENCE

```

Si solo nos interesa hacer un filtrado puede resultar más cómodo emplear el operador `%in% (?'%in%')`. Por ejemplo, podemos buscar los documentos correspondientes a revistas (que contengan "Chem" en el nombre ISO de la revista). Para ello utilizamos la función `grepl()` que busca las coincidencias de un patrón dentro de cada elemento de un vector de caracteres:

```

iidj <- with(db$Journals, idj[grepl('Chem', JI)])
db$Journals$JI[iidj]

## [1] "J. Am. Chem. Soc."

```

```
## [2] "Inorg. Chem."
## [3] "J. Chem. Phys."
## [4] "J. Chem. Thermodyn."
## [5] "J. Solid State Chem."
## [6] "Chemosphere"
## [7] "Antimicrob. Agents Chemother."
## [8] "Trac-Trends Anal. Chem."
## [9] "Eur. J. Med. Chem."
## [10] "J. Chem. Technol. Biotechnol."
## [11] "J. Antimicrob. Chemother."
## [12] "Food Chem."
## [13] "Cancer Chemother. Pharmacol."
## [14] "Int. J. Chem. Kinet."
## [15] "Chem.-Eur. J."
## [16] "J. Phys. Chem. A"
## [17] "New J. Chem."
## [18] "Chem. Commun."
## [19] "Chem. Eng. J."
## [20] "Comb. Chem. High Throughput Screen"
## [21] "Mini-Rev. Med. Chem."
## [22] "Phys. Chem. Chem. Phys."
## [23] "Org. Biomol. Chem."
## [24] "J. Chem Inf. Model."
## [25] "ACS Chem. Biol."
## [26] "Environ. Chem. Lett."
## [27] "Anal. Bioanal. Chem."
## [28] "J. Cheminformatics"
## [29] "J. Mat. Chem. B"
```

```
idd <- with(Docs, idj %in% iidj)
which(idd)
```

```
## [1] 2 4 16 23 43 69 119 126 138 175 188 190 203 208
## [15] 226 240 272 337 338 341 342 357 382 385 386 387 388 394
## [29] 411 412 428 460 483 518 525 584 600 604 605 616 620 665
## [43] 697 751 753 775 784 796 806 808 847 848
```

```
# View(Docs[idd, ])
head(Docs[idd, 1:3])
```

```
## idd idj
## 2 2 37
## 4 4 272
## 16 16 195
## 23 23 436
## 43 43 455
## 69 69 37
##
## 2
like Structure
## 4
Membered Macrocyclic Platform
## 16 Reduced susceptibility to biocides in Acinetobacter baumannii: association with resistance to antimicrobials
## 23 Two Catechol Siderophores, Acinetobactin and Amonabactin, Are Essential for Iron Acquisition in Acinetobacter baumannii
## 43
## 69
Based Magnetic Resonance Imaging Contrast Agent Responsive to Zn2+
```

Role of Temperature and Pressure on the Mu

Exceptionally Inert

Como ejemplo adicional, se buscan los documentos correspondientes a autores (que contiene "Abad" en su nombre):

```
# View(db$Authors)
iida <- with(db$Authors, ida[grepl('Abad', AF)])
db$Authors$AF[iida]

## [1] "Mato Abad, Virginia" "Abad, Maria-Jose"
## [3] "Abad Vicente, J."    "Abada, Sabah"

idd <- with(db$AutDoc, idd[ida %in% iida])
idd

## [1] 273 291 518 586

# View(Docs[idd, ])
head(Docs[idd, 1:3])

##      idd idj
## 273 273 282
## 291 291 141
## 518 518 272
## 586 586 311
##
## 273                               Classification of mild cognitive impairment and Alzheimer's Disease with machine
learning techniques using H-1 Magnetic Resonance Spectroscopy data
## 291 Identifying a population of patients suitable for the implantation of a subcutaneous defibrillator (TV-ICD) among patients implanted with a conventional transvenous device (TV-ICD)
## 518      Importance of Outer-Sphere and Aggregation Phenomena in the Relaxation Properties of Phosphoric Acid
## 586                               Enhanced thermal conductivity of rheologically non-Newtonian fluids
```

2.4 Eficiencia en R: preasignación, vectorización y desfases

R permite escribir código muy expresivo, pero no todas las formas de hacer lo mismo son igual de eficientes. A continuación se ilustran tres buenas prácticas habituales, empleando el paquete `microbenchmark` para comparar tiempos de ejecución (más fiable que una única medición con `proc.time()`, ya que repite cada alternativa varias veces).

Preasignar en lugar de crecer un vector dentro de un bucle. Cada vez que se amplía un vector elemento a elemento (`a[i] <- i` sin haber fijado antes su longitud), R puede tener que reservar memoria nueva y copiar todo el vector; si se preasigna con `numeric()` (o `vector()`), el tamaño ya es el definitivo y no hay redimensionamientos:

```
library(microbenchmark)
n <- 3e4
microbenchmark(
  crecer      = { a <- numeric(); for (i in seq_len(n)) a[i] <- i },
  preasignado = { d <- numeric(n); for (i in seq_len(n)) d[i] <- i },
  times = 10
)

## Unit: milliseconds
##      expr      min       lq      mean     median        uq
##  crecer 13.6638 14.486 15.8603 15.62375 17.4813
## preasignado 3.8006  6.188  7.1354  7.25350  8.3659
##      max neval cld
## 18.7349   10  a
```

```
## 9.3998 10 b
```

Con un `n` mucho mayor la diferencia se dispara (no se evalúa en el libro por el tiempo de cómputo que implicaría repetirlo varias veces):

```
n <- 1e8
a <- numeric()
system.time(for (i in seq_len(n)) a[i] <- i)
d <- numeric(n)
system.time(for (i in seq_len(n)) d[i] <- i)
```

Usar funciones vectorizadas en lugar de `apply()`. Funciones como `colMeans()` o `rowSums()` están implementadas internamente en C y evitan el bucle (implícito) de `apply()`:

```
m <- matrix(rnorm(1e6), nrow = 1000)
microbenchmark(
  apply_mean = apply(m, 2, mean),
  colMeans   = colMeans(m),
  apply_sum  = apply(m, 1, sum),
  rowSums    = rowSums(m),
  times = 20
)
```

```
## Unit: milliseconds
##      expr      min       lq      mean   median      uq
##  apply_mean 23.1764 25.29135 26.599060 26.50930 27.97680
##   colMeans   1.2329  1.33870  1.425380  1.43375  1.49715
##  apply_sum  14.9857 17.17095 23.269110 18.00785 18.80410
##   rowSums   1.2687  1.36790  2.016065  1.68185  2.38955
##      max neval cld
## 29.8401    20  a
##  1.6456    20  b
##125.8440    20  a
##  3.9170    20  b
```

Calcular desfases (*lag/lead*) sin bucles. Para comparar cada valor con el anterior (o el siguiente) de un vector, en R base basta con desplazar el vector y alinearlos con `cbind()`, en lugar de recorrerlo con un bucle:

```
a <- 1:11
diff(a) # diferencias consecutivas
```

```
## [1] 1 1 1 1 1 1 1 1 1 1
```

```
y <- c(NA, a[-length(a)]) # "lag": a desplazado una posición hacia atrás
x <- c(a[-1], NA)         # "lead": a desplazado una posición hacia adelante
cbind(original = a, lag = y, lead = x)
```

```
##      original lag lead
## [1,]         1  NA   2
## [2,]         2   1   3
## [3,]         3   2   4
## [4,]         4   3   5
## [5,]         5   4   6
## [6,]         6   5   7
## [7,]         7   6   8
## [8,]         8   7   9
## [9,]         9   8  10
```

```
## [10,]      10   9   11
## [11,]      11  10  NA
```

En el Capítulo 5 se retoma este mismo desfase con `dplyr::lag()/lead()` (Sección 5.6), comprobando que su sintaxis más legible no penaliza el rendimiento.

2.5 Resumen: R base

A modo de referencia rápida, la siguiente tabla recoge las tareas más habituales de manipulación de datos vistas en este capítulo. En el Capítulo 5 se retomará esta misma lista mostrando el equivalente en `dplyr/tidyr` para cada tarea (ver el resumen comparativo de la Sección 5.7):

Tarea	R base
Leer texto/CSV	<code>read.table(), read.delim()</code>
Leer Excel	<code>openxlsx::read.xlsx()</code>
Escribir texto/CSV	<code>write.table()</code>
Seleccionar columnas	<code>df[, c("a", "b")]</code>
Renombrar columnas	<code>names(df)[i] <- "x"</code>
Filtrar filas	<code>subset(df, cond), df[cond,]</code>
Ordenar filas	<code>df[order(df\$x),]</code>
Crear/transformar variable	<code>df\$z <- expr</code>
Recodificar/categorizar	<code>cut(), ifelse()</code>
Resumir	<code>aggregate(), tapply()</code>
Agrupar y resumir	<code>aggregate(y ~ g, df, FUN)</code>
Unir tablas (<i>join</i>)	<code>merge()</code>
Filtrar por coincidencia	<code>df1[df1\$x %in% df2\$x,]</code>
Encadenar operaciones	llamadas anidadas

2.6 Ejercicios

Para practicar lo visto en este capítulo se proponen los siguientes ejercicios (los enunciados se dan sin solución; las soluciones se distribuyen aparte).

```
load("data/empleados.RData")
```

2.6.1 Ejercicio 1: incremento salarial (empleados)

A partir de los datos de `empleados` (Sección 2.1), calcula para cada empleado el incremento salarial relativo respecto a su salario inicial: `incremento = salario / salini - 1`. Muestra los 5 empleados con mayor incremento porcentual, junto con su categoría laboral (`catlab`) y su antigüedad en meses (`tiempemp`).

Para los siguientes ejercicios se empleará el conjunto de datos `starwars` (incluido en el paquete `dplyr`, aunque aquí se usa únicamente como fuente de datos, sin necesidad de emplear sus funciones):

```
data(starwars, package = "dplyr")
starwars <- as.data.frame(starwars) # para evitar el formato tibble al imprimir
```

Para el ejercicio de unión de tablas se empleará además la siguiente tabla, creada manualmente, con información de algunos planetas:

```
planetas <- data.frame(
  homeworld = c("Tatooine", "Naboo", "Alderaan", "Coruscant", "Kamino",
                "Corellia", "Kashyyyk", "Ryloth", "Mirial", "Hoth"),
  region    = c("Outer Rim", "Mid Rim", "Core Worlds", "Core Worlds", "Outer Rim",
                "Core Worlds", "Mid Rim", "Outer Rim", "Outer Rim", "Outer Rim"),
  clima     = c("Desértico", "Templado", "Templado", "Urbano", "Oceánico",
                "Urbano", "Boscoso", "Húmedo", "Templado", "Helado")
)
```

2.6.2 Ejercicio 2: Filtrado y selección (starwars)

Obtén el nombre, la altura, el peso y el planeta natal (`homeworld`) de los personajes de especie (`species`) "Human" con una altura (`height`) superior a 180 cm.

2.6.3 Ejercicio 3: Creación de una variable y categorización

Calcula el índice de masa corporal ($IMC = \text{peso} / \text{altura}^2$, con la altura en metros) de cada personaje, descartando aquellos con valores faltantes en peso o altura. A continuación, categoriza el IMC empleando los umbrales estándar de la OMS: menos de 18.5 “Bajo peso”, de 18.5 a 25 “Normal”, de 25 a 30 “Sobrepeso” y 30 o más “Obesidad”. Indica cuántos personajes hay en cada categoría.

2.6.4 Ejercicio 4: Resumen y agrupación

Calcula la altura media y el número de personajes por especie, considerando únicamente las especies con al menos 3 personajes, y ordena el resultado de mayor a menor altura media.

2.6.5 Ejercicio 5: Unión de tablas

Une `starwars` con `planetas` por la columna `homeworld` y muestra, para cada personaje, su nombre, `homeworld`, región y clima. ¿Qué ocurre con los personajes cuyo planeta no aparece en `planetas` (o cuyo `homeworld` es NA)? ¿Y con "Hoth", que está en `planetas` pero no es el planeta natal de ningún personaje?

2.6.6 Ejercicio 6 (opcional): columnas-lista

`starwars` incluye columnas que son listas, como `films` (las películas en las que aparece cada personaje). Calcula en cuántas películas aparece cada personaje y muestra los que más aparecen.

Capítulo 3

Introducción al lenguaje SQL

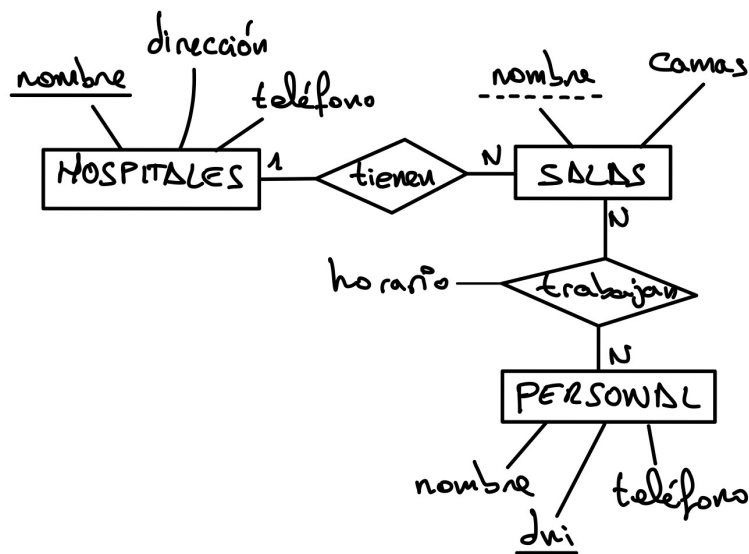
Los sistemas de información gestionan repositorios de información en múltiples formatos, siendo el más popular las bases de datos relacionales a las que se accede mediante SQL (Structured Query Language).

El ejemplo que trabajaremos en este capítulo está disponible en Kaggle: kaggle.com/code/diegodx/txd-2026-tutorialsql

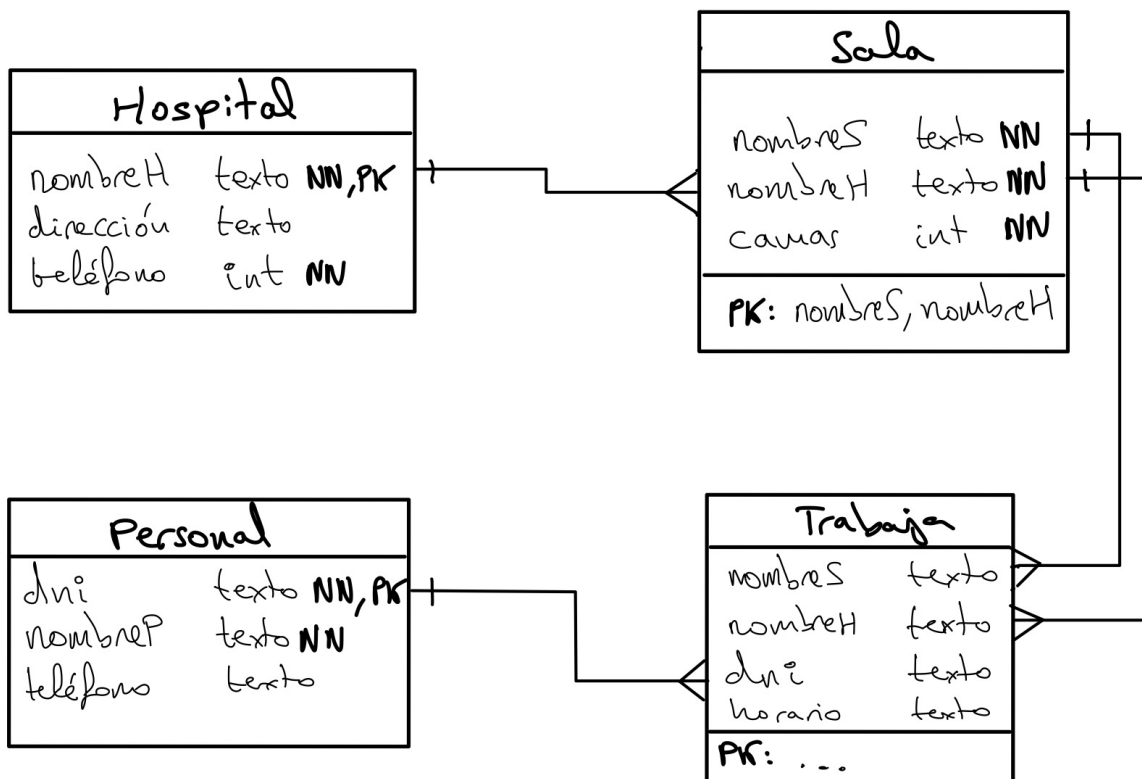
3.1 Bases de Datos Relacionales

3.1.1 Definiciones

- **Dominio:** contexto (organización, empresa, evento...) objeto de gestión de la información.
 - **Dato:** hecho con significado implícito, registable, relevante en un determinado dominio.
 - **Base de datos:** colección de datos de un determinado dominio relacionados entre sí, organizados de forma que sea posible manipularlos y recuperarlos de forma eficiente.
 - Sistema de Gestión de Bases de Datos (**SGBD**) (en inglés **RDBMS**, Relational Database Management System): software que permite a los usuarios crear y manipular bases de datos mediante operaciones **CRUD**:
 - **Create:** Crear / Insertar datos
 - **Rread:** Consultar / Leer datos
 - **Update:** Actualizar / Modificar datos
 - **Delete:** Eliminar datos
-
- **Modelo de datos:** abstracción conceptual que propone una manera de organizar y manipular los datos. Definido mediante:
 - Estructura: elementos para organizar datos
 - Integridad: reglas para relaciones los elementos
 - Manipulación: operaciones sobre los datos adaptadas a la estructura y reglas
 - Modelo **Entidad Relación** (entidades, relaciones, atributos)



- Modelo de datos lógico o de representación (modelo relacional de Codd)
 - Datos en relaciones (tablas)
 - Base matemática formal
 - Flexible



- Modelo de datos físico (tal y como se almacenan los datos)

Una fila de la tabla (relación) es una tupla y una columna un atributo.

Estudiantes			
<u>dni</u>	nombre	apellidos	email
11.222.333Z	Pedro	Pérez Pérez	infppp@udc.es
22.333.444Z	Ana	López Pérez	infalp@udc.es
11.888.999Z	Alberto	López López	infall@udc.es

Una base de datos es un conjunto de tablas (al menos una).

dni	nombre	...
11.222.333Z	Pedro	...
...	...	...

codigo	nombre	creditos
614G01002	IB	6
...	...	...

dniEstudiante	codMateria	curso
11.222.333Z	614G01002	2018/2019
...	...	...

La tabla no es una relación porque la relación es un conjunto sin orden y una tabla puede tener filas repetidas y tiene orden.

-
- **Esquema:** estructura de la base de datos
 - **Estado:** contenido de la base de datos
 - Restricción de **integridad:** regla que debe cumplir la información registrada en la base de datos para garantizar la integridad de la información.

3.2 Restricciones

Cualquier Base de Datos basada en el modelo relacional debería cumplir como mínimo estas restricciones (además de las propias del dominio):

- **Restricción de dominio:** Cada atributo debe tener un tipo de valores permitido, asegurando que sólo se almacenen datos válidos y consistentes

- **Atributos atómicos:** Cada atributo debe almacenar valores indivisibles (nombre completo descomponible en nombre y apellidos, domicilio en calle, CP, localidad, etc...)
- **Unicidad:** Co pueden existir dos tuplas iguales. Para ello se definen claves:
 - Una **superclave** es un subconjunto de atributos tal que no existen dos tuplas con la misma superclave. > Ejercicio. En la relación Empleado(dni, nombre, apellidos, email) ¿cuántas superclaves existen?
 - Una **clave candidata** es una superclave mínima (superclave mínima es la clave a la que no se le puede eliminar un atributo). > ¿Cuántas claves candidatas hay en el ejemplo anterior?
 - La **clave primaria** es la clave candidata que elegimos que identificar de forma unívoca las tuplas de una relación. Restricción de integridad de entidad: Ningún valor de la clave primaria puede ser un valor nulo.
- Una **clave foránea** es un conjunto de atributos de una relación R_1 que, para cada tupla, identifican a otra tupla de una relación R_2 con la que está relacionada. La Restricción de integridad referencial nos dice que la clave foránea ha de corresponderse con la clave primaria de R_2, y si la clave foránea no es nula ha de refer a una tupla en R_2.

<u>dni</u>	nombre	...
11.222.333Z	Pedro	...
22.333.444Z	Ana	...
11.888.999Z	Alberto	...

<u>codigo</u>	nombre	creditos
614G01002	Informática Básica	6
614G01013	Bases de Datos	6
650G01022	Bases de Datos	6

Identifica al estudiante que realizó la matrícula

<u>dniEstudiante</u>	<u>codMateria</u>	<u>curso</u>
11.222.333Z	614G01002	2018/2019

<u>dni</u>	nombre	apellidos	despacho
11.222.333Z	Pedro	Pérez Pérez	D.01
22.333.444Z	Ana	López Pérez	D.01
11.888.999Z	Alberto	López López	

<u>numero</u>	piso	capacidad
D.01	1º	2
D.02	1º	2
D.03	2º	1

La foránea *despacho* toma valor nulo en esta tupla
(El docente no tiene todavía despacho asignado)
ES CORRECTO Y NO VIOLA LA RESTRICCIÓN

Si borramos/actualizamos un valor de clave foránea podemos: (a) prohibir el cambio, o (b) poner a nulo la clave foránea (borrado) o propagar el cambio (modificación).

3.3 Sistemas Gestores de Bases de Datos (SGDB)

Utilizar un SGDB tiene múltiples ventajas:

- Administración centralizada de los datos (por un administrador en un servidor/plataforma central que evita la información en silos -redundante/inconsistente)
- Desacoplamiento del almacenamiento físico de los datos (no es necesario conocerlo)
- Simplicidad de acceso (ODBC + SQL, lenguaje declarativo)
- Control de integridad (restricciones genéricas, integridad de entidad y referencial, de dominio, y las del dominio en software)
- Control de acceso concurrente (evita inconsistencia)
- Seguridad (autenticación, roles de acceso)
- Recuperación ante fallos (backup, logs y transacciones -rollback-)

Existen muchos SGDBs, pero los más populares son:

- **SQLite**: es muy ligero, no necesita un servidor para ejecutarse y es muy rápido cuando el conjunto de datos es pequeño.
 - Su escalabilidad y concurrencia son muy limitadas
 - Ideal para proyectos pequeños, aplicaciones locales o móviles y prácticas de TXD
 - Usado en Google Chrome, Firefox, Safari, Dropbox (app de escritorio)
 - Es de dominio público
- **MySQL**: requiere un servidor para ejecutarse, cuenta con soporte en la nube
 - Presenta buena escalabilidad y concurrencia
 - Es muy utilizado en aplicaciones web de tamaño medio
 - Usado en Wordpress, y en general un gran porcentaje de páginas y aplicaciones web de tamaño pequeño y mediano. Incluso sitios grandes como Wikipedia o Facebook emplean variantes de MySQL.
 - Licencia GPL (con variantes comerciales)
- **PostgreSQL**: también requiere un servidor y tiene soporte en la nube
 - Su escalabilidad es excelente, óptimo para grandes cantidades de datos
 - Cuenta con muchas extensiones para, por ejemplo, información geográfica (PostGIS), series temporales (TimescaleDB), etc.
 - Ideal para sistemas complejos, analíticos o con alta concurrencia
 - Usado en Reddit, Instagram, Spotify, Netflix, ...
 - Licencia propia tipo MIT
- Otros SGDBs relacionales: MariaDB (derivado de MySQL), Microsoft SQL Server, Oracle

Ranking de popularidad según DB-Engines. La puntuación está calculada en función del número de menciones en páginas web, interés general en los sistemas, frecuencia de discusiones técnicas, cantidad de ofertas de trabajo, relevancia en redes sociales, etc.

2025	2024	SGDB	Modelo	Puntuacion	Var
1.	1.	Oracle	Relacional, Multi-modelo	1212.77	-96.67
2.	2.	MySQL	Relacional, Multi-modelo	879.66	-143.09
3.	3.	Microsoft SQL Server	Relacional, Multi-modelo	715.05	-87.04
4.	4.	PostgreSQL	Relacional, Multi-modelo	643.20	-8.96

2025	2024	SGDB	Modelo	Puntuacion	Var
5.	5.	MongoDB	Documental, Multi-modelo	368.01	-37.20
6.	7.	Snowflake	Relacional	198.65	+58.05
7.	6.	Redis	Clave-valor, Multi-modelo	142.33	-7.30
8.	14.	Databricks	Multi-modelo	128.80	+43.21
9.	9.	IBM Db2	Relacional, Multi-modelo	122.37	-0.40
10.	8.	Elasticsearch	Multi-modelo	116.67	-15.18
11.	11.	Apache Cassandra	Wide column, Multi-modelo	105.16	+7.56
12.	10.	SQLite	Relacional	104.56	+2.64
13.	15.	MariaDB	Relacional, Multi-modelo	87.77	+2.88
14.	12.	Microsoft Access	Relacional	80.79	-11.36
15.	17.	Amazon DynamoDB	Multi-modelo	75.91	+4.06

3.4 Sintaxis SQL

SQL (Structured Query Language) es un lenguaje declarativo. Es un lenguaje estándar: tiene un estándar oficial definido por ISO y ANSI. Sin embargo, en la práctica cada SGDB implementa solo una parte de él, y además tiene sus propios dialectos, en los que puede variar por ejemplo lo siguiente:

- Tipos de datos (TEXT, VARCHAR, BLOB, etc.)
- Distinción o no de mayúsculas y minúsculas (cap sensitiveness)
- Cómo se manejan las transacciones
- Funciones (LENGTH(), LEN(), etc.)
- Formato de fechas
- Gestión de triggers
- ...

Sintaxis general:

- Consulta

```
SELECT <campo/s>
FROM <tabla>
WHERE <condición>
GROUP BY <campo>
HAVING <condición>
ORDER BY <campo>
LIMIT <m> OFFSET <n>
```

- Modificación

```
UPDATE <tabla>
SET <cambios>
WHERE <condición>
```

- Borrado

```
DELETE FROM <tabla>  
WHERE <condición>
```

3.5 Cláusulas básicas de SQL

3.5.1 Selección de campos

- Seleccionar todas las columnas de una tabla:

```
SELECT * FROM Track;
```

- Seleccionar columnas específicas:

```
SELECT name, composer FROM Track;
```

- Alias de columna y tabla:

```
SELECT name AS Canción FROM Track;
```

```
SELECT T.name FROM Track AS T;
```

- Funciones de agregación:

```
SELECT COUNT(*), SUM(UnitPrice), MIN(UnitPrice), MAX(UnitPrice) FROM Track;
```

```
SELECT AVG(milliseconds) AS 'Duración Media' FROM Track;
```

- Formato:

```
SELECT CONCAT(FirstName, ' ', LastName) AS Nombre FROM Employee;
```

```
SELECT (FirstName || ' ' || LastName) AS Nombre FROM Employee;
```

```
SELECT ROUND(AVG(Total), 2) AS 'Facturacion Media' FROM Invoice;
```

3.5.2 Número de filas (paginación)

- Obtener las N primeras:

```
SELECT * FROM Track  
ORDER BY title ASC  
LIMIT 5;
```

- Obtener las N filas siguientes:

```
SELECT * FROM Track  
ORDER BY title ASC  
OFFSET 5 LIMIT 5;
```

3.5.3 Filtrado de Resultados

- Filtrado de duplicados:

```
SELECT DISTINCT FirstName FROM Customer;
```

```
SELECT COUNT(DISTINCT FirstName) FROM Customer;
```

- Seleccionar filas con condiciones:

```
SELECT name FROM Track
WHERE UnitPrice < 2.0;
```

- Múltiples condiciones:

- Operadores: 'AND', 'OR', 'LIKE', 'NOT', 'IS NULL', 'IS NOT NULL', 'BETWEEN x AND y', 'IN (lista)'

```
SELECT name FROM Track
WHERE milliseconds > 120000 AND UnitPrice < 2.0;
```

```
SELECT name FROM Track
WHERE composer LIKE 'Metallica' OR composer LIKE 'Ulrich';
```

- Coincidencias parciales:

```
SELECT * FROM Track
WHERE name LIKE '%Love%';
```

- '%' : Reemplazo por un conjunto de caracteres
- '_' : Reemplazo por un caracter
- Rangos:

```
SELECT * FROM Track
WHERE UnitPrice BETWEEN 0.5 AND 1.5;
```

- Valores en una lista:

```
SELECT * FROM Track
WHERE composer IN ('Metallica', 'Ulrich');
```

3.5.4 Ordenación de Resultados

- Ordenar por una columna:

```
SELECT * FROM Track
ORDER BY title ASC;
```

```
SELECT * FROM Track
ORDER BY title DESC;
```

- Ordenar por múltiples columnas:

```
SELECT * FROM Track
ORDER BY composer ASC, title DESC;
```

3.5.5 Unión de tablas

- Intersección por clave primaria / clave foránea:

- Operador A *[INNER] JOIN* B: intersección de A y B

```
SELECT ar.Name AS Artista, a.Title AS Album
FROM Artist ar JOIN Album a ON ar.ArtistId = a.ArtistId;
```

- Operador A *LEFT JOIN* B: intersección de A y B, y filas de A sin correspondencia en B

```
SELECT ar.Name AS Artista, a.Title AS Album
FROM Artist ar LEFT JOIN Album a ON ar.ArtistId = a.ArtistId;
```

- Operador A *RIGHT JOIN* B: intersección de A y B, y filas de B sin correspondencia en A
- A 'FULL [OUTER] JOIN' B: intersección de A y B, y filas de A y B sin correspondencia
- sqlite no soporta directamente RIGHT JOIN ni FULL JOIN
- Producto cartesiano

```
SELECT g.Name AS Genero, m.Name AS Formato
FROM Genre g CROSS JOIN MediaType m
```

3.5.6 Agrupamiento

- Funciones de agregación parciales

```
SELECT A.Title AS Album, COUNT(*) AS Canciones, SUM(UnitPrice) AS Precio
FROM Album A JOIN Track T ON A.AlbumId = T.AlbumId
GROUP BY A.Title;
```

- *CONSEJO*: Es importante agrupar por *clave candidata* o *superclave*. La consulta anterior daría resultados incorrectos si existen dos álbumes diferentes con el mismo título. Mejor:

```
SELECT A.Title AS Album, COUNT(*) AS Canciones, SUM(UnitPrice) AS Precio
FROM Album A JOIN Track T ON A.AlbumId = T.AlbumId
GROUP BY A.AlbumId;
```

- Además, también es importante que ese campo de agrupación o algo que garantice la identificación unívoca se presente en los resultados. En el ejemplo anterior sí aparecerían los álbumes por separado, pero no sería posible identificar cuál es cuál en los resultados. Mejor:

```
SELECT A.AlbumId, A.Title AS Album, COUNT(*) AS Canciones, SUM(UnitPrice) AS Precio
FROM Album A JOIN Track T ON A.AlbumId = T.AlbumId
GROUP BY A.AlbumId;
```

- Incluso mejor, si asumimos que no habrá un artista con dos álbumes con el mismo nombre:

```
SELECT AR.Name AS Artista, AS Album, COUNT(*) AS Canciones, SUM(UnitPrice) AS Precio
FROM Album A JOIN Track T ON A.AlbumId = T.AlbumId
      JOIN Artist AR ON A.ArtistId = AR.ArtistId
GROUP BY A.AlbumId;
```

- Filtrado por grupos (seleccionar grupos con condiciones)

```
SELECT AR.Name AS Artista, AS Album, COUNT(*) AS Canciones, SUM(UnitPrice) AS Precio
FROM Album A JOIN Track T ON A.AlbumId = T.AlbumId
GROUP BY A.AlbumId
HAVING Canciones > 6;
```

3.5.7 Subconsultas (subqueries)

- Subqueries en la cláusula SELECT:
 - La consulta debe devolver una única columna, y típicamente un único valor

```
SELECT T.Name, (SELECT COUNT(*)
                FROM Track T2
                WHERE T2.GenreId = T.GenreId) AS Canciones
FROM Track T JOIN Genre G ON T.GenreId = G.GenreId;
```

- *Observación:* en una subquery se puede hacer referencia a columnas de la consulta externa.
- Subqueries en la cláusula WHERE / HAVING:

```
SELECT T.Name, T.Milliseconds
FROM Track T
WHERE T.Milliseconds > (SELECT AVG(Milliseconds) FROM Track);
```

- Subqueries en la cláusula FROM:

```
SELECT T.Song, G.Name
FROM Genre G JOIN (SELECT Name AS Song, GenreId as Id
                  FROM Track) AS T
ON Id = G.GenreId
```

3.5.8 Operaciones con conjuntos de resultados

```
Query1
{ UNION [ ALL ] | INTERSECT | EXCEPT }
Query2
```

- Query1 y Query2 deben devolver el *mismo número de columnas*, pero no necesariamente el mismo tipo de dato. Elimina filas duplicadas.
- **Unión:** Resultados existentes en alguna consulta (eq. OR)
- **Intersección:** Resultados existentes en ambas consultas (eq. AND)
- **Diferencia (Except):** Resultados existentes en Query1 pero no en Query2 (eq. MINUS)
- Cláusula **UNION ALL:** Elimina la restricción de filas duplicadas y muestra todo

3.6 Gestión de datos

3.6.1 Operaciones CRUD

3.6.1.1 Inserción de tuplas

```
INSERT INTO table_name( column1, column2....columnN)
VALUES ( value1, value2....valueN);
```

- El número de valores **debe coincidir** con el número de columnas especificadas
- Si alguna columna de la tabla no se indica, su valor será el indicado por defecto o **NULL**
 - Por ejemplo, AUTOINCREMENT
- Si no se especifica alguna columna con restricción **NOT NULL**, la inserción producirá un error
- Igualmente, dará error si no se cumple cualquier otra restricción
- Es posible insertar datos a partir de otra consulta:

```
INSERT INTO TopClientes (nombre, gasto_total)
SELECT nombre, SUM(importe)
```

```
FROM Ventas
GROUP BY nombre
HAVING SUM(monto) > 1000;
```

3.6.1.2 Modificación de datos

```
UPDATE table_name
SET column1 = value1, column2 = value2...columnN=valueN
[ WHERE CONDITION ];
```

- Si no se incluye WHERE, se actualiza toda la tabla. Lo más habitual es utilizar la clave primaria en el WHERE.
- Se puede actualizar una columna a partir de un cálculo:

```
UPDATE Ventas
SET total = cantidad * precio_unitario;
```

- Se puede actualizar a partir de una consulta

```
UPDATE Album
SET artista_id = (
    SELECT id FROM Artista WHERE nombre = 'Queen'
)
WHERE titulo = 'A Night at the Opera';
```

- Si la actualización afecta una columna usada en la condición, la comprobación se hace sobre el valor original, no el nuevo.
- Al actualizar también se verifican las restricciones
- Buenas prácticas:
 - Realizar copias de seguridad antes de actualizaciones masivas.
 - Usar transacciones si se actualizan varias tablas relacionadas.
 - No modificar claves primarias salvo necesidad justificada.
 - Revisar restricciones de integridad antes de ejecutar.

3.6.1.3 Eliminación de datos

```
DELETE FROM table_name
WHERE {CONDITION};
```

- Antes de borrar, se comprueban las restricciones de la Base de Datos.
- Al usar DELETE, los valores de una columna AUTOINCREMENT no se modifican: el siguiente INSERT seguirá con el siguiente número disponible.
- Buenas prácticas:
 - Siempre incluir WHERE salvo que se quiera vaciar la tabla.
 - Comprobar cuántas filas se afectarán con SELECT COUNT(*) ... antes.
 - Usar transacciones para borrados masivos o dependientes.
 - Revisar reglas ON DELETE en claves foráneas (para evitar borrados en cascada inesperados).
 - Realizar copia de seguridad antes de operaciones grandes.

3.6.1.4 Gestión de transacciones

```
BEGIN;

...

COMMIT;

ROLLBACK;
```

- Ejemplo: si alguna operación falla, no se realiza ninguna

```
BEGIN;
INSERT INTO Pedido VALUES (1, '2025-11-04', 100);
UPDATE Cliente SET saldo = saldo - 100 WHERE id = 1;
COMMIT;
```

- Las transacciones cumplen las propiedades **ACID**

Propiedad	Significado
Atomicidad	Todas las operaciones se ejecutan como una sola — o todas, o ninguna.
Consistencia	Los datos pasan de un estado válido a otro estado válido.
Aislamiento (Isolation)	Cada transacción se ejecuta como si fuera la única.
Durabilidad	Una vez hecho COMMIT, los cambios se hacen persistentes.

- Los cambios realizados no son visibles para otras conexiones hasta que se hace COMMIT
- Inserciones/actualizaciones en bloque son mucho más rápidas dentro de una transacción

3.7 Gestión de Bases de Datos

3.7.1 Creación de una base de datos

```
CREATE DATABASE database_name;
```

3.7.2 Eliminación de una base de datos

```
DROP DATABASE database_name;
```

3.7.3 Selección de base de datos

```
USE database_name;
```

3.8 Gestión de tablas

3.8.1 Creación de tablas

```
CREATE TABLE [IF NOT EXISTS] table_name (
    column1 datatype [constraints],
```

```

column2 datatype [constraints],
...
columnN datatype [constraints],
[table_constraints])
);

```

3.8.1.1 Tipos de datos

Propósito / tipo lógico	SQLite	MySQL	PostgreSQL
Enteros (números sin decimales)	INTEGER / INT / SMALLINT / BIGINT	TINYINT, SMALLINT, MEDIUMINT, INT, BIGINT	SMALLINT, INTEGER, BIGINT
Números reales / decimales	REAL / FLOAT / DOUBLE / NUMERIC	FLOAT, DOUBLE, DECIMAL(p,s)	REAL, DOUBLE PRECISION, NUMERIC(p,s)
Texto / cadenas	TEXT, CHAR(n), VARCHAR(n)	CHAR(n), VARCHAR(n), TEXT	CHAR(n), VARCHAR(n), TEXT
Valores booleanos	BOOLEAN	BOOLEAN / TINYINT(1) (interpreta 0/1)	BOOLEAN (<i>nativo</i>)
Fechas y horas	DATE, TIME, DATETIME, TIMESTAMP	DATE, TIME, DATETIME, TIMESTAMP	DATE, TIME, TIMESTAMP, INTERVAL
Información binaria	BLOB	BLOB, BINARY, VARBINARY	BYTEA

3.8.1.2 Restricciones a nivel de columna:

Restricción	Descripción	Ejemplo
PRIMARY KEY	Clave primaria	id INTEGER PRIMARY KEY
AUTOINCREMENT	Hace que un INTEGER PRIMARY KEY aumente siempre	id INTEGER PRIMARY KEY AUTOINCREMENT
NOT NULL	No permite valores nulos	nombre TEXT NOT NULL
UNIQUE	Impide valores duplicados	email TEXT UNIQUE
CHECK (condición)	Impone una condición lógica	edad INTEGER CHECK (edad >= 0)
DEFAULT valor	Asigna un valor por defecto si no se especifica	pais TEXT DEFAULT 'España'
REFERENCES tabla(columna)	Define una clave foránea	artista_id INTEGER REFERENCES Artista(id)

- En una **clave foránea** se puede especificar el trigger **ON DELETE** para determina qué ocurre cuando se elimina la tupla referenciada:
 - **CASCADE**: Se borran todas las filas que referencian a la tupla eliminada
 - **SET NULL**: Se pone a nulo la clave foránea
 - **SET DEFAULT**: Se pone la clave foránea al valor definido como DEFAULT
 - **RESTRICT**: Impide eliminar la fila referenciada (valor por defecto)
- Con ninguna de estas acciones la base de datos puede quedar en un estado inconsistente

3.8.1.3 Restricciones a nivel de tabla:

Tipo	Descripción	Ejemplo
PRIMARY KEY (col1, col2)	Clave primaria compuesta	PRIMARY KEY (id, nombre)
UNIQUE (col1, col2)	Combinación única	UNIQUE (email, pais)
CHECK (condición)	Condición general	CHECK (precio > 0 OR descuento IS NOT NULL)
FOREIGN KEY (col) REFERENCES ...	Clave foránea explícita	FOREIGN KEY (artista_id) REFERENCES Artista(id)

3.8.2 Creación de índices

```
CREATE UNIQUE INDEX index_name
ON table_name ( column1, column2, ... columnN);
```

3.8.3 Borrado de tablas

```
DROP TABLE table_name;
```

3.8.4 Modificación de tablas

```
ALTER TABLE table_name
DROP INDEX index_name;

ALTER TABLE table_name
{ADD|DROP|MODIFY} column_name {data_type};

ALTER TABLE table_name RENAME TO new_table_name;
```

3.9 Vistas

Una vista es una **consulta guardada** que actúa como una tabla virtual. Facilita reutilizar consultas o simplificar código.

- Creación de una vista

```
CREATE [TEMP] VIEW [IF NOT EXISTS] view_name AS
SELECT ...;
```

- Vistas temporales

Al añadir “TEMP” a la creación de la vista, sólo existe durante la sesión actual. Se borra al desconectarse de la Base de Datos.

- Eliminación de vistas

Se puede eliminar una vista, tanto temporal como permanente:

```
DROP VIEW view_name;
```

- Las vistas en SQLite **no almacenan datos**, solo la consulta.
- Para hacer vistas modificables (actualizables) la consulta debe cumplir ciertas condiciones: una sola tabla, sin agregaciones.

3.10 Ejemplos de consultas SQL

```
SELECT Nombre, Apellido1, Apellido2, Municipio, Provincia
FROM Cliente
WHERE Municipio = 'Lugo'
ORDER BY Apellido1

INSERT Proveedor(Nombre, PersonaContacto, Ciudad, País)
VALUES ('Café Candelas', 'Ivana Candelas', 'Lugo', 'España')

UPDATE Pedidos
SET Cantidad = 2
WHERE IdProducto = 963

DELETE Cliente
WHERE Email = 'alexandregb@gmail.com'
```

3.11 Conexión con bases de datos desde R

3.11.1 Introducción a SQL en R

SQL se usa para manipular datos dentro de una base de datos. Si la base de datos no es muy grande se puede cargar toda en un `data.frame`. No obstante, por escalabilidad y offloading de la carga de trabajo al servidor SGBD utilizaremos SQL.

Existen varios SGBD (SQLite, Microsoft SQL Server, MySQL, PostgreSQL, etc) los cuales comparten el soporte de SQL (en concreto ANSI SQL) aunque cada gestor extiende SQL de formas sutiles buscando minar cierta portabilidad de código (*vendor-locking*). En efecto, un código SQL desarrollado para SQLite es probable que falle con MySQL aunque tras aplicar ligeras modificaciones ya funcionará. Asimismo el mecanismo de conexión, configuración, rendimiento y operación suele diferir entre SGBD.

A continuación se lista una serie de paquetes utilizados en el acceso a los datos, lo que suele ser el principal esfuerzo a realizar cuando se trabaja con SGBD:

- DBI
- RODBC
- dbConnect
- RSQLite
- RMySQL
- RPostgreSQL

3.11.2 El paquete `sqldf`

A continuación se presenta una serie de ejercicios con la sintaxis de SQL operando sobre un `data.frame` con el paquete `sqldf`. Esto inicialmente no incluye los detalles de conectarse a un SGBD, ni modificar los datos, solamente el uso de SQL para extraer datos con el objetivo de ser analizados en R.

```
library(sqldf)
```

```
sqldf('SELECT age, circumference FROM Orange WHERE Tree = 1 ORDER BY circumference ASC')
```

```
##    age circumference
## 1  118             30
## 2  484             58
## 3  664             87
## 4 1004            115
## 5 1231            120
## 6 1372            142
## 7 1582            145
```

3.11.3 SQL Queries

El comando inicial es SELECT. SQL no es case-sensitive, por lo que esto va a funcionar:

```
sqldf("SELECT * FROM iris")
sqldf("select * from iris")
```

pero lo siguiente no va a funcionar (a menos que tengamos un objeto IRIS:

```
sqldf("SELECT * FROM IRIS")
```

La sintaxis básica de SELECT es:

```
SELECT variable1, variable2 FROM data
```

3.11.3.1 Asterisco/Wildcard

Lo extrae todo

```
bod2 <- sqldf('SELECT * FROM BOD')
```

3.11.3.2 Limit

Limita el número de resultados

```
sqldf('SELECT * FROM iris LIMIT 5')
```

```
##    Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1          5.1         3.5         1.4         0.2   setosa
## 2          4.9         3.0         1.4         0.2   setosa
## 3          4.7         3.2         1.3         0.2   setosa
## 4          4.6         3.1         1.5         0.2   setosa
## 5          5.0         3.6         1.4         0.2   setosa
```

3.11.3.3 Order By

Ordena las variables

```
ORDER BY var1 {ASC/DESC}, var2 {ASC/DESC}
```

```
sqldf("SELECT * FROM Orange ORDER BY age ASC, circumference DESC LIMIT 5")
```

```
##    Tree age circumference
## 1     2 118             33
```

```
## 2      4 118      32
## 3      1 118      30
## 4      3 118      30
## 5      5 118      30
```

3.11.3.4 Where

Sentencias condicionales, donde se puede incorporar operadores lógicos AND y OR, expresando el orden de evaluación con paréntesis en caso de ser necesario.

```
sqldf('SELECT demand FROM BOD WHERE Time < 3')
```

```
##      demand
## 1         8.3
## 2        10.3
```

```
sqldf('SELECT * FROM rock WHERE (peri > 5000 AND shape < .05) OR perm > 1000')
```

```
##      area      peri      shape perm
## 1 5048    941.543 0.328641 1300
## 2 1016    308.642 0.230081 1300
## 3 5605   1145.690 0.464125 1300
## 4 8793   2280.490 0.420477 1300
```

Y extendiendo su uso con IN o LIKE (es último sólo con %), pudiendo aplicárseles el NOT:

```
sqldf('SELECT * FROM BOD WHERE Time IN (1,7)')
```

```
##      Time demand
## 1         1      8.3
## 2         7     19.8
```

```
sqldf('SELECT * FROM BOD WHERE Time NOT IN (1,7)')
```

```
##      Time demand
## 1         2     10.3
## 2         3     19.0
## 3         4     16.0
## 4         5     15.6
```

```
sqldf('SELECT * FROM chickwts WHERE feed LIKE "%bean" LIMIT 5')
```

```
##      weight      feed
## 1       179 horsebean
## 2       160 horsebean
## 3       136 horsebean
## 4       227 horsebean
## 5       217 horsebean
```

```
sqldf('SELECT * FROM chickwts WHERE feed NOT LIKE "%bean" LIMIT 5')
```

```
##      weight      feed
## 1       309 linseed
## 2       229 linseed
## 3       181 linseed
## 4       141 linseed
## 5       260 linseed
```

3.12 Ejemplo Scopus data

Ver ejemplo *citan.zip* y paquete CITAN (descontinuado).

Las herramientas de este paquete realizan consultas SQL de forma continuada a una base de datos por lo que pueden resultar notablemente lentas. La recomendación es realizar el menor número de consultas posible y después operar en R:

“If your data fits in memory there is no advantage to putting it in a database: it will only be slower and more frustrating”

— Hadley Wickham – <https://dbplyr.tidyverse.org/articles/dbplyr.html>

3.13 Ejercicios SQL con RSQLite

3.13.1 Setup de RSQLite

Vamos a utilizar RSQLite desde Kaggle. Pero si lo queréis instalar en local La información para su instalación está en el siguiente enlace.

```
library(DBI)

# Create an ephemeral in-memory RSQLite database
con <- dbConnect(RSQLite::SQLite(), ":memory:")
dbListTables(con)
```

```
## character(0)
```

```
dbWriteTable(con, "mtcars", mtcars)
dbListTables(con)
```

```
## [1] "mtcars"
```

```
dbListFields(con, "mtcars")
```

```
## [1] "mpg" "cyl" "disp" "hp" "drat" "wt" "qsec" "vs" "am" "gear"
## [11] "carb"
```

```
dbReadTable(con, "mtcars")
```

```
##      mpg  cyl  disp  hp  drat    wt  qsec vs  am  gear  carb
## 1  21.0    6  160.0  110  3.90  2.620  16.46  0   1    4    4
## 2  21.0    6  160.0  110  3.90  2.875  17.02  0   1    4    4
## 3  22.8    4  108.0   93  3.85  2.320  18.61  1   1    4    1
## 4  21.4    6  258.0  110  3.08  3.215  19.44  1   0    3    1
## 5  18.7    8  360.0  175  3.15  3.440  17.02  0   0    3    2
## 6  18.1    6  225.0  105  2.76  3.460  20.22  1   0    3    1
## 7  14.3    8  360.0  245  3.21  3.570  15.84  0   0    3    4
## 8  24.4    4  146.7   62  3.69  3.190  20.00  1   0    4    2
## 9  22.8    4  140.8   95  3.92  3.150  22.90  1   0    4    2
## 10 19.2    6  167.6  123  3.92  3.440  18.30  1   0    4    4
## 11 17.8    6  167.6  123  3.92  3.440  18.90  1   0    4    4
## 12 16.4    8  275.8  180  3.07  4.070  17.40  0   0    3    3
## 13 17.3    8  275.8  180  3.07  3.730  17.60  0   0    3    3
## 14 15.2    8  275.8  180  3.07  3.780  18.00  0   0    3    3
## 15 10.4    8  472.0  205  2.93  5.250  17.98  0   0    3    4
## 16 10.4    8  460.0  215  3.00  5.424  17.82  0   0    3    4
## 17 14.7    8  440.0  230  3.23  5.345  17.42  0   0    3    4
```

```
## 18 32.4 4 78.7 66 4.08 2.200 19.47 1 1 4 1
## 19 30.4 4 75.7 52 4.93 1.615 18.52 1 1 4 2
## 20 33.9 4 71.1 65 4.22 1.835 19.90 1 1 4 1
## 21 21.5 4 120.1 97 3.70 2.465 20.01 1 0 3 1
## 22 15.5 8 318.0 150 2.76 3.520 16.87 0 0 3 2
## 23 15.2 8 304.0 150 3.15 3.435 17.30 0 0 3 2
## 24 13.3 8 350.0 245 3.73 3.840 15.41 0 0 3 4
## 25 19.2 8 400.0 175 3.08 3.845 17.05 0 0 3 2
## 26 27.3 4 79.0 66 4.08 1.935 18.90 1 1 4 1
## 27 26.0 4 120.3 91 4.43 2.140 16.70 0 1 5 2
## 28 30.4 4 95.1 113 3.77 1.513 16.90 1 1 5 2
## 29 15.8 8 351.0 264 4.22 3.170 14.50 0 1 5 4
## 30 19.7 6 145.0 175 3.62 2.770 15.50 0 1 5 6
## 31 15.0 8 301.0 335 3.54 3.570 14.60 0 1 5 8
## 32 21.4 4 121.0 109 4.11 2.780 18.60 1 1 4 2
```

```
# You can fetch all results:
```

```
res <- dbSendQuery(con, "SELECT * FROM mtcars WHERE cyl = 4")
dbFetch(res)
```

```
##      mpg cyl  disp  hp drat   wt  qsec vs am gear carb
## 1  22.8   4  108.0  93 3.85 2.320 18.61 1 1  4    1
## 2  24.4   4  146.7  62 3.69 3.190 20.00 1 0  4    2
## 3  22.8   4  140.8  95 3.92 3.150 22.90 1 0  4    2
## 4  32.4   4   78.7  66 4.08 2.200 19.47 1 1  4    1
## 5  30.4   4   75.7  52 4.93 1.615 18.52 1 1  4    2
## 6  33.9   4   71.1  65 4.22 1.835 19.90 1 1  4    1
## 7  21.5   4  120.1  97 3.70 2.465 20.01 1 0  3    1
## 8  27.3   4   79.0  66 4.08 1.935 18.90 1 1  4    1
## 9  26.0   4  120.3  91 4.43 2.140 16.70 0 1  5    2
## 10 30.4   4   95.1 113 3.77 1.513 16.90 1 1  5    2
## 11 21.4   4  121.0 109 4.11 2.780 18.60 1 1  4    2
```

```
dbClearResult(res)
```

```
# Or a chunk at a time
```

```
res <- dbSendQuery(con, "SELECT * FROM mtcars WHERE cyl = 4")
while(!dbHasCompleted(res)){
  chunk <- dbFetch(res, n = 5)
  print(nrow(chunk))
}
```

```
## [1] 5
## [1] 5
## [1] 1
```

```
# Clear the result
```

```
dbClearResult(res)
```

```
# Disconnect from the database
```

```
dbDisconnect(con)
```

3.14 Práctica 1: SQL

Vamos a utilizar la base de datos Chinook del tutorial de SQLite



Los ejercicios pedidos en Kaggle kaggle.com/gltaboada/sqlite-tutorial-in-r se entregarán preferentemente antes del 14/10 compartiendo un notebook con las soluciones (¡notebook privado!) con el usuario **gltaboada**. Antes me tenéis que enviar un email comunicando qué usuario tenéis cada uno. En caso de incidencia me podéis mandar un notebook descargado (.ipynb), o el mecanismo que hayamos acordado previamente.

Capítulo 4

Python

Python es un lenguaje de programación de alto nivel, interpretado y de propósito general.

Se caracteriza por ser legible, versátil y contar con una gran comunidad de usuarios, lo que lo hace ideal tanto para principiantes como para profesionales.

En este documento se presenta una introducción básica al lenguaje, su sintaxis esencial y algunas de sus aplicaciones más comunes.

Enlaces de interés:

- Intérpretes de Python online
 - Online-Python
 - Programiz
 - Python Online
- Tutoriales
 - Oficial 3.14
 - J2Logo (modular)
 - Python España (listado de cursos)
- IDEs
 - pyCharm
 - Visual Studio Code

4.0.1 Instalación

Para instalar Python, puedes descargar la versión más reciente desde:

- <https://www.python.org/downloads/>

En muchos sistemas Linux y macOS, Python ya viene instalado por defecto.

4.1 Introducción a Python

4.1.1 ¿Qué es Python?

Python es un lenguaje creado por Guido van Rossum en 1991.

Se destaca por:

- Sintaxis clara y legible

- Tipado dinámico
- Amplia biblioteca estándar
- Gran ecosistema de paquetes
- Multiplataforma

Python puede ejecutarse de diferentes maneras según la necesidad: - Desde la **línea de comandos**

```
python -c "print('Hola desde Python')"
```

- En modo **interactivo**

Este entorno interactivo se conoce como **REPL** (Read, Eval, Print, Loop)

```
$ python3
Python 3.13.5 (main, Jun 25 2025, 18:55:22) [GCC 14.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Ahora se pueden escribir comandos:

```
>>> 2 + 2
4
>>> print("Hola")
Hola
>>> exit()
```

- Ejecutando archivos **.py**

```
$ python archivo.py
```

- Usando **entornos interactivos** como IPython
- Empleando herramientas para generar una versión “compilada”

4.1.2 Primeros pasos: Hola Mundo

Podemos comenzar escribiendo nuestro primer programa:

```
print("Hola mundo")
```

Para ejecutarlo desde la terminal

```
python mi_programa.py
```

También es posible utilizarlo, al igual que R, de forma interactiva:

4.1.3 Sintaxis básica

Los comentarios se escriben con **#**:

```
# Esto es un comentario
print("Hola")
```

4.1.3.1 Variables y Tipos de Datos

Python no requiere declarar el tipo de las variables (tipado dinámico):

```
x = 10          # Entero (int / long)
pi = 3.1416     # Flotante (float / double)
```

```
nombre = "Ana"      # Cadena (string)
es_valido = True    # Booleano
```

Para mirar cuál es el tipo de dato de una variable se usa la función `type`:

```
print(type(x))
print(type(pi))
print(type(nombre))
print(type(es_valido))
```

Para solicitar una entrada de texto al usuario se usa la instrucción `input`:

```
nombre = input("Introduce tu nombre: ")
```

IMPORTANTE: `input` siempre devuelve un string aunque se introduzca un número. Si se quiere convertir a otro tipo de dato hay que hacerlo explícitamente:

```
edad = int(input("Edad: "))
```

Para imprimir un string, se puede formatear la salida. Indicando una “f” antes del string se indica que es una salida formateada y se pueden indicar variables entre corchetes “{ }”:

```
print(f"Nombre: {nombre}, Edad: {edad}")
print(f"precio: .2f")
```

Una variante más antigua es con el método `format` sobre el string:

```
print("Nombre: {}, Edad: {}".format(nombre, edad))
print("Nombre: {n}, Edad: {e}".format(n=nombre, e=edad))
```

Se puede usar un formato similar al de C (aún más antiguo)

```
print("Nombre: %s, Edad: %d" % (nombre, edad))
```

Y también concatenar strings:

```
print("Hola " + nombre)
```

4.1.3.2 Tuplas

Una **tupla** es una colección **ordenada e inmutable** de elementos.

- Mantienen el **orden** de inserción.
- **No se pueden modificar** (inmutables).
- Pueden contener valores de **distintos tipos**.
- Se usan frecuentemente para representar **filas o registros de datos**.
- Se crean con paréntesis ().

```
tupla = ("Ana", 30, "Madrid")
print(tupla[0])    # Acceso por índice
```

La tupla es útil cuando necesitas una estructura fija que no cambiará (por ejemplo, una fila SQL).

El tipo más cercano en R es un vector (`c(1, 2, 3)`) si los datos son homogéneos o una lista (`list("Ana", 30, "Madrid")`) si son heterogéneos.

4.1.3.3 Listas

Una **lista** es una colección **ordenada y mutable**.

- Mantienen el **orden**.
- **Sí pueden modificarse**: agregar, borrar, reordenar, reemplazar elementos.
- Pueden contener cualquier tipo de dato.
- Se crean con corchetes `[ ]`.

```
numeros = [1, 2, 3, 4]
numeros.append(5)
print(numeros)

numeros[0] = 5      # la lista queda: [5,2,3,4]
numeros.append(17)  # ahora la lista queda [5,2,3,4,17]
print(numeros)
```

La lista es útil cuando la colección va a crecer, modificarse o reordenarse.

4.1.3.4 Diccionarios

Un **diccionario** es una colección **no ordenada** (en versiones modernas, conserva el orden de inserción) de pares **clave-valor**.

- Cada elemento tiene una **clave única**.
- Permiten acceso muy rápido a los valores mediante la clave.
- Se usan para representar **datos estructurados**, como registros con nombre.
- Se crean con llaves `{ }`.

```
persona = {"nombre": "Luis", "edad": 30}
print(persona["nombre"])
persona["edad"] = 31
```

El diccionario es útil cuando necesitas asociar valores a nombres o claves. También es interesante para acceder a filas SQL por nombre de atributo.

El tipo más cercano en R sería una lista nombrada: `list(nombre = "Luis", edad = 30)`, o bien un `environment`:

```
env <- new.env()
env$nombre <- "Luis"
env$edad <- 30
```

También es similar un `dataframe`

```
df <- data.frame(nombre = "Luis", edad = 30)
```

4.1.3.5 Operadores

- Operadores lógicos:

Operación	Python	R	C
AND lógico	<code>and</code>	<code>&</code> (element-wise), <code>&&</code> (control-flow)	<code>&&</code>
OR lógico	<code>or</code>	<code> </code> (element-wise), <code> </code> (control-flow)	<code> </code>
NOT lógico	<code>not</code>	<code>!</code>	<code>!</code>

- Operadores bitwise:

Operación	Python	R	C
AND bitwise	<code>&</code>	<code>bitwAnd(a,b)</code>	<code>&</code>
OR bitwise	<code> </code>	<code>bitwOr(a,b)</code>	<code> </code>
XOR bitwise	<code>^</code>	<code>bitwXor(a,b)</code>	<code>^</code>
NOT bitwise	<code>~</code>	<code>bitwNot(a)</code>	<code>~</code>
Shift izquierda	<code><<</code>	<code>bitwShiftL(a,n)</code>	<code><<</code>
Shift derecha	<code>>></code>	<code>bitwShiftR(a,n)</code>	<code>>></code>

4.1.3.6 Indentación

Python usa la indentación para definir bloques de código. Esto es fundamental: si la indentación es incorrecta, el código no funciona.

La convención es usar 4 espacios por nivel (mejor que usar tabs).

```
if x > 5:
    print("x es mayor que 5")
    print("Esto sigue dentro del bloque")
print("Esto ya está fuera del bloque")
```

Ejemplo **incorrecto** (producirá un error):

```
if x > 5:
    print("Error por mezclar espacios")
    print("Indentación inconsistente")
```

4.1.3.7 Estructuras de control

- Condicionales

```
edad = 18

if edad >= 18:
    print("Mayor de edad")
else:
    print("Menor de edad")
```

- Bucle for

```
for i in range(5):
    print(i)
```

- Bucle while

```
contador = 0
while contador < 5:
    print(contador)
    contador += 1
```

4.1.3.8 Funciones

```
def saludar(nombre):  
    return f"Hola, {nombre}"  
  
print(saludar("María"))
```

4.1.3.9 Errores Comunes de Sintaxis

- Olvidar los dos puntos (:) al iniciar un bloque
- Mezclar tabs y espacios
- Indentación incorrecta

4.1.4 Librerías populares

Python cuenta con miles de bibliotecas. Para usar una biblioteca se utiliza “**import ...**”, de forma análoga a “**library(...)**” en R.

```
import math  
print(math.sqrt(16))
```

Algunas destacadas:

4.1.4.1 Ciencia de Datos

- **NumPy**: Cálculo numérico y manejo de arreglos multidimensionales.
 - **Pandas**: Análisis y manipulación de datos mediante DataFrames.
 - **SciPy**: Funciones científicas y matemáticas avanzadas.
 - **Matplotlib**: Visualización básica y clásica.
 - **Seaborn**: Visualización estadística de alto nivel.
 - **Plotly**: Gráficos interactivos y dinámicos.
 - **Statsmodels**: Modelos estadísticos y econometría.
-

4.1.4.2 Machine Learning e Inteligencia Artificial

- **scikit-learn**: Algoritmos tradicionales de machine learning.
 - **TensorFlow**: Deep learning desarrollado por Google.
 - **PyTorch**: Deep learning desarrollado por Meta.
 - **Keras**: API de alto nivel para redes neuronales.
 - **XGBoost**: Algoritmo de gradient boosting optimizado.
 - **LightGBM**: Boosting rápido para grandes datasets.
 - **CatBoost**: Boosting especializado en variables categóricas.
-

4.1.4.3 Procesamiento del Lenguaje Natural (NLP)

- **NLTK**: Procesamiento de texto tradicional.
 - **spaCy**: NLP industrial rápido y eficiente.
 - **Transformers (Hugging Face)**: Modelos modernos como BERT, GPT, etc.
 - **Gensim**: Modelado de tópicos y embeddings semánticos.
-

4.1.4.4 Desarrollo Web

- **Django**: Framework completo para aplicaciones web.
 - **Flask**: Microframework web ligero y flexible.
 - **FastAPI**: Framework moderno y rápido para APIs (asíncrono).
 - **Bottle**: Alternativa aún más ligera a Flask.
-

4.1.4.5 Utilidades y Herramientas

- **Requests**: Manejo de peticiones HTTP.
 - **BeautifulSoup**: Análisis y extracción de datos HTML.
 - **Selenium**: Automatización de navegadores web.
 - **PyTest**: Framework de testing en Python.
 - **Pillow (PIL)**: Manipulación de imágenes.
 - **Click**: Creación de interfaces de línea de comandos.
-

4.1.4.6 Desarrollo y Arquitectura

- **SQLAlchemy**: ORM y manejo avanzado de bases de datos.
 - **Pydantic**: Validación de datos basada en tipos.
 - **Celery**: Ejecución de tareas distribuidas.
 - **Redis-py**: Cliente de Python para Redis.
-

4.1.4.7 Data Engineering y Big Data

- **PySpark**: Uso de Apache Spark desde Python.
 - **Dask**: Computación distribuida para conjuntos grandes de datos.
 - **Vaex**: Manipulación de datos grandes sin cargarlos completamente en memoria.
 - **Ray**: Framework para computación distribuida y ML escalable.
-

4.1.4.8 Librerías Estándar Muy Usadas

- **os**: Interacción con el sistema operativo.
- **sys**: Acceso directo al intérprete de Python.
- **json**: Lectura y escritura de archivos JSON.
- **re**: Manejo de expresiones regulares.
- **datetime**: Manejo de fechas y horas.
- **subprocess**: Ejecución de comandos externos.

4.2 Python y SQLite

`sqlite3` es un módulo estándar de Python que permite:

- Conectarse a una base de datos SQLite
- Ejecutar consultas SQL

- Gestionar transacciones (`commit`, `rollback`)
- Recibir datos como tuplas o diccionarios
- Crear tablas, insertar, actualizar y borrar datos

```
import sqlite3

conn = sqlite3.connect("chinook.db")
cursor = conn.cursor()
```

`conn` es un *objeto conexión* análogo al que se usa en R. Representa un canal abierto entre tu programa y el archivo de base de datos.

Sirve para:

- Iniciar y finalizar transacciones
- Ejecutar consultas
- Confirmar cambios (`commit`)
- Revertir cambios (`rollback`)
- Crear cursores
- Cerrar la base de datos

Para ejecutar consultas se obtiene un **cursor** a partir de esa conexión. El cursor es uno de los conceptos más importantes cuando trabajas con cualquier base de datos desde Python. Un cursor es un objeto intermediario entre el programa y la base de datos. Se usa para:

- enviar consultas SQL a la base de datos
- recibir resultados de las consultas
- iterar sobre las filas devueltas
- ejecutar operaciones como INSERT, UPDATE, DELETE, CREATE TABLE

En otras palabras, es como una “pluma” que escribe y lee en la base de datos a través de la conexión.

A diferencia de R, Python separa estos dos conceptos:

- **conexión** (`conn`): maneja transacciones
- **cursor**: ejecuta consultas y maneja resultados

Cada cursor mantiene un “estado”

Cuando haces un SELECT, el cursor “apunta” a la posición actual dentro del conjunto de resultados.

Los cursores no necesitan cerrarse explícitamente, pero es buena práctica hacerlo:

```
cursor.close()
```

4.2.1 Operaciones de inserción, borrado o modificación de datos

- Igual que en R, se utiliza el método `execute` del objeto python
- Python inicia transacciones automáticamente en estas operaciones
- Por legibilidad normalmente las consultas se escriben en varias líneas. Para escribir cadenas multi-línea se usa `"""` en lugar de `"`.

```
cursor.execute("""
CREATE TABLE IF NOT EXISTS usuarios (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre TEXT NOT NULL,
    edad INTEGER
```

```

)
"""

cursor.executemany(
    "INSERT INTO usuarios (nombre, edad) VALUES (?, ?)",
    [("Luis", 25), ("María", 22), ("Pedro", 40)]
)

cursor.execute(
    "UPDATE usuarios SET edad = ? WHERE nombre = ?",
    (31, "Ana")
)

cursor.execute("DELETE FROM usuarios WHERE edad < ?", (30,))

conn.commit()

```

- Para ejecutar varias sentencias a la vez, en lugar del método `execute` se usa `executescript`

```

cursor.executescript("""
    INSERT INTO usuarios (nombre, edad) VALUES ('Roi', 20);
    INSERT INTO usuarios (nombre, edad) VALUES ('Sara', 33);
""")

conn.commit()

```

4.2.2 Operaciones de consulta

Cuando se ejecuta una consulta, el cursor apunta a la primera fila de resultado. Para obtener los datos hay 3 métodos útiles:

- `fetchone()`: Devuelve una fila cada vez. Cuando ya no quedan filas, devuelve `None`.
- `fetchall()`: Devuelve todas las filas como una lista. Si la consulta devuelve muchas filas, es mejor optar por una de las otras opciones.
- `fetchmany(n)`: Devuelve una lista con un bloque de como máximo `n` filas. Si quedan menos, devuelve un bloque menor. Si no quedan filas, devuelve una lista vacía `[]`.
- `fetchmany()`: Si no se utiliza el parámetro `n`, se usa el tamaño por defecto del cursor (`cursor.arraysize`), que si no se modifica tiene valor 1.

Ejemplo iterando sobre el cursor:

```

cursor.execute("SELECT * FROM Artist")
filas = cursor.fetchall()

fila = cursor.fetchone()
while fila is not None:
    print(fila)
    fila = cursor.fetchone()

```

Ejemplo iterando sobre los resultados:

```

cursor.execute("SELECT * FROM Artist")
filas = cursor.fetchall()

for fila in filas:
    print(fila)

```

4.2.2.1 Seleccionar una fila

```
cursor.execute("SELECT * FROM usuarios WHERE id = ?", (1,))
fila = cursor.fetchone()
print(fila)
```

El valor que devuelve es de tipo tupla. Es una **estructura inmutable**, rápida y ligera que garantiza que los datos no se modifiquen accidentalmente. Refleja una fila tal cual la devuelve SQLite.

Se puede acceder a los campos por índices:

```
id = fila[0]
nombre = fila[1]
edad = fila[2]
```

Se puede desempaquetar la tupla:

```
id, nombre, edad = fila
print(nombre)
```

Si la consulta devuelve un conjunto de tuplas, se puede recorrer en bucle:

```
cursor.execute("SELECT id, nombre, edad FROM usuarios")

for fila in cursor.fetchall():
    print(f"ID={fila[0]}, Nombre={fila[1]}, Edad={fila[2]}")
```

Se puede convertir en lista si se necesita modificar:

```
fila_lista = list(fila)
fila_lista[1] = "NuevoNombre"
```

O en diccionario para trabajar con mayor comodidad:

```
columnas = [desc[0] for desc in cursor.description]
dic = dict(zip(columnas, fila))
print(dic)
```

4.2.2.2 Devolver resultados como diccionarios

También se puede cambiar el tipo devuelto para que sea un diccionario en lugar de una tupla y evitar la conversión explícita:

```
conn.row_factory = sqlite3.Row
cursor = conn.cursor()

cursor.execute("SELECT * FROM usuarios")
for fila in cursor.fetchall():
    print(dict(fila))
```

De esta forma también se puede acceder a los campos de una forma más human-readable:

```
cursor.execute("SELECT * FROM usuarios")
for fila in cursor.fetchall():
    print(fila["nombre"])
```

4.2.3 Usar SQLite con Pandas

```
import pandas as pd

df = pd.read_sql_query("SELECT * FROM usuarios", conn)
print(df)
```

4.3 Context Manager

En Python existe una construcción muy útil llamada **context manager**, que se utiliza a través de la palabra clave **with**.

El objetivo de un context manager es **gestionar recursos de forma automática y segura**, garantizando que se liberen correctamente incluso si ocurre un error durante su uso.

Este mecanismo se basa en entrar en un “contexto” con `__enter__()` y salir de él con `__exit__()`, lo cual permite automatizar tareas como:

- cerrar archivos
- liberar conexiones
- realizar `commit` o `rollback` en bases de datos
- liberar bloqueos de hilos
- limpiar recursos temporales

Sin un context manager, el programador debe encargarse de abrir y cerrar recursos manualmente.

Por ejemplo, al trabajar con archivos:

```
f = open("datos.txt", "r")
contenido = f.read()
f.close()
```

Si ocurre un error entre `open()` y `close()`, el archivo podría quedar **abierto**, generando fugas de recursos o comportamientos inesperados.

El uso de `with` garantiza que el recurso se cierre automáticamente cuando termina el bloque, incluso si ocurre una excepción.

```
with open("datos.txt", "r") as f:
    contenido = f.read()
```

Aquí:

- se abre el archivo
- se ejecuta el bloque indentado
- al salir del bloque, Python llama automáticamente a `f.close()`

Esto hace que el código sea más **seguro**, **limpio** y **fácil de leer**.

```
with open("datos.txt", "w") as f:
    f.write("Ejemplo de context manager")
```

En este caso, `open()` devuelve un objeto archivo que actúa como context manager.

Al finalizar el bloque, Python ejecuta automáticamente el cierre del archivo.

4.3.1 Ejemplo: conexión SQLite

Las conexiones SQLite también son context managers.

El siguiente código garantiza un `commit()` si todo va bien, o un `rollback()` si hay errores:

```
import sqlite3

with sqlite3.connect("personas.sqlite") as conn:
    cursor = conn.cursor()
    cursor.execute("INSERT INTO usuarios(nombre) VALUES ('Ana')")
```

4.4 CSV en python

4.4.1 La librería csv

La librería `csv` viene incluida en Python, por lo que no requiere instalación.

4.4.1.1 Leer un CSV con `csv.reader()`

```
import csv

with open("datos.csv", newline='', encoding="utf-8") as f:
    reader = csv.reader(f)
    for fila in reader:
        print(fila)
```

4.4.1.2 Leer un CSV con cabeceras: `csv.DictReader()`

```
import csv

with open("datos.csv", newline='', encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for fila in reader:
        print(fila["Nombre"], fila["Edad"])
```

4.4.1.3 Escribir un CSV con `csv.writer()`

```
import csv

datos = [
    ["Nombre", "Edad"],
    ["Ana", 40],
    ["Diego", 41]
]

with open("salida.csv", "w", newline='', encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerows(datos)
```


Escribir un CSV con cabeceras: csv.DictWriter()


```
```python
```


```

```
import csv

personas = [
    {"Nombre": "Ana", "Edad": 40},
    {"Nombre": "Diego", "Edad": 41}
]

with open("salida.csv", "w", newline='', encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["Nombre", "Edad"])
    writer.writeheader()
    writer.writerows(personas)
```

4.4.2 Trabajar con CSV usando pandas

pandas es la librería más potente para análisis de datos en Python.

4.4.2.1 Leer un CSV

```
import pandas as pd

df = pd.read_csv("datos.csv")
print(df.head())
```

4.4.2.2 Guardar un CSV

```
df.to_csv("salida.csv", index=False)
```

4.4.2.3 Operaciones básicas

```
df["Nombre"]          # Acceder a una columna
df["Edad"].mean()     # Media de una columna numérica
df[df["Edad"] > 30]   # Filtrar filas
```

4.4.3 Insertar datos en SQLite a partir de CSV

```
with open('venues.csv') as csvfile:
    reader = csv.DictReader(csvfile)
    for row in reader:
        cursor.execute('''
            INSERT INTO Venue (Name, Capacity, Street, Number, City)
            VALUES (?, ?, ?, ?, ?)
            ''', (row['Name'], row['Capacity'], row['Street'], row['Number'], row['City']))
```

A menudo un CSV contiene nombres en vez de IDs, por ejemplo:

```
with open("concerts.csv") as f:
    reader = csv.DictReader(f)

    for row in reader:
        venue_name = row["Venue"].strip()

        # Buscar VenueId
```

```
cur.execute("SELECT VenueId FROM Venue WHERE Name = ?", (venue_name,))
result = cur.fetchone()

if result is None:
    print("ERROR: No existe el Venue:", venue_name)
    continue

venue_id = result[0]

# Insertar en la tabla Concert
cur.execute("""
    INSERT INTO Concert (ConcertId, VenueId, Name, Tickets, TicketsSold, Date)
    VALUES (?, ?, ?, ?, ?, ?)
""", (
    int(row["ConcertId"]),
    venue_id,
    row["Name"],
    int(row["Tickets"]),
    int(row["TicketsSold"]),
    row["Date"]
))
```

Capítulo 5

Manipulación de datos con tidyverse

En la primera parte de este capítulo (Sección 5.1), se pretende realizar una breve introducción al *ecosistema Tidyverse*, una colección de paquetes diseñados de forma uniforme (con la misma filosofía y estilo) para trabajar conjuntamente, y en particular al paquete `dplyr`, que resuelve las mismas tareas de lectura, filtrado, transformación y combinación de tablas vistas en el Capítulo 2, pero con una sintaxis más uniforme.

La referencia recomendada para usuarios de R que deseen iniciarse en el uso de estos paquetes es:

Wickham, H., y Grolemund, G. (2016). *R for data science: import, tidy, transform, visualize, and model data*, online-castellano, O'Reilly.

En las consecutivas secciones se presentan las alternativas *tidyverse* a la lectura, manipulación y escritura de datos tratadas en Capítulo 2.

Más adelante, en la Sección 5.2 se realiza una breve introducción al paquete `dplyr` y en la Sección 5.3 se comentan algunas de las utilidades del paquete `tidyr` que pueden resultar de interés¹. Finalmente, las secciones 5.4 y 5.5 se muestra las utilizadas para tratar tablas y bases de datos respectivamente.

5.1 Introducción al ecosistema tidyverse

El paquete `tidyverse` está diseñado para facilitar la instalación y carga de los paquetes principales de la colección tidyverse con un solo comando. Al instalar este paquete se instalan paquetes que forman el denominado núcleo de tidyverse (se cargan con `library(tidyverse)`):

- `ggplot2`: visualización de datos.
- `dplyr`: manipulación de datos.
- `tidyr`: reorganización (limpieza) de datos.
- `readr`: importación de datos.
- `tibble`: tablas de datos (extensión de `data.frame`).
- `purrr`: programación funcional.
- `stringr`: manipulación de cadenas de texto.
- `forcats`: manipulación de factores.
- `lubridate`: manipulación de fechas y horas.

¹ Otra alternativa (más rápida) es `data.table` pero en versiones recientes ya se puede emplear desde `dplyr`, como se comenta más adelante.

y un conjunto de paquetes recomendados:

- **feather**: almacenamiento eficiente de data frames. - **haven**: lectura y escritura de datos de SPSS, Stata y SAS en R - **modelr**: crear pipelines² elegantes al modelar datos en R (obsoleto). - **broom**: resúmenes estadísticos en formato Tidy.

Otros paquetes de interés son: - **readxl**: lectura de archivos Excel. - **writexl**: exportación a Excel. - **hms**: manipulación de medidas de tiempo. - **httr2**: interactuar con web APIs³. - **jsonlite**: Lectura y escritura de archivos JSON (*JavaScript Object Notation*). - **rvest**: extracción de datos (estructurados) de páginas web *web scraping*. - **xml2**: lectura y escritura de archivos XML. - **vroom**: lectura eficiente de archivos delimitados

```
library(tidyverse)
```

Al cargar **tidyverse** es habitual que aparezcan avisos de *conflictos*: algunas funciones de estos paquetes tienen el mismo nombre que funciones de R base o de otros paquetes ya cargados (por ejemplo, `dplyr::filter()` enmascara a `stats::filter()`). Si esto ocurre, la función del paquete cargado en último lugar es la que se usará por defecto; para evitar ambigüedad se puede especificar explícitamente el paquete, p. ej. `stats::filter()`.

También hay paquetes “asociados”:

- **rlang**: herramientas para programación funcional.
- **tidyselect** Sintaxis seleccionar variables (columnas).
- **tune**: hiperparámetros en modelos estadísticos
- **tidymodels** meta-paquete para todo el proceso de modelado (juega para el modelado predictivo el mismo papel que **tidyverts/fable** para las series temporales, ver más abajo).

Muchos otros paquetes están adaptando este estilo, por ejemplo, el meta paquete **tidyverts** para el análisis de series temporales (*time series*, TS), que incluye, por ejemplo:

- **tsibble** (infra)estructuras de datos.
- **fable** predicción (*forecasting*)⁴.
- **feasts** extracción de características (predictores).

Otro ejemplo, en este caso, para el tratamiento de datos espaciales, sería el paquete **sf**, para más detalles ver Sección 2.2 Introducción al paquete sf del libro **Estadística Espacial con R**

Resumiendo, está muy de moda y puede terminar convirtiéndose en un dialecto del lenguaje R, todo lo que resulte de utilidad es bien venido... Aunque se recomienda evitar estos paquetes en las primeras etapas de formación en R.

El estilo de programación tiene como origen la gramática de **ggplot2** para crear gráficos de forma declarativa, basado a su vez en:

Wilkinson, L. (2005). *The Grammar of Graphics*. Springer.

Este paquete se ha convertido en un sustituto de los gráficos **lattice**, de utilidad en algunos informes finales, aplicaciones para empresas, o para gráficos muy especializados. Aunque, en condiciones normales, suele ser más rápido generar o programar gráficos estándar de R.

² serie de pasos conectados (tuberías) que procesan datos y los transforman en un formato deseado para su análisis o modelado

³ Sustituye a **httr**, que se mantiene pero ya no recibe nuevas funcionalidades.

⁴ El libro **Forecasting: Principles and Practice** donde se describe su uso, y el paquete **fpp3** asociado, también siguen una filosofía *tidy*.

Para iniciarse en este paquete lo recomendado es consultar los capítulos Data Visualización y Graphics for communication de R for Data Science. También puede resultar de interés la chuleta). La referencia que cubre con mayor profundidad este paquete es:

Wickham, H. (2016). *ggplot2: Elegant graphics for Data Analysis* (3ª edición, en desarrollo junto a Navarro, D. y Pedersen, T.L.). Springer.

Otra alternativa sería:

Chang, W. (2023). *The R Graphics Cookbook*. O'Reilly.

En `ggplot2` se emplea el operador `+` para añadir componentes de los gráficos; en *Tidyverse* se emplea un operador de redirección (ver Sección 5.1.1) para añadir operaciones.

5.1.1 Operador *pipe* (redirección)

El operador `%>%` (paquete `magrittr`) permite canalizar la salida de una función a la entrada de otra. Se utiliza para mejorar la legibilidad y la claridad del código al encadenar múltiples operaciones en una secuencia fluida. Por ejemplo, `segundo(primeros(datos))` se traduce en `datos %>% primeros %>% segundo`, lo que facilita la lectura de operaciones al escribir las funciones de izquierda a derecha.

Desde la versión 4.1 de R está disponible un operador interno `|>`. Por ejemplo, para el conjunto de datos `empleados.RData` que contiene datos de empleados de un banco. Supongamos, por ejemplo, que estamos interesados en estudiar si hay discriminación por cuestión de sexo o raza.

```
load("data/empleados.RData")
# NOTA: Cuidado con la codificación latin1 (no declarada)
# al abrir archivos creados en versiones anteriores de R < 4.2:
# load("data/empleados.latin1.RData")

# Listamos las etiquetas
#knitr::kable(attr(empleados, "variable.labels"),
#             col.names = "Etiqueta")
# Eliminamos las etiquetas para que no molesten...
attr(empleados, "variable.labels") <- NULL

empleados |>
  subset(catlab == "Directivo", catlab:sexoraza) |>
  summary()
```

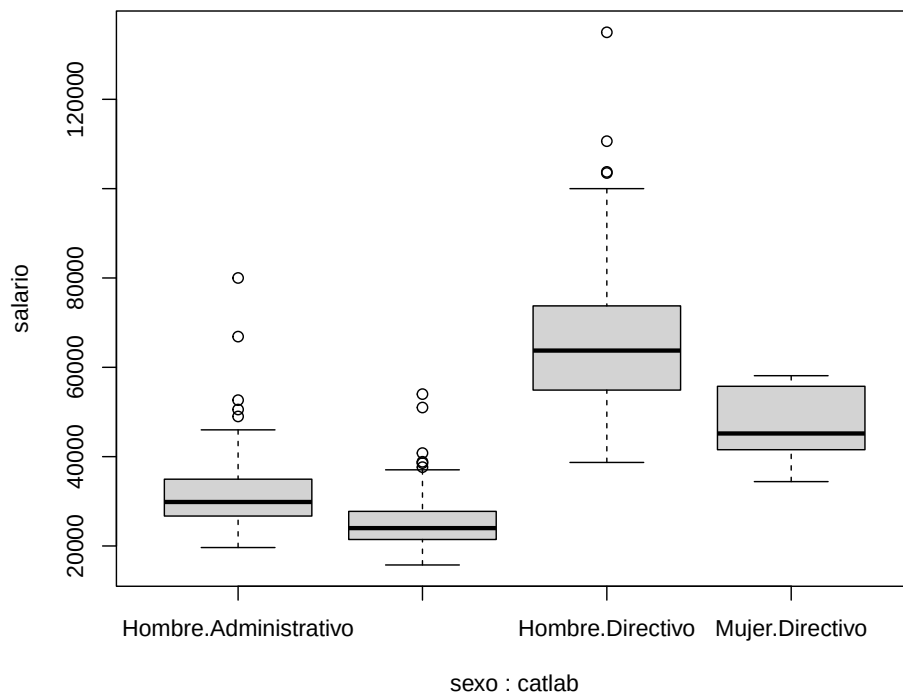
```
##           catlab      salario      salini      tiempemp
## Administrativo: 0   Min.    : 34410   Min.    :15750   Min.    :64.00
## Seguridad        : 0   1st Qu.: 51956   1st Qu.:23063   1st Qu.:73.00
## Directivo        :84   Median : 60500   Median :28740   Median :81.00
##                  Mean    : 63978   Mean    :30258   Mean    :81.15
##                  3rd Qu.: 71281   3rd Qu.:34058   3rd Qu.:91.00
##                  Max.    :135000   Max.    :79980   Max.    :98.00
##      expprev      minoria      sexoraza
## Min.    : 3.00   No      :80   Blanca varón :70
## 1st Qu.: 19.75   S\xed: 4   Minoría varón: 4
## Median : 52.00           Blanca mujer :10
## Mean    : 77.62           Minoría mujer: 0
## 3rd Qu.:125.25
## Max.    :285.00
```

Para que una función sea compatible con este tipo de operadores el primer parámetro debería ser siempre

los datos. Sin embargo, el operador `%>%` permite redirigir el resultado de la operación anterior a un parámetro distinto mediante un `..` Por ejemplo:

```
# ?"|>"
# empleados |> subset(catlab != "Seguridad") |> droplevels |>
#   boxplot(salario ~ sexo*catlab, data = .) # ERROR

library(magrittr)
empleados %>%
  subset(catlab != "Seguridad") %>%
  droplevels() %>%
  boxplot(salario ~ sexo*catlab, data = .)
```



5.1.2 Lectura y escritura de archivos de texto

En esta sección se presenta la alternativa *tidyverse* a la tradicional, vista en las secciones 2.1.3 y 2.1.6 del Capítulo 2.

Para leer archivos de texto en distintos formatos se puede emplear el paquete `readr`, disponible en la colección de paquetes *tidyverse*. Para más información, se recomienda consultar el Capítulo 11 del libro *R for Data Science* (Wickham et al., 2023) o la versión en español “*R Para Ciencia de Datos*”.

```
library(readr)
# ?readr
datos <- read_csv2("./data/coches.csv")
class(datos)
```

```
## [1] "spec_tbl_df" "tbl_df"      "tbl"         "data.frame"
```

También se puede importación desde Excel fácilmente:

```
library(readxl)
datos<-read_excel("./data/coches.xlsx")
class(datos)
```

```
## [1] "tbl_df"      "tbl"        "data.frame"
```

```
excel_sheets("./data/coches.xlsx") # listado de hojas
```

```
## [1] "coches"
```

Otra alternativa, sería emplear el paquete `data.table`. La función `fread()` puede considerarse como alternativa a `read_csv()` cuando el proceso de lectura resulta lento, especialmente con datos numéricos pesados. Esta función intenta *adivinar* automáticamente algunos argumentos sin tener que especificarse como, por ejemplo, el delimitador, las filas omitidas y la cabecera. Sin embargo, si requiere especificar el separador del decimal, como a continuación:

```
library(data.table)
# ?fread
datos <- fread(file = "./data/coches.csv", dec = ",")
class(datos)
```

```
## [1] "data.table" "data.frame"
```

Para más información, se recomienda ver la viñeta *Introduction to data.table*.

5.1.3 Escritura

Con el ecosistema *tidyverse*, también con el paquete `readr` se puede utilizar la función `write_csv2()`:

```
write_csv2(datos, file = "datos.csv")
```

y como opción más rápida, se podría usar `fwrite()` del paquete `data.table`:

```
# datos2 <- data.table(datos)
fwrite(datos2, file = "datos2.csv")
```

5.2 Manipulación de datos con dplyr y tidyr

En esta sección se realiza una breve introducción al paquete `dplyr` y se comentan algunas de las utilidades del paquete `tidyr` que pueden resultar de interés⁵.

La referencia recomendada para iniciarse en esta herramienta es el Capítulo 5 Data transformation de R for Data Science. También puede resultar de utilidad la viñeta del paquete *Introduction to dplyr* o la chuleta (menú de RStudio *Help > Cheat Sheets > Data Transformation with dplyr*).

5.2.1 El paquete dplyr

```
library(dplyr)
```

La principal ventaja de `dplyr` es que permite trabajar (de la misma forma) con datos en distintos formatos:

- `data.frame`, `tibble`.
- `data.table`: extensión (paquete *backend*) `dtplyr`.

⁵ Otra alternativa (más rápida) es `data.table` pero en versiones recientes ya se puede emplear desde `dplyr`, como se comenta más adelante.

- conjuntos de datos más grandes que la memoria disponible: extensiones **duckdb** y **arrow** (incluyendo almacenamiento en la nube, e.g. AWS).
- bases de datos relacionales (lenguaje SQL, locales o remotas); extensión **dbplyr**.
- grandes volúmenes de datos (incluso almacenados en múltiples servidores; ecosistema Hadoop/Spark): extensión **sparklyr** (ver menú de RStudio *Help > Cheat Sheets > Interfacing Spark with sparklyr*).

El paquete **dplyr** permite sustituir operaciones con funciones base de R (como **subset**, **split**, **apply**, **sapply**, **lapply**, **tapply**, **aggregate**...) por una “gramática” más sencilla para la manipulación de datos. En lugar de operar sobre vectores como la mayoría de las funciones base, opera sobre conjuntos de datos (de forma que es compatible con el operador **%>%**). Los principales “verbos” (funciones) son:

- **select()**: seleccionar variables (ver también **rename**, **relocate**, **pull**).
- **mutate()**: crear variables (ver también **transmute()**).
- **filter()**: seleccionar casos/filas (ver también **slice()**).
- **arrange()**: ordenar casos/filas.
- **summarise()**: resumir valores.
- **group_by()**: permite operaciones por grupo empleando el concepto “dividir-aplicar-combinar” (**ungroup()** elimina el agrupamiento).

NOTA: Para entender el funcionamiento de ciertas funciones (como **rowwise()**) y las posibilidades en el manejo de datos, hay que tener en cuenta que un **data.frame** no es más que una lista cuyas componentes (variables) tienen la misma longitud. Realmente las componentes también pueden ser listas de la misma longitud y, por tanto, podemos almacenar casi cualquier estructura de datos en un **data.frame**.

En la primera parte de este capítulo consideraremos solo **data.frame** por comodidad. Emplearemos como ejemplo los datos de empleados de banca almacenados en el fichero *empleados.RData* (y supondremos que estamos interesados en estudiar si hay discriminación por cuestión de sexo o raza).

```
load("data/empleados.RData")
attr(empleados, "variable.labels") <- NULL
```

En la Sección 5.5 final emplearemos una base de datos relacional como ejemplo.

5.2.2 Operaciones con variables (columnas)

Podemos seleccionar variables con **select()**:

```
emplea2 <- empleados %>% select(id, sexo, minoria, tiempemp,
                                salini, salario)
head(emplea2)
```

```
##   id  sexo minoria tiempemp salini salario
## 1  1 Hombre      No       98  27000  57000
## 2  2 Hombre      No       98  18750  40200
## 3  3 Mujer       No       98  12000  21450
## 4  4 Mujer       No       98  13200  21900
## 5  5 Hombre      No       98  21000  45000
## 6  6 Hombre      No       98  13500  32100
```

Se puede cambiar el nombre (ver también **rename()**):

```
empleados %>% select(sexo, noblanca = minoria, salario) %>% head()
```

```
##      sexo noblanca salario
## 1 Hombre      No   57000
## 2 Hombre      No   40200
## 3  Mujer      No   21450
## 4  Mujer      No   21900
## 5 Hombre      No   45000
## 6 Hombre      No   32100
```

Se pueden emplear los nombres de variables como índices:

```
empleados %>% select(sexo:salario) %>% head()
```

```
##      sexo   fechnac educ      catlab salario
## 1 Hombre 1952-02-03   15   Directivo  57000
## 2 Hombre 1958-05-23   16 Administrativo 40200
## 3  Mujer 1929-07-26   12 Administrativo 21450
## 4  Mujer 1947-04-15    8 Administrativo 21900
## 5 Hombre 1955-02-09   15 Administrativo 45000
## 6 Hombre 1958-08-22   15 Administrativo 32100
```

```
# empleados %>% select(-(sexo:salario)) %>% head()
empleados %>% select(!(sexo:salario)) %>% head()
```

```
##   id salini tiempemp expprev minoria  sexoraza
## 1  1  27000      98    144      No Blanca varón
## 2  2  18750      98     36      No Blanca varón
## 3  3  12000      98    381      No Blanca mujer
## 4  4  13200      98    190      No Blanca mujer
## 5  5  21000      98    138      No Blanca varón
## 6  6  13500      98     67      No Blanca varón
```

Se pueden emplear distintas herramientas (*selection helpers*) para seleccionar variables (ver paquete `tidyselect`):

- `starts_with`, `ends_with`, `contains`, `matches`, `num_range`: variables que coincidan con un patrón.
- `all_of`, `any_of`: variables de un vector de caracteres.
- `everything`, `last_col`: todas las variables o la última variable.
- `where()`: a partir de una función (e.g. `where(is.numeric)`)

Por ejemplo:

```
empleados %>% select(starts_with("s")) %>% head()
```

```
##      sexo salario salini  sexoraza
## 1 Hombre   57000  27000 Blanca varón
## 2 Hombre   40200  18750 Blanca varón
## 3  Mujer   21450  12000 Blanca mujer
## 4  Mujer   21900  13200 Blanca mujer
## 5 Hombre   45000  21000 Blanca varón
## 6 Hombre   32100  13500 Blanca varón
```

Podemos crear variables con `mutate()`:

```
emplea2 %>%
  mutate(incsal = salario - salini, tsal = incsal/tiempemp) %>%
  head()
```

```
##   id  sexo minoria tiempemp salini salario incsal      tsal
## 1  1 Hombre      No      98  27000  57000  30000 306.12245
## 2  2 Hombre      No      98  18750  40200  21450 218.87755
## 3  3 Mujer       No      98  12000  21450   9450  96.42857
## 4  4 Mujer       No      98  13200  21900   8700  88.77551
## 5  5 Hombre      No      98  21000  45000  24000 244.89796
## 6  6 Hombre      No      98  13500  32100  18600 189.79592
```

5.2.3 Operaciones con casos (filas)

Podemos seleccionar casos con `filter()`:

```
emplea2 %>% filter(sexo == "Mujer", minoria == "Sí") %>% head()
```

```
## [1] id      sexo      minoria tiempemp salini  salario
## <0 rows> (o 0- extensión row.names)
```

Podemos reordenar casos con `arrange()`:

```
emplea2 %>% arrange(salario) %>% head()
```

```
##   id  sexo minoria tiempemp salini salario
## 1 378 Mujer      No      70  10200  15750
## 2 338 Mujer      No      74  10200  15900
## 3  90 Mujer      No      92   9750  16200
## 4 224 Mujer      No      82  10200  16200
## 5 411 Mujer      No      68  10200  16200
## 6 448 Mujer S\xed     66  10200  16350
```

```
emplea2 %>% arrange(desc(salini), salario) %>% head()
```

```
##   id  sexo minoria tiempemp salini salario
## 1  29 Hombre      No      96  79980  135000
## 2 343 Hombre      No      73  60000  103500
## 3 205 Hombre      No      83  52500   66750
## 4 160 Hombre      No      86  47490   66000
## 5 431 Hombre      No      66  45000   86250
## 6  32 Hombre      No      96  45000  110625
```

Podemos resumir valores con `summarise()`:

```
empleados %>% summarise(sal.med = mean(salario), n = n())
```

```
##   sal.med  n
## 1 34419.57 474
```

Para realizar operaciones con múltiples variables podemos emplear `across()` (admite selección de variables tidyselect):

```
empleados %>% summarise(across(where(is.numeric), mean), n = n())
```

```
##   id      educ  salario  salini tiempemp expprev  n
## 1 237.5 13.49156 34419.57 17016.09 81.1097 95.86076 474
```

```
# empleados %>% summarise(across(where(is.numeric) & !id, mean), n = n())
```

NOTA: Esta función sustituye a las “variantes de ámbito” `_at()`, `_if()` y `_all()` de versiones anteriores de *dplyr* (como `summarise_at()`, `summarise_if()`, `summarise_all()`, `mutate_at()`, `mutate_if()`...) y también el uso de `vars()`. En el caso de `filter()` se puede emplear `if_any()` e `if_all()`.

Podemos agrupar casos con `group_by()`:

```
empleados %>% group_by(sexo, minoria) %>%
  summarise(sal.med = mean(salario), n = n()) %>%
  ungroup()
```

```
## # A tibble: 4 x 4
##   sexo  minoria sal.med    n
##   <fct> <fct>    <dbl> <int>
## 1 Hombre No      44475.   194
## 2 Hombre S       32246.    64
## 3 Mujer  No      26707.   176
## 4 Mujer  S       23062.    40
```

```
empleados %>% group_by(sexo, minoria) %>%
  summarise(sal.med = mean(salario), n = n(), .groups = "drop")
```

```
## # A tibble: 4 x 4
##   sexo  minoria sal.med    n
##   <fct> <fct>    <dbl> <int>
## 1 Hombre No      44475.   194
## 2 Hombre S       32246.    64
## 3 Mujer  No      26707.   176
## 4 Mujer  S       23062.    40
```

```
# dplyr >= 1.1.0 # packageVersion("dplyr")
# empleados %>% summarise(sal.med = mean(salario), n = n(),
#                          .by = c(sexo, minoria))
```

Por defecto la agrupación se mantiene para el resto de operaciones, habría que emplear `ungroup()` (o el argumento `.groups = "drop"`) para eliminarla (se puede emplear `group_vars()` o `str()` para ver la agrupación). Desde *dplyr* 1.1.0 (2023-01-29) está disponible un parámetro `.by/by` en `mutate()`, `summarise()`, `filter()` y `slice()` como alternativa a agrupar y desagrupar posteriormente. Para más detalles ver *Per-operation grouping with .by/by*.

5.2.4 Datos faltantes

Continuamos con el ejemplo de la Sección 2.2.3. *tidyverse* dispone de muchas herramientas para el tratamiento de los datos faltantes.

```
data("airquality")
datos <- airquality
library(visdat)
vis_dat(airquality)
# vis_miss(airquality)
```

Visualización (amigable) de la estructura de datos:

```
library(naniar)
bind_shadow(airquality)
```

```
## # A tibble: 153 x 12
##   Ozone Solar.R Wind Temp Month Day Ozone_NA Solar.R_NA Wind_NA Temp_NA
##   <int>   <int> <dbl> <int> <int> <int> <fct>   <fct>      <fct>   <fct>
## 1    41    190   7.4    67    5    1 !NA      !NA        !NA     !NA
## 2    36    118    8     72    5    2 !NA      !NA        !NA     !NA
## 3    12    149  12.6    74    5    3 !NA      !NA        !NA     !NA
## 4    18    313  11.5    62    5    4 !NA      !NA        !NA     !NA
## 5    NA     NA  14.3    56    5    5 NA        NA         !NA     !NA
## 6    28     NA  14.9    66    5    6 !NA      NA         !NA     !NA
## 7    23    299   8.6    65    5    7 !NA      !NA        !NA     !NA
## 8    19     99  13.8    59    5    8 !NA      !NA        !NA     !NA
## 9     8     19  20.1    61    5    9 !NA      !NA        !NA     !NA
## 10   NA    194   8.6    69    5   10 NA        !NA        !NA     !NA
## # i 143 more rows
## # i 2 more variables: Month_NA <fct>, Day_NA <fct>
```

```
# nabular(airquality)
```

Distribución por variables de los datos faltantes:

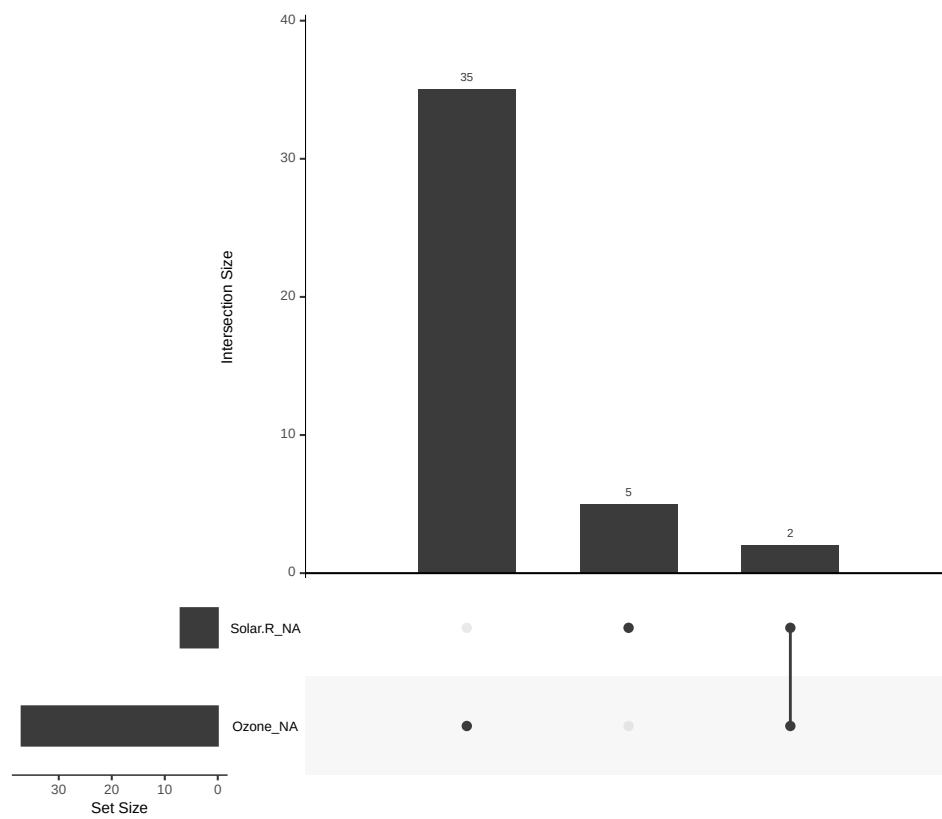
```
miss_var_table(airquality)
```

```
## # A tibble: 3 x 3
##   n_miss_in_var n_vars pct_vars
##           <int> <int>   <dbl>
## 1             0     4    66.7
## 2             7     1    16.7
## 3            37     1    16.7
```

```
prop_miss_case(airquality)
```

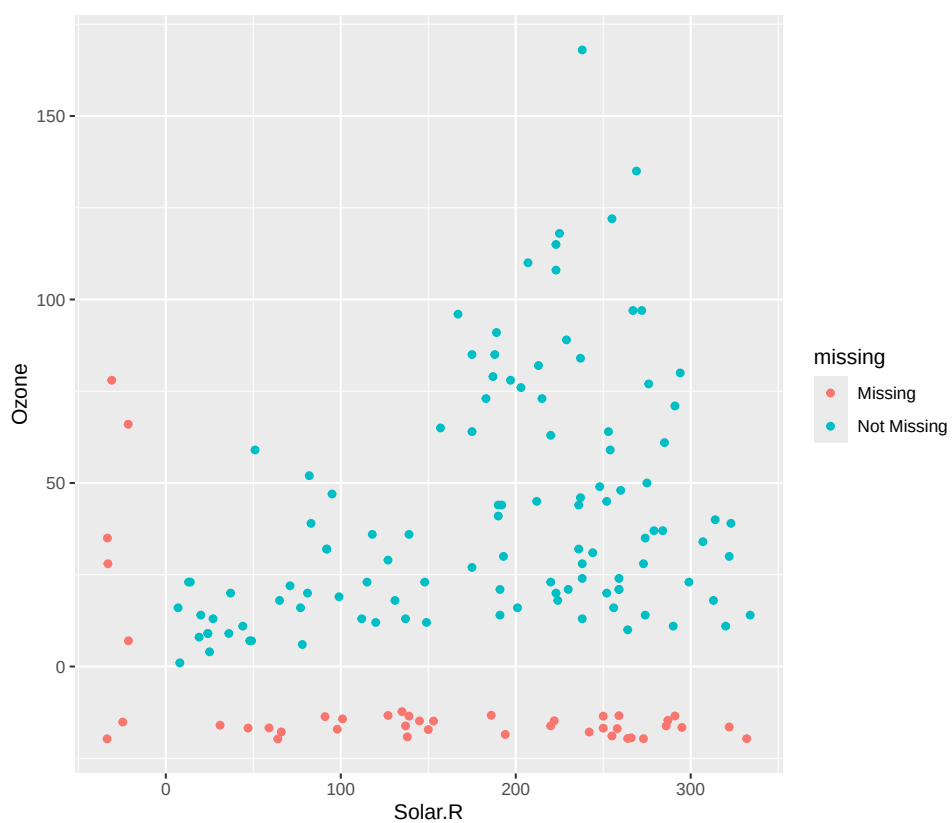
```
## [1] 0.2745098
```

```
gg_miss_upset(airquality)
```



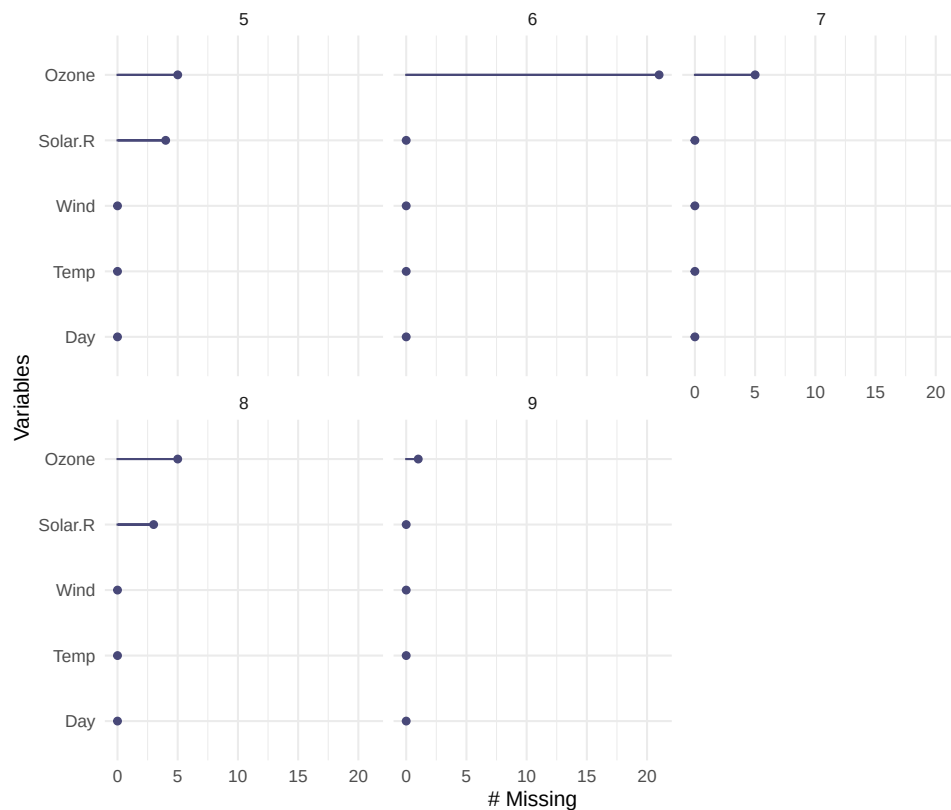
Distribución conjunta de los valores faltantes para la radiación solar y ozono:

```
library(naniar)
library(ggplot2)
ggplot(airquality,
       aes(x = Solar.R,
           y = Ozone)) +
  geom_miss_point()
```



Distribución mensual de los valores faltantes:

```
# gg_miss_var(airquality)
gg_miss_var(airquality, facet = Month)
```



Antes de decidir cómo tratar los valores faltantes conviene tener en cuenta por qué faltan, ya que no todos los mecanismos son igual de “inofensivos”:

- **MCAR** (*missing completely at random*): la probabilidad de que falte un valor no depende ni de los datos observados ni de los no observados (p. ej. un fallo aleatorio del sensor). Es el caso más favorable.
- **MAR** (*missing at random*): la probabilidad de que falte depende de otras variables observadas, pero no del propio valor ausente (p. ej. un sensor que falla más en ciertos meses).
- **MNAR** (*missing not at random*): la probabilidad de que falte depende del propio valor no observado (p. ej. valores muy altos de un contaminante que saturan y estropean el sensor). Es el caso más problemático: ignorar estos datos puede sesgar el análisis.

Una vez localizados y entendidos los valores faltantes, una opción sencilla es la **imputación**, es decir, sustituirlos por un valor “razonable” en lugar de eliminar la fila. Con `replace_na()` se puede imputar, por ejemplo, con la media de cada variable:

```
airquality %>%
  mutate(
    Ozone = tidyr::replace_na(as.numeric(Ozone), mean(Ozone, na.rm = TRUE)),
    Solar.R = tidyr::replace_na(as.numeric(Solar.R), mean(Solar.R, na.rm = TRUE))
  ) %>%
  summary()
```

| ## | Ozone | Solar.R | Wind | Temp |
|-------------|-------|---------------|----------------|---------------|
| ## Min. : | 1.00 | Min. : 7.0 | Min. : 1.700 | Min. :56.00 |
| ## 1st Qu.: | 21.00 | 1st Qu.:120.0 | 1st Qu.: 7.400 | 1st Qu.:72.00 |
| ## Median : | 42.13 | Median :194.0 | Median : 9.700 | Median :79.00 |
| ## Mean : | 42.13 | Mean :185.9 | Mean : 9.958 | Mean :77.88 |
| ## 3rd Qu.: | 46.00 | 3rd Qu.:256.0 | 3rd Qu.:11.500 | 3rd Qu.:85.00 |

```
## Max.    :168.00    Max.    :334.0    Max.    :20.700    Max.    :97.00
##      Month          Day
## Min.    :5.000    Min.    : 1.0
## 1st Qu.:6.000    1st Qu.: 8.0
## Median :7.000    Median :16.0
## Mean   :6.993    Mean   :15.8
## 3rd Qu.:8.000    3rd Qu.:23.0
## Max.    :9.000    Max.    :31.0
```

Esta imputación simple (con la media, la mediana o el último valor válido, `fill()`) es rápida pero **reduce artificialmente la variabilidad** de los datos y puede introducir sesgo si el mecanismo no es MCAR. Para conjuntos de datos más exigentes existen paquetes especializados en imputación múltiple, como `mice` o `missForest`, que quedan fuera del alcance de esta introducción.

5.3 Herramientas tidyr

Algunas funciones del paquete `tidyr` que pueden resultar de especial interés son:

- `pivot_wider()`: permite transformar valores de grupos de casos a nuevas variables.
- `pivot_longer()`: realiza la transformación inversa, colapsar varias columnas en una.

Ver la viñeta *Pivoting* para más detalles.

- `separate()`: permite separar una columna de texto en varias (ver también `extract()`).

Ver ejemplo *mortalidad.R*.

5.4 Operaciones con tablas de datos

Se emplean funciones `xxx_join()` (ver la documentación del paquete *Join two tbls together*, o la vignette *Two-table verbs*):

- `inner_join()`: devuelve las filas de `x` que tienen valores coincidentes en `y`, y todas las columnas de `x` e `y`. Si hay varias coincidencias entre `x` e `y`, se devuelven todas las combinaciones.
- `left_join()`: devuelve todas las filas de `x` y todas las columnas de `x` e `y`. Las filas de `x` sin correspondencia en `y` contendrán `NA` en las nuevas columnas. Si hay varias coincidencias entre `x` e `y`, se devuelven todas las combinaciones (duplicando las filas).

`right_join()` hace lo contrario, devuelve todas las filas de `y`.

`full_join()` devuelve todas las filas de `x` e `y` (duplicando o asignando `NA` si es necesario).

- `semi_join()`: devuelve las filas de `x` que tienen valores coincidentes en `y`, manteniendo sólo las columnas de `x` (al contrario que `inner_join()` no duplica filas).

`anti_join()` hace lo contrario, devuelve las filas sin correspondencia.

El parámetro `by` determina las variables clave para las correspondencias. Si no se establece se considerarán todas las que tengan el mismo nombre en ambas tablas. Se puede establecer a un vector de nombres coincidentes y en caso de que los nombres sean distintos a un vector con nombres de la forma `c("clave_x" = "clave_y")`.

Adicionalmente, si las tablas `x` e `y` tienen las mismas variables, se pueden combinar las observaciones con operaciones de conjuntos:

- `intersect(x, y)`: observaciones en `x` y en `y`.
- `union(x, y)`: observaciones en `x` o `y` no duplicadas.
- `setdiff(x, y)`: observaciones en `x` pero no en `y`.

5.5 Bases de datos con dplyr

Para poder usar tablas en bases de datos relacionales con `dplyr` hay que emplear el paquete `dbplyr` (convierte automáticamente el código de `dplyr` en consultas SQL).

Algunos enlaces:

- [Best Practices in Working with Databases](#)
- [Introduction to dbplyr](#)
- [Data Carpentry: SQL databases and R](#),
- [R and Data – When Should we Use Relational Databases?](#)

5.5.1 Ejemplos

Como ejemplo emplearemos la base de datos de SQLite Sample Database Tutorial, almacenada en el archivo `chinook.db`.

```
# install.packages('dbplyr')
library(dplyr)
library(dbplyr)
```

En primer lugar hay que conectar la base de datos:

```
chinook <- DBI::dbConnect(RSQLite::SQLite(), "data/chinook.db")
```

Podemos listar las tablas:

```
src_dbi(chinook)
```

```
## src: sqlite 3.53.3 [C:\Users\Manuel Oviedo\OneDrive - Universidade da Coruña\MTE\TXD\2026_2027\tgdboc
master\data\chinook.db]
## tbls: albums, artists, customers, employees, genres, invoice_items, invoices,
## media_types, playlist_track, playlists, sqlite_sequence, sqlite_stat1, tracks
```

Para enlazar una tabla:

```
invoices <- tbl(chinook, "invoices")
invoices
```

```
## # A query: ?? x 9
## # Database: sqlite 3.53.3 [C:\Users\Manuel Oviedo\OneDrive - Universidade da Coruña\MTE\TXD\2026_2027
master\data\chinook.db]
##   InvoiceId CustomerId InvoiceDate      BillingAddress BillingCity BillingState
##   <int>      <int> <chr>      <chr>      <chr>      <chr>
## 1         1         2 2009-01-01 00:0~ Theodor-Heuss~ Stuttgart    <NA>
## 2         2         4 2009-01-02 00:0~ Ullevålsveien~ Oslo         <NA>
## 3         3         8 2009-01-03 00:0~ Grétrystraat ~ Brussels    <NA>
## 4         4        14 2009-01-06 00:0~ 8210 111 ST NW Edmonton AB
## 5         5        23 2009-01-11 00:0~ 69 Salem Stre~ Boston      MA
## 6         6        37 2009-01-19 00:0~ Berger Straße~ Frankfurt    <NA>
```



```
# show_query(res)
res %>% collect
```

```
## # A tibble: 1 x 3
##   min    max    med
##   <dbl> <dbl> <dbl>
## 1  0.99  25.9  5.65
```

3. Para obtener el total de las facturas de cada uno de los países:

```
res <- invoices %>% group_by(BillingCountry) %>%
  summarise(n = n(), total = sum(Total, na.rm = TRUE))
# show_query(res)
res %>% collect
```

```
## # A tibble: 24 x 3
##   BillingCountry      n total
##   <chr>          <int> <dbl>
## 1 Argentina         7  37.6
## 2 Australia         7  37.6
## 3 Austria           7  42.6
## 4 Belgium           7  37.6
## 5 Brazil           35 190.
## 6 Canada            56 304.
## 7 Chile             7  46.6
## 8 Czech Republic   14  90.2
## 9 Denmark           7  37.6
## 10 Finland          7  41.6
## # i 14 more rows
```

4. Para obtener un listado con Nombre y Apellidos de cliente y el importe de cada una de sus facturas (Hint: WHERE customer.CustomerID=invoices.CustomerID):

```
customers <- tbl(chinook, "customers")
tbl_vars(customers)
```

```
## <dplyr::vars>
## [1] "CustomerId" "FirstName" "LastName" "Company" "Address"
## [6] "City" "State" "Country" "PostalCode" "Phone"
## [11] "Fax" "Email" "SupportRepId"
```

```
res <- customers %>%
  inner_join(invoices, by = "CustomerId") %>%
  select(FirstName, LastName, Country, Total)
show_query(res)
```

```
## <SQL>
## SELECT `FirstName`, `LastName`, `Country`, `Total`
## FROM `customers`
## INNER JOIN `invoices`
##   ON (`customers`.`CustomerId` = `invoices`.`CustomerId`)
```

```
res %>% collect
```

```
## # A tibble: 412 x 4
##   FirstName LastName Country Total
##   <chr>      <chr>    <chr>   <dbl>
## 1 Luís      Gonçalves Brazil    3.98
## 2 Luís      Gonçalves Brazil    3.96
```

```
## 3 Luís      Gonçalves Brazil  5.94
## 4 Luís      Gonçalves Brazil  0.99
## 5 Luís      Gonçalves Brazil  1.98
## 6 Luís      Gonçalves Brazil 13.9
## 7 Luís      Gonçalves Brazil  8.91
## 8 Leonie    Köhler    Germany 1.98
## 9 Leonie    Köhler    Germany 13.9
## 10 Leonie   Köhler    Germany  8.91
## # i 402 more rows
```

5. Para listar los 10 mejores clientes (aquellos a los que se les ha facturado más cantidad) indicando Nombre, Apellidos, País y el importe total de su facturación:

```
customers %>% inner_join(invoices, by = "CustomerId") %>% group_by(CustomerId) %>%
  summarise(FirstName, LastName, country, total = sum(Total, na.rm = TRUE)) %>%
  arrange(desc(total)) %>% head(10) %>% collect
```

6. Listar los 10 mejores clientes (aquellos a los que se les ha facturado más cantidad) indicando Nombre, Apellidos, País y el importe total de su facturación.

```
customers %>% inner_join(invoices, by = "CustomerId") %>% group_by(CustomerId) %>%
  summarise(FirstName, LastName, Country, total = sum(Total, na.rm = TRUE)) %>%
  arrange(desc(total)) %>% head(10) %>% collect
```

```
## # A tibble: 10 x 5
##   CustomerId FirstName LastName Country      total
##   <int> <chr>      <chr>    <chr>    <dbl>
## 1         6 Helena     Holý      Czech Republic 49.6
## 2        26 Richard   Cunningham USA          47.6
## 3        57 Luis      Rojas      Chile          46.6
## 4        45 Ladislav Kovács    Hungary          45.6
## 5        46 Hugh     O'Reilly  Ireland          45.6
## 6        24 Frank     Ralston   USA             43.6
## 7        28 Julia    Barnett   USA             43.6
## 8        37 Fynn     Zimmermann Germany          43.6
## 9         7 Astrid    Gruber    Austria          42.6
## 10       25 Victor    Stevens   USA             42.6
```

7. Listar los géneros musicales por orden decreciente de popularidad (definida la popularidad como el número de canciones de ese género), indicando el porcentaje de las canciones de ese género.

```
tracks <- tbl(chinook, "tracks")
tracks %>% inner_join(tbl(chinook, "genres"), by = "GenreId") %>% count(Name.y) %>%
  arrange(desc(n)) %>% collect %>% mutate(freq = n / sum(n))
```

```
## # A tibble: 25 x 3
##   Name.y          n  freq
##   <chr>        <int> <dbl>
## 1 Rock          1297 0.370
## 2 Latin          579 0.165
## 3 Metal          374 0.107
## 4 Alternative & Punk 332 0.0948
## 5 Jazz           130 0.0371
## 6 TV Shows         93 0.0265
## 7 Blues           81 0.0231
## 8 Classical        74 0.0211
## 9 Drama           64 0.0183
## 10 R&B/Soul        61 0.0174
```

```
## # i 15 more rows
```

8. Listar los 10 artistas con mayor número de canciones de forma descendente según el número de canciones.

```
tracks %>% inner_join(tbl(chinook, "albums"), by = "AlbumId") %>%
  inner_join(tbl(chinook, "artists"), by = "ArtistId") %>%
  count(Name.y) %>% arrange(desc(n)) %>% collect
```

```
## # A tibble: 204 x 2
##   Name.y          n
##   <chr>        <int>
## 1 Iron Maiden    213
## 2 U2             135
## 3 Led Zeppelin   114
## 4 Metallica      112
## 5 Lost           92
## 6 Deep Purple    92
## 7 Pearl Jam      67
## 8 Lenny Kravitz  57
## 9 Various Artists 56
## 10 The Office    53
## # i 194 more rows
```

Al finalizar hay que desconectar la base de datos:

```
DBI::dbDisconnect(chinook)
```

5.6 Eficiencia y desfases con dplyr

En la Sección 2.4 se calculó el desfase (*lag/lead*) de un vector desplazándolo “a mano” en R base. `dplyr` ofrece las funciones `lag()` y `lead()`, que hacen lo mismo con una sintaxis más legible (y son especialmente útiles dentro de `mutate()`, por ejemplo para calcular incrementos entre observaciones consecutivas):

```
a <- 1:11
dplyr::lag(a)
```

```
## [1] NA  1  2  3  4  5  6  7  8  9 10
```

```
dplyr::lead(a)
```

```
## [1]  2  3  4  5  6  7  8  9 10 11 NA
```

¿Tiene esta comodidad algún coste en rendimiento? Comparamos con `microbenchmark` frente al desplazamiento manual de la Sección 2.4:

```
library(microbenchmark)
x <- rnorm(1e6)
microbenchmark(
  base_r = c(NA, x[-length(x)]),
  dplyr_lag = dplyr::lag(x),
  times = 20
)
```

```
## Unit: milliseconds
##      expr      min       lq     mean   median      uq     max neval cld
##   base_r 5.9441 11.92335 30.46873 12.86035 15.08850 364.818   20   a
##  dplyr_lag 4.4052  9.99200 28.43778 11.80500 14.35645 350.438   20   a
```

`lag()`/`lead()` están vectorizadas internamente, por lo que su rendimiento es equiparable al del código base equivalente: la sintaxis más uniforme de `dplyr` no implica aquí una penalización relevante.

5.7 Resumen: dplyr/tidyr y R base

A modo de referencia rápida, la siguiente tabla recoge las tareas más habituales de manipulación de datos vistas en este capítulo, junto con su equivalente en R base (Capítulo 2):

| Tarea | dplyr / tidyr | R base |
|-----------------------------|---|---|
| Leer texto/CSV | <code>read_csv2()</code> , <code>read_delim()</code> (<code>readr</code>) | <code>read.table()</code> , <code>read.delim()</code> |
| Leer Excel | <code>read_excel()</code> (<code>readxl</code>) | <code>openxlsx::read.xlsx()</code> |
| Escribir texto/CSV | <code>write_csv2()</code> (<code>readr</code>) | <code>write.table()</code> |
| Seleccionar columnas | <code>select(df, a, b)</code> | <code>df[, c("a","b")]</code> |
| Renombrar columnas | <code>rename(df, x = antiguo)</code> | <code>names(df)[i] <- "x"</code> |
| Filtrar filas | <code>filter(df, cond)</code> | <code>subset(df, cond)</code> , <code>df[cond,]</code> |
| Ordenar filas | <code>arrange(df, x)</code> | <code>df[order(df\$x),]</code> |
| Crear/transformar variable | <code>mutate(df, z = expr)</code> | <code>df\$z <- expr</code> |
| Recodificar/categorizar | <code>mutate(df, z = case_when(...))</code> | <code>cut()</code> , <code>ifelse()</code> |
| Resumir | <code>summarise(df, ...)</code> | <code>aggregate()</code> , <code>tapply()</code> |
| Agrupar y resumir | <code>group_by(df, g) %>% summarise(...)</code> | <code>aggregate(y ~ g, df, FUN)</code> |
| Unir tablas (<i>join</i>) | <code>inner_join()</code> , <code>left_join()</code> , ... | <code>merge()</code> |
| Filtrar por coincidencia | <code>semi_join(df1, df2)</code> | <code>df1[df1\$x %in% df2\$x,]</code> |
| Encadenar operaciones | <code>%>%</code> / <code> ></code> | llamadas anidadas |

5.8 Ejercicios

Los siguientes ejercicios son los mismos que los planteados al final del Capítulo 2 (Sección 2.6), para resolverlos ahora empleando `dplyr`/`tidyr` en lugar de las funciones de R base. Los enunciados se dan sin solución; las soluciones se distribuyen aparte.

```
load("data/empleados.RData")
attr(empleados, "variable.labels") <- NULL
```

5.8.1 Ejercicio 1: incremento salarial (empleados)

A partir de los datos de `empleados` (Sección 5.1.2), calcula para cada empleado el incremento salarial relativo respecto a su salario inicial: `incremento = salario / salini - 1`. Muestra los 5 empleados con mayor incremento porcentual, junto con su categoría laboral (`catlab`) y su antigüedad en meses (`tiempemp`).

Para los siguientes ejercicios se empleará el conjunto de datos `starwars` (incluido en el propio paquete `dplyr`):

```
data(starwars, package = "dplyr")
```

Para el ejercicio de unión de tablas se empleará además la siguiente tabla, creada manualmente, con información de algunos planetas:

```
planetas <- tibble::tibble(
  homeworld = c("Tatooine", "Naboo", "Alderaan", "Coruscant", "Kamino",
                "Corellia", "Kashyyyk", "Ryloth", "Mirial", "Hoth"),
  region     = c("Outer Rim", "Mid Rim", "Core Worlds", "Core Worlds", "Outer Rim",
                "Core Worlds", "Mid Rim", "Outer Rim", "Outer Rim", "Outer Rim"),
  clima      = c("Desértico", "Templado", "Templado", "Urbano", "Oceánico",
                "Urbano", "Boscoso", "Húmedo", "Templado", "Helado")
)
```

5.8.2 Ejercicio 2: Filtrado y selección (starwars)

Obtén el nombre, la altura, el peso y el planeta natal (`homeworld`) de los personajes de especie (`species`) "Human" con una altura (`height`) superior a 180 cm.

5.8.3 Ejercicio 3: Creación de una variable y categorización

Calcula el índice de masa corporal ($IMC = \text{peso} / \text{altura}^2$, con la altura en metros) de cada personaje, descartando aquellos con valores faltantes en peso o altura. A continuación, categoriza el IMC empleando los umbrales estándar de la OMS: menos de 18.5 “Bajo peso”, de 18.5 a 25 “Normal”, de 25 a 30 “Sobrepeso” y 30 o más “Obesidad”. Indica cuántos personajes hay en cada categoría.

5.8.4 Ejercicio 4: Resumen y agrupación

Calcula la altura media y el número de personajes por especie, considerando únicamente las especies con al menos 3 personajes, y ordena el resultado de mayor a menor altura media.

5.8.5 Ejercicio 5: Unión de tablas

Une `starwars` con `planetas` por la columna `homeworld` y muestra, para cada personaje, su nombre, `homeworld`, región y clima. ¿Qué ocurre con los personajes cuyo planeta no aparece en `planetas` (o cuyo `homeworld` es NA)? ¿Y con "Hoth", que está en `planetas` pero no es el planeta natal de ningún personaje?

5.8.6 Ejercicio 6 (opcional): columnas-lista

`starwars` incluye columnas que son listas, como `films` (las películas en las que aparece cada personaje). Calcula en cuántas películas aparece cada personaje y muestra los que más aparecen.

Capítulo 6

Introducción a Tecnologías NoSQL

Son tecnologías de almacenamiento de datos en servicios web altamente escalables.

6.1 Conceptos y tipos de bases de datos NoSQL (documental, columnar, clave/valor y de grafos)

NoSQL - “Not Only SQL” - es una nueva categoría de bases de datos no-relacionales y altamente distribuidas.

Las bases de datos NoSQL nacen de la necesidad de:

- Simplicidad en los diseños
- Escalado horizontal
- Mayor control en la disponibilidad

Pero cuidado, en muchos escenarios las BBDD relacionales siguen siendo la mejor opción.

6.1.1 Características de las bases de datos NoSQL

- Libre de esquemas – no se diseñan las tablas y relaciones por adelantado, además de permitir la migración del esquema.
- Proporcionan replicación a través de escalado horizontal.
- Este escalado horizontal se traduce en arquitectura distribuida
- Generalmente ofrecen consistencia débil
- Hacen uso de estructuras de datos sencillas, normalmente pares clave/valor a bajo nivel
- Suelen tener un sistema de consultas propio (o SQL-like)
- Siguen el modelo BASE (*Basic Availability, Soft state, Eventual consistency*) en lugar de ACID (*Atomicity, Consistency, Isolation, Durability*)

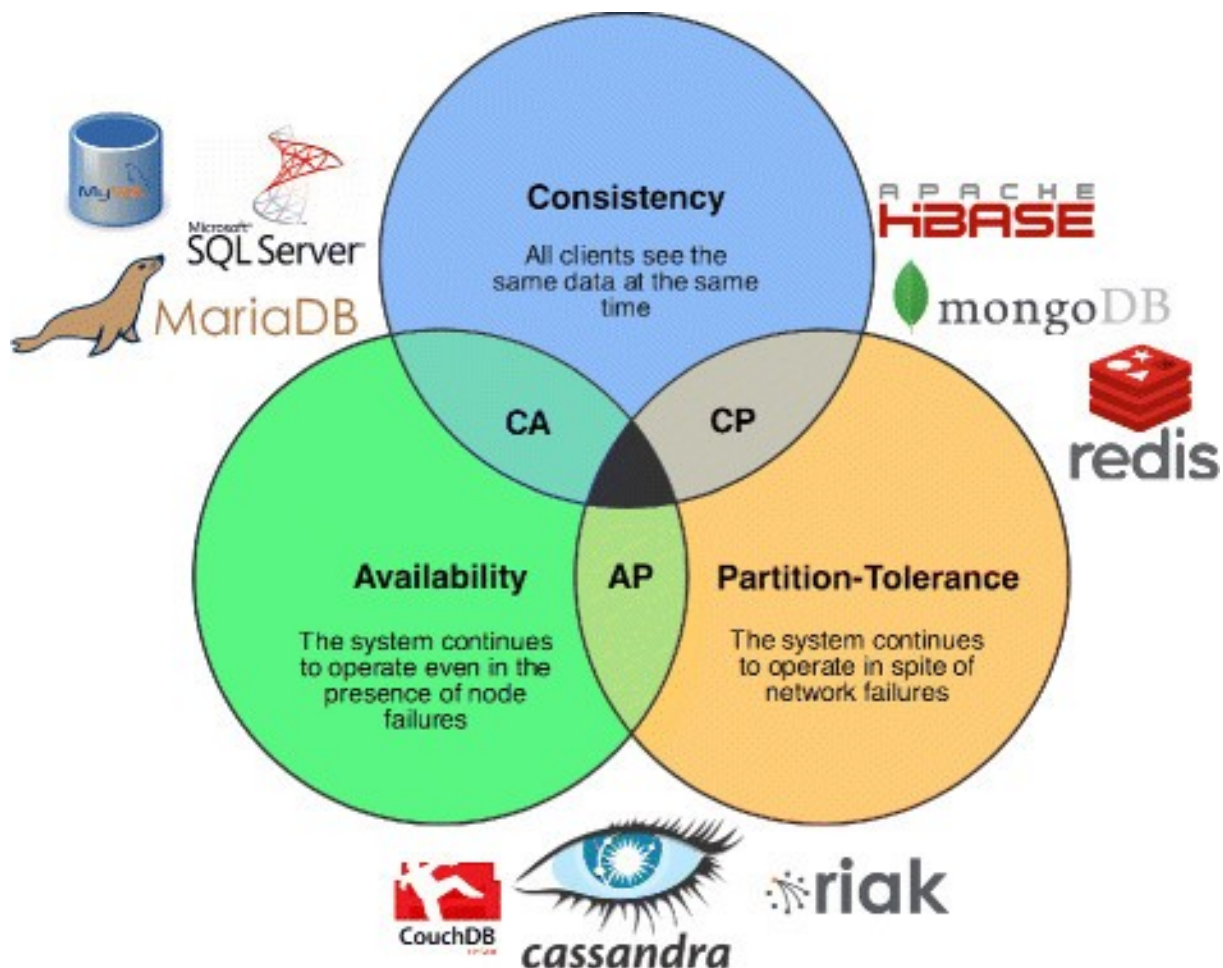
El modelo BASE consiste en:

- Basic Availability – el sistema garantiza disponibilidad, en términos del teorema CAP.
- Soft state – el estado del sistema puede cambiar a lo largo del tiempo, incluso sin entrada. Esto es provocado por el modelo de consistencia eventual.
- Eventual consistency – el sistema alcanzará un estado consistente con el tiempo, siempre y cuando no reciba entrada durante ese tiempo.

6.1.1.1 Teorema CAP

Es imposible para un sistema de cómputo distribuido garantizar simultáneamente:

- Consistency – Todos los nodos ven los mismos datos al mismo tiempo
- Availability – Toda petición obtiene una respuesta en caso tanto de éxito como fallo
- Partition Tolerance – El sistema seguirá funcionando ante pérdidas arbitrarias de información o fallos parciales



Las razones para escoger NoSQL son:

- Analítica
- Gran cantidad de escrituras, análisis en bloque
- Escalabilidad
- Tan fácil como añadir un nuevo nodo a la red, bajo coste.
- Redundancia
- Están diseñadas teniendo en cuenta la redundancia
- Rápido desarrollo
- Al ser schema-less o schema on-read son más flexibles que schema on-write
- Flexibilidad en el almacenamiento de datos
- Almacenan todo tipo de datos: texto, imágenes, BLOBs
- Gran rendimiento en consultas sobre datos que no implican relaciones jerárquicas
- Gran rendimiento sobre BBDD desnormalizadas
- Tamaño

- El tamaño del esquema de datos es demasiado grande
- Muchos datos temporales fuera de almacén principal

Razones para NO escoger NoSQL: * Consistencia y Disponibilidad de los datos son críticas * Relaciones entre datos son importantes + E.g. joins numerosos y/o importantes * En general, cuando el modelo ACID encaja mejor

6.1.2 Tipos de Bases de Datos NoSQL

Columnar:

| | | | | | | | |
|---|-----------|---|-----|---|------|--------|------------|
| 1 | Things | A | foo | B | bar | C | baz |
| 2 | Things | C | bam | E | coh | People | A Emmanuel |
| 3 | Languages | A | C | B | java | C | Ceylon |

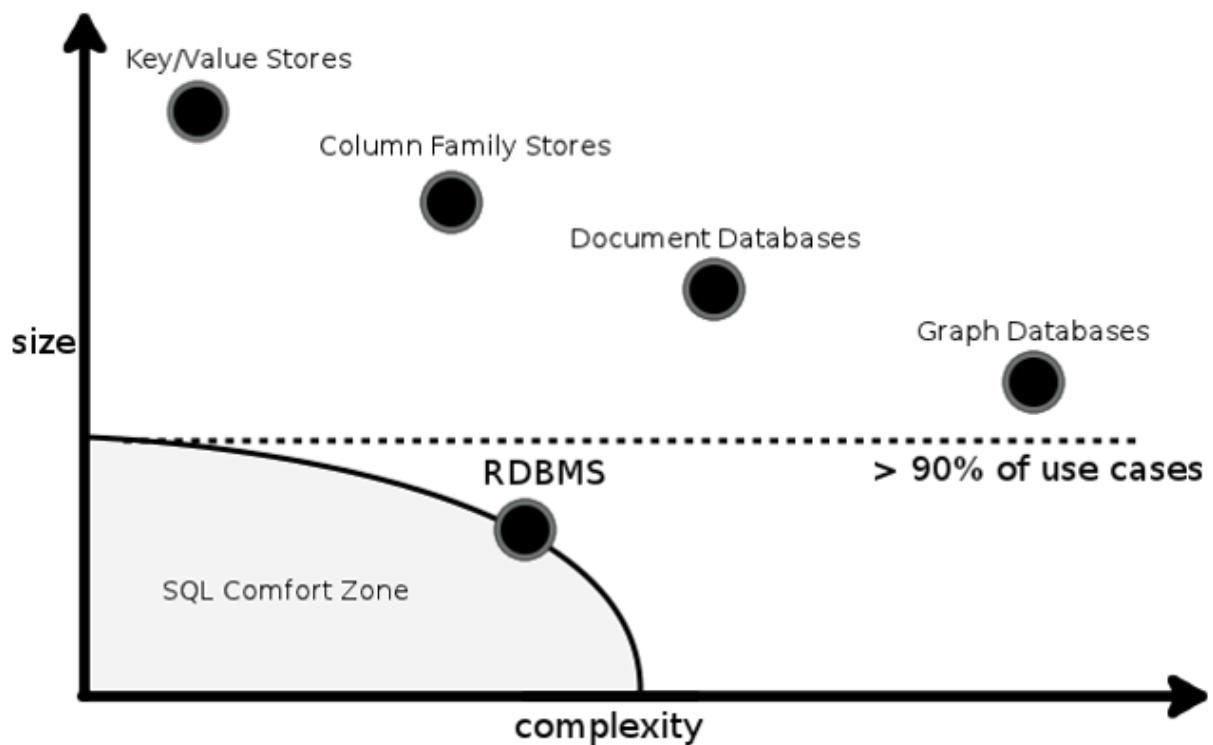
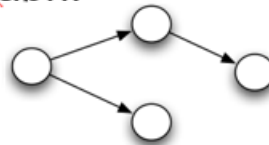
Documental:

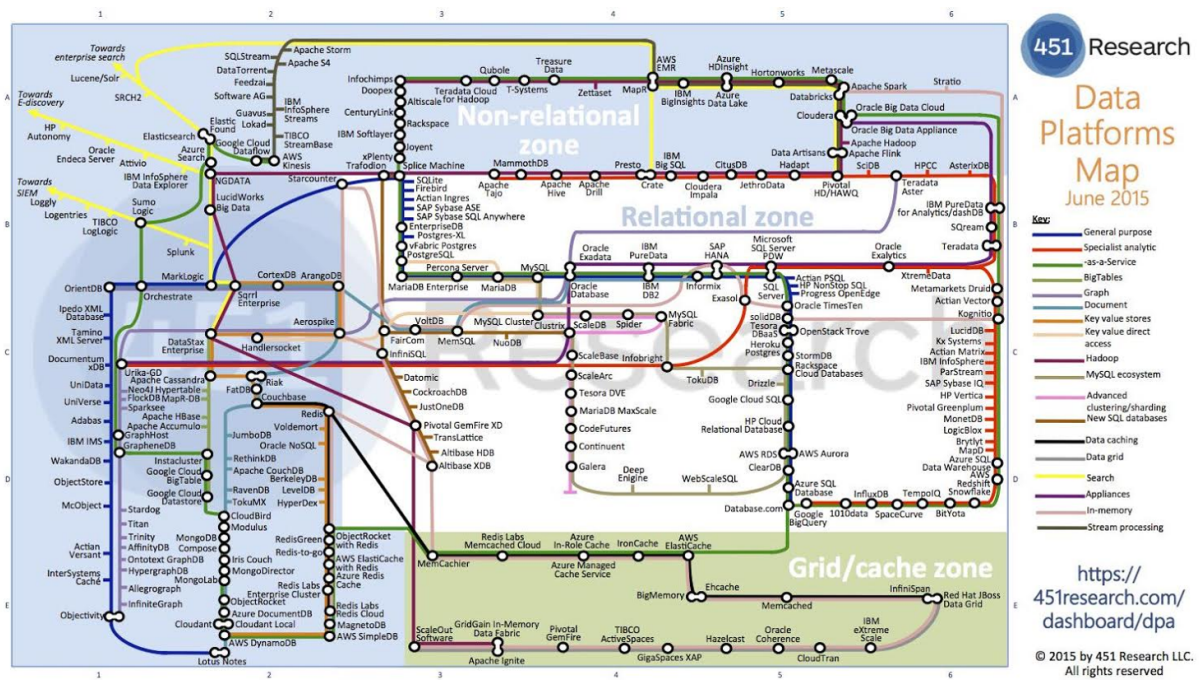
```
{ "user" : {
  "id": "124",
  "name": "Emmanuel",
  "addresses" : [
    { "city": "Paris", "country": "France" },
    { "city": "Atlanta", "country": "USA" }
  ]
}
```

Key-value:

| key | value |
|-----|------------|
| 123 | Address@23 |
| 126 | "Booya" |

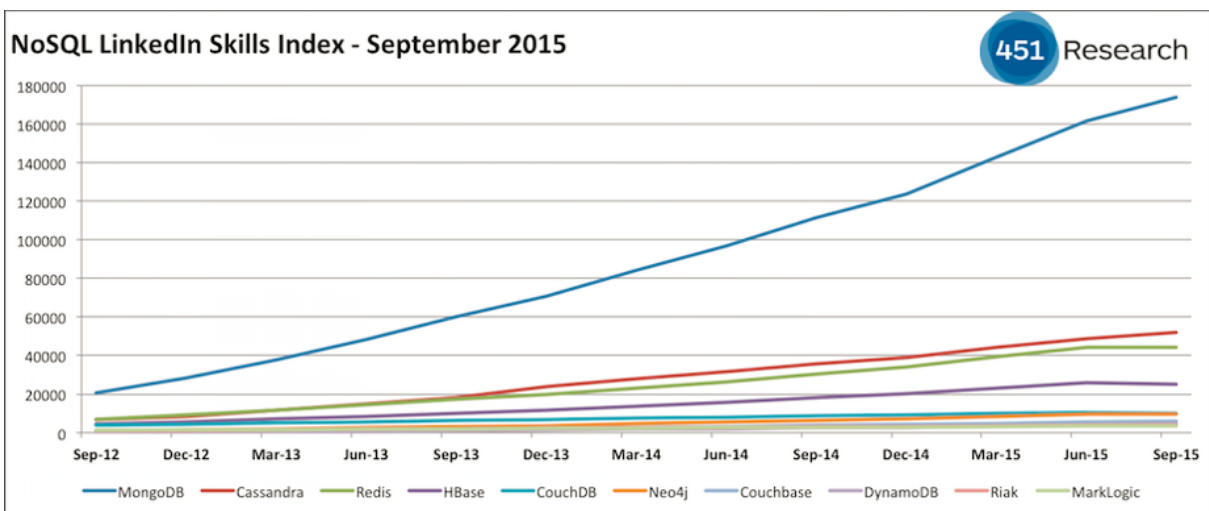
Graph:





352 systems in ranking, September 2019

| Rank | Rank | | | DBMS | Database Model | Score | | |
|------|----------|----------|----------|------------------------|----------------------------|----------|----------|----------|
| | Sep 2019 | Aug 2019 | Sep 2018 | | | Sep 2019 | Aug 2019 | Sep 2018 |
| 1. | 1. | 1. | 1. | Oracle + | Relational, Multi-model | 1346.66 | +7.18 | +37.54 |
| 2. | 2. | 2. | 2. | MySQL + | Relational, Multi-model | 1279.07 | +25.39 | +98.60 |
| 3. | 3. | 3. | 3. | Microsoft SQL Server + | Relational, Multi-model | 1085.06 | -8.12 | +33.78 |
| 4. | 4. | 4. | 4. | PostgreSQL + | Relational, Multi-model | 482.25 | +0.91 | +75.82 |
| 5. | 5. | 5. | 5. | MongoDB + | Document | 410.06 | +5.50 | +51.27 |
| 6. | 6. | 6. | 6. | IBM Db2 + | Relational, Multi-model | 171.56 | -1.39 | -9.50 |
| 7. | 7. | 7. | 7. | Elasticsearch + | Search engine, Multi-model | 149.27 | +0.19 | +6.67 |
| 8. | 8. | 8. | 8. | Redis + | Key-value, Multi-model | 141.90 | -2.18 | +0.96 |
| 9. | 9. | 9. | 9. | Microsoft Access | Relational | 132.71 | -2.63 | -0.69 |
| 10. | 10. | 10. | 10. | Cassandra + | Wide column | 123.40 | -1.81 | +3.85 |



| | |
|--------------|---|
| Rich Queries | <ul style="list-style-type: none"> Find Paul's cars Find everybody in London with a car built between 1970 and 1980 |
| Geospatial | <ul style="list-style-type: none"> Find all of the car owners within 5km of Trafalgar Sq. |
| Text Search | <ul style="list-style-type: none"> Find all the cars described as having leather seats |
| Aggregation | <ul style="list-style-type: none"> Calculate the average value of Paul's car collection |
| Map Reduce | <ul style="list-style-type: none"> What is the ownership pattern of colors by geography over time? (is purple trending up in China?) |

6

MongoDB

```
{
  first_name: 'Paul',
  surname: 'Miller',
  city: 'London',
  location:
[45.123,47.232],
  cars: [
    { model: 'Bentley',
      year: 1973,
      value: 100000, ... },
    { model: 'Rolls Royce',
      year: 1965,
      value: 330000, ... }
  ]
}
```

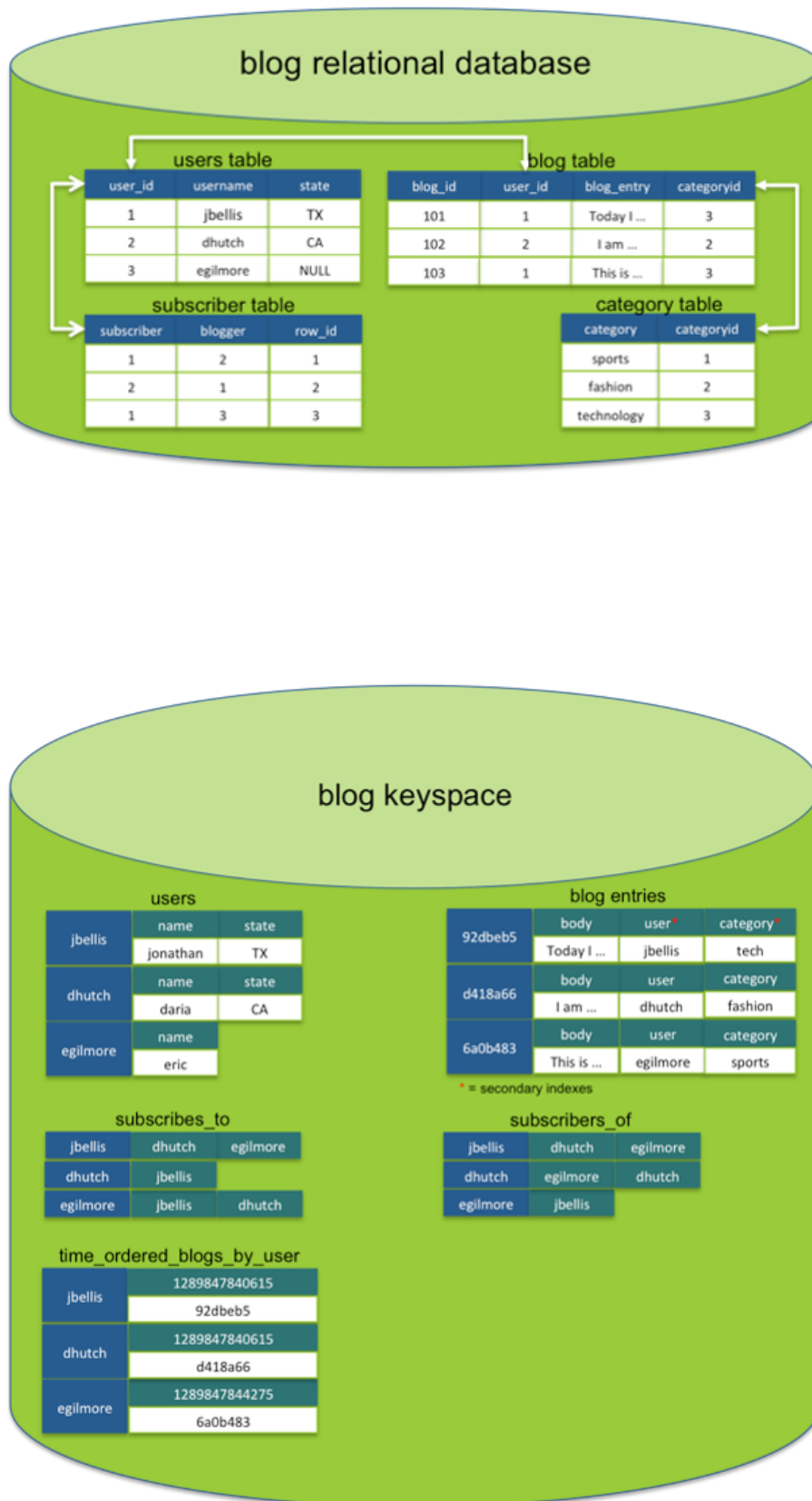


6.1.4 Redis: NoSQL key-value

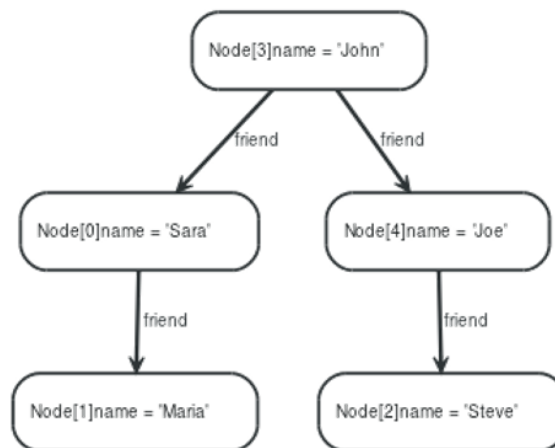
In-memory data structure store, útil para base de datos de login-password, sensor-valor, URL-respuesta, con una sintaxis muy sencilla:

- El comando SET almacena valores
- SET server:name "luna"
- Recuperamos esos valores con GET
- GET server:name
- INCR incrementa atómicamente un valor
- INCR clients
- DEL elimina claves y sus valores asociados
- DEL clients
- TTL (Time To Live) útil para cachés
- EXPIRE promocion 60

6.1.5 Cassandra: NoSQL columnar



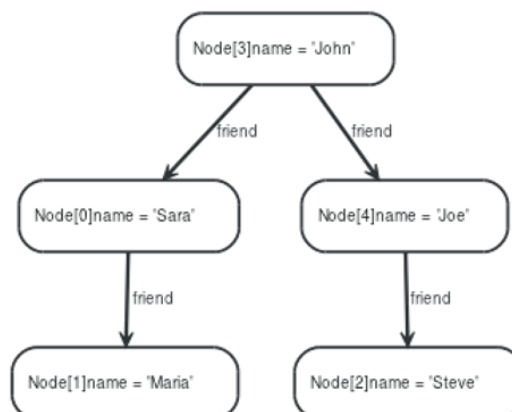
6.1.6 Neo4j: NoSQL grafos



```

MATCH (john {name: 'John'})-[:friend]->()-[:friend]->(fof)
RETURN john, fof

```



| john | fof |
|----------------------|-----------------------|
| Node[3]{name:"John"} | Node[1]{name:"Maria"} |
| Node[3]{name:"John"} | Node[2]{name:"Steve"} |
| 2 rows | |

6.1.7 Otros: search engines

Son sistemas especializados en búsquedas, procesamiento de lenguaje natural como ElasticSearch, Solr, Splunk (logs de aplicaciones), etc...

6.2 Conexión de R a MongoDB

A través del paquete mongolite, aquí tenéis un Tutorial

```
install.packages("mongolite")
```

```
library(mongolite)
```

```
# Connect to a local MongoDB
```

```
my_collection = mongo(collection = "restaurants", db = "Restaurants") # create connection, database  
my_collection$count
```

6.3 Ejercicios prácticos con MongoDB

Estos ejercicios se pueden hacer en un notebook Kaggle accediendo a un clúster de MongoDB en el cloud de MongoDB. Se carga la base de datos de ejemplo y se puede hacer con la colección de restaurantes (o alternativamente con otras colecciones).

1. Mostrar todos los documentos de la colección restaurants (u otra)
2. Mostrar nombre de restaurante, barrio y cocina de la colección restaurants (o los campos de otra colección)
3. Mostrar los primeros 5 restaurantes del barrio Bronx.
4. Mostrar los restaurantes con una longitud menor que -75.7541
5. Mostrar los restaurantes con una puntuación superior a 90
6. Mostrar los restaurantes de comida American o Chinese del barrio Queens.
7. Mostrar los restaurantes con un grado "A" y puntuación 9 obtenida en fecha 2014-08-11T00:00:00Z
8. Propón un JSON para descargar (de algún repositorio OpenData o disponible en un API), indícame la URL, si has de hacer algún proceso antes de importarlo en MongoDB, cómo lo importas, dame un pantallazo del análisis exploratorio de ese JSON y una query que harías contra ese JSON (la query en MongoDB, Compass o Rmongodb)

Capítulo 7

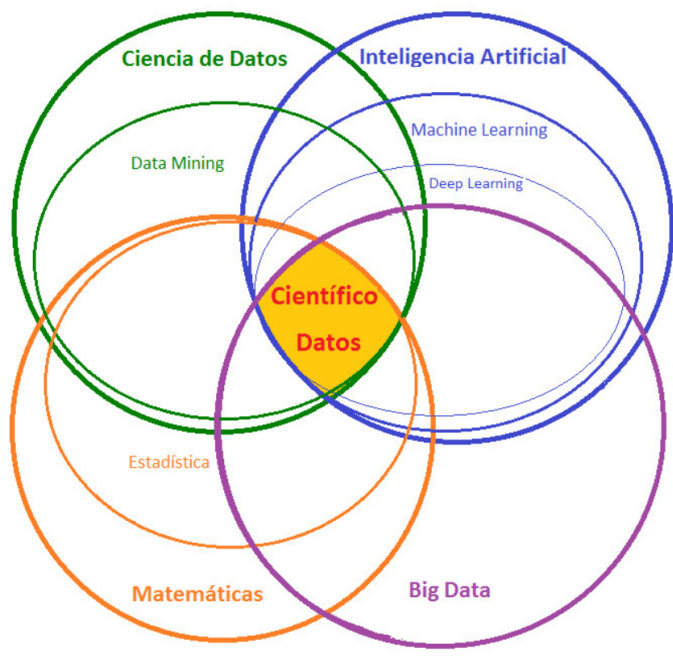
Tecnologías para el Tratamiento de Datos Masivos

En este apartado trataremos los siguientes epígrafes:

1. Introducción al Aprendizaje Estadístico
2. Tecnologías Big Data (Hadoop, Spark, Sparklyr)
3. Ejercicios de análisis de datos masivos.

7.1 Introducción al Aprendizaje Estadístico

Este apartado es una adaptación (reducida, pensada para 2 sesiones de 2 horas) del Capítulo 1 de Fernández-Casal, Costa y Oviedo, *Métodos predictivos de aprendizaje estadístico*, que se sigue empleando como referencia completa para quien quiera profundizar (incluye, entre otros contenidos que aquí solo se tratarán brevemente por falta de tiempo, un desarrollo más completo de la maldición de la dimensionalidad y del paquete `caret`).



La denominada **ciencia de datos** (*data science*) se ha vuelto muy popular hoy en día. Se trata de un campo multidisciplinar, con importantes aportaciones estadísticas e informáticas, dentro del que se incluyen disciplinas como **minería de datos** (*data mining*), **aprendizaje automático** (*machine learning*), **aprendizaje profundo** (*deep learning*), **modelado predictivo** (*predictive modeling*), **extracción de conocimiento** (*knowledge discovery*) y también el **aprendizaje estadístico** (*statistical learning*). Puede resumirse en el diagrama de Venn de la ciencia de datos de Drew Conway (Figura 7.1).

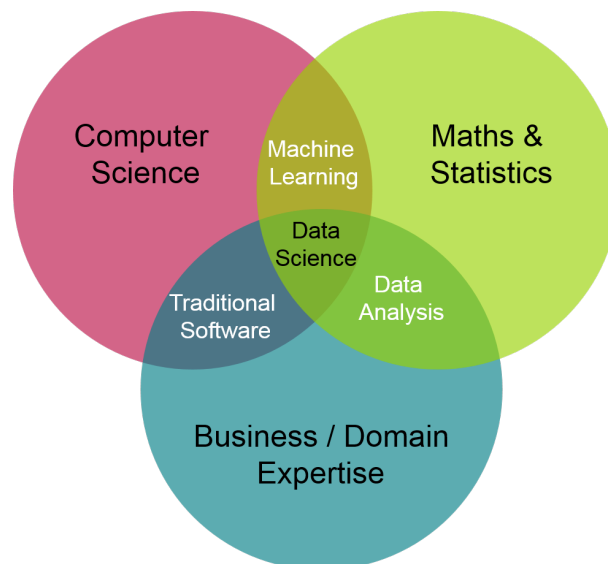


Figura 7.1: Diagrama de Venn de la ciencia de datos

Podemos definir la ciencia de datos como el conjunto de conocimientos y herramientas utilizados en las distintas etapas del análisis de datos (Figura 2.1), incluyendo la gestión, obtención y manipulación de los datos (ver Capítulo 2).

Una de estas etapas es la construcción de modelos, a partir de los datos, para aprender y predecir.

El **aprendizaje estadístico** (AE) se encarga de este problema desde un punto de vista estadístico: se consideran modelos estocásticos (con componente aleatoria) para tener en cuenta la incertidumbre debida a no disponer de toda la información sobre las variables que influyen en el fenómeno de interés.

En la inferencia estadística clásica se trata de explicar por completo lo que ocurre en la población y, suponiendo que esto se puede hacer con modelos tratables analíticamente, emplear resultados teóricos (típicamente asintóticos) para realizar inferencias, incluida la predicción. Los avances en computación han permitido el uso de modelos estadísticos más avanzados, principalmente métodos no paramétricos, muchos de los cuales no se pueden tratar analíticamente (o no por completo): este es el campo de la **estadística computacional**, en el que se enmarcaría el AE.

Cuando pensamos en AE, pensamos en:

- **Flexibilidad:** se tratan de obtener las conclusiones basándose únicamente en los datos, evitando asumir hipótesis para poder emplear resultados teóricos. La idea es “dejar hablar” a los datos, no “encorsetarlos” a priori.
- **Procesamiento automático de datos:** el proceso de aprendizaje puede realizarse con la menor intervención interactiva posible por parte del analista.
- **Big data:** en sentido amplio, no solo tamaño muestral, también datos complejos o con necesidad de alta velocidad de proceso.
- **Predicción:** el objetivo (inicial) suele ser únicamente la predicción de nuevas observaciones, con métodos que son simples algoritmos.

Por el contrario, muchos de los métodos del AE no se preocupan (o se preocupan poco) por:

- **Reproducibilidad:** pequeños cambios en los datos pueden producir cambios notables en el modelo ajustado, aunque no deberían influir mucho en las predicciones. Muchas técnicas son además aleatorias, por lo que el resultado puede depender de la semilla empleada.
- **Cuantificación de la incertidumbre:** se obtienen medidas globales de la eficiencia del algoritmo, pero resulta complicado cuantificar la precisión de una predicción concreta.
- **Inferencia:** aparte de la predicción, la mayoría de los métodos no permiten realizar inferencias sobre características de la población (como contrastes de hipótesis).

Además, esta aproximación puede presentar diversos inconvenientes: algunos métodos son poco interpretables (“cajas negras”), pueden aparecer problemas de sobreajuste (*overfitting*, ver Sección 7.1.3) y pueden presentar más problemas al extrapolar o interpolar que los métodos clásicos (cuanto más flexible el algoritmo, más cuidado hay que tener al predecir en observaciones alejadas de la muestra empleada en el ajuste).

7.1.1 Aprendizaje estadístico vs. aprendizaje automático

El término *machine learning* (ML; aprendizaje automático) se utiliza en *inteligencia artificial* desde 1959 para referirse, fundamentalmente, a algoritmos de predicción. Muchas de sus herramientas provienen de la estadística, que es la base de todos estos enfoques (conviene no perder la base formal). Por ello, desde la estadística computacional se introdujo el término *statistical learning* (aprendizaje estadístico) para referirse a este tipo de herramientas, pero desde el punto de vista estadístico (teniendo en cuenta la incertidumbre debida a no disponer de toda la información).

Tradicionalmente, ML no se preocupa del origen de los datos; incluso es habitual considerar que un conjunto enorme de datos equivale a disponer de toda la información (la población). Por el contrario,

en AE se trata de comprender, en la medida de lo posible, el proceso subyacente del que provienen los datos y si son representativos de la población de interés (si tienen algún tipo de sesgo, especialmente de selección).

7.1.1.1 Las dos culturas

Breiman (2001), en su célebre artículo *Statistical modeling: the two cultures*, diferencia dos objetivos en el análisis de datos, *información* (en el sentido de inferencia) y *predicción*, que dan lugar a dos culturas:

- *Modelización de datos*: desarrollo de modelos (estocásticos) que permitan ajustar los datos y hacer inferencia. Es el trabajo habitual de los estadísticos académicos.
- *Modelización algorítmica* (predictiva): no está interesada en los mecanismos que generan los datos, solo en los algoritmos de predicción. Es el trabajo habitual de muchos estadísticos industriales y de muchos ingenieros informáticos. El ML es el núcleo de esta cultura, que pone todo el énfasis en la precisión predictiva (de ahí el papel dinamizador de competiciones entre algoritmos, al estilo del Netflix Challenge).

7.1.2 Métodos de aprendizaje estadístico

Dentro del AE se suelen diferenciar dos grandes bloques. El **aprendizaje no supervisado** comprende los métodos exploratorios, en los que no hay una variable respuesta explícita, y cuyo objetivo es entender las relaciones y estructuras presentes en los datos: análisis descriptivo, métodos de reducción de la dimensión (componentes principales, análisis factorial...), métodos de agrupación (análisis clúster) y detección de datos atípicos.

El **aprendizaje supervisado** engloba los métodos predictivos, en los que una de las variables está definida como variable respuesta, y cuyo objetivo es construir modelos que se utilizarán sobre todo para predecir. Dependiendo del tipo de variable respuesta se diferencia entre:

- **Clasificación**: si la respuesta es categórica.
- **Regresión**: cuando la respuesta es numérica.

En lo que sigue nos centraremos en el aprendizaje supervisado, combinando terminología estadística con la empleada en AE/ML.

7.1.2.1 Notación y terminología

Denotaremos por $\mathbf{X} = (X_1, X_2, \dots, X_p)$ al vector de variables predictoras (explicativas o independientes; *inputs* o *features* en ML), cada una numérica o categórica. Emplearemos $Y(\mathbf{X})$ para la variable objetivo (respuesta o dependiente; *output* o *target* en ML), numérica (regresión) o categórica (clasificación).

Supondremos que el objetivo principal es, a partir de una muestra $\{(y_i, x_{1i}, \dots, x_{pi}) : i = 1, \dots, n\}$, ajustar en regresión un modelo general de la forma (posiblemente tras una transformación de la respuesta):

$$Y(\mathbf{X}) = m(\mathbf{X}) + \varepsilon \quad (7.1)$$

donde $m(\mathbf{x}) = E(Y|\mathbf{X} = \mathbf{x})$ es la media condicional (función de regresión o tendencia) y ε un error aleatorio de media cero y varianza σ^2 , independiente de $\mathbf{X}$.

Además, salvo que se indique lo contrario, se asume que las n observaciones de la muestra son **independientes entre sí** (y en muchos desarrollos teóricos, también idénticamente distribuidas, supuesto

habitualmente conocido como i.i.d.). Cuando esto no se cumple —por ejemplo con **series temporales** (observaciones consecutivas en el tiempo, ver **tidyverts/fable** en la Sección 5), datos **agrupados o longitudinales** (varias medidas repetidas sobre un mismo individuo) o datos **espaciales** (observaciones cercanas geográficamente más parecidas entre sí)— la dependencia entre observaciones debe tenerse en cuenta explícitamente, tanto al ajustar el modelo como al evaluarlo (por ejemplo, particionar entrenamiento/test al azar dejaría de ser adecuado; ver Sección 7.1.3.3). El tratamiento de datos dependientes requiere métodos específicos que quedan fuera del alcance de esta introducción.

7.1.2.2 Métodos (de aprendizaje supervisado) y paquetes de R

Hay una gran cantidad de métodos de aprendizaje supervisado implementados en centenares de paquetes de R (ver CRAN Task View: Machine Learning). Algunos de los principales:

Métodos (principalmente) de clasificación:

- Análisis discriminante, regresión logística, multinomial...: **stats**, **MASS**.
- k -vecinos más próximos (KNN): **class**, **kkn**, **caret**.
- Árboles de decisión, *bagging*, bosques aleatorios, *boosting*: **rpart**, **party**, **C50**, **randomForest**, **adabag**, **xgboost**.
- Máquinas de soporte vectorial: **kernlab**, **e1071**.

Métodos (principalmente) de regresión:

- Modelos lineales (**lm()**), regularizados (*ridge*, LASSO: **glmnet**) y lineales generalizados (**glm()**).
- Modelos paramétricos no lineales (**nls()**).
- Regresión local y suavizado: **kkn**, **loess()**, **np**.
- Modelos aditivos generalizados: **mgcv**.
- Redes neuronales: **nnet**, **neuralnet**.

Como todos estos paquetes emplean opciones y convenciones distintas, se han desarrollado paquetes que ofrecen interfaces unificadas, entre ellos **caret**, **mlr3** y **tidymodels** (este último juega, para el modelado predictivo, un papel similar al que **tidyverts/fable** de Hyndman juega para las series temporales; ver Capítulo 5). También existen paquetes con entornos gráficos que evitan el uso de código, como **rattle**.

7.1.3 Construcción y evaluación de los modelos

En la inferencia estadística clásica se emplea toda la información disponible para ajustar un modelo y, asumiendo que es correcto, se usan resultados teóricos para evaluar su precisión (por ejemplo, el coeficiente de determinación ajustado en regresión lineal múltiple). En estadística computacional, en cambio, es habitual evaluar la precisión mediante técnicas de remuestreo, como la validación cruzada, el *jackknife* o el *bootstrap*, entrenando el modelo con los datos disponibles y estimando el error de predicción de forma empírica.

Muchos métodos de AE son muy flexibles y pueden llegar a sobreajustarse a los datos de entrenamiento. Para evitarlo hay que controlar el proceso de aprendizaje mediante **hiperparámetros** (*tuning parameters*), que imponen restricciones al modelo. Una elección inadecuada puede dar lugar a sobreajuste (*overfitting*) o infraajuste (*underfitting*); además, una mayor complejidad suele reducir la interpretabilidad, por lo que el objetivo es lograr buenas predicciones con el modelo más simple posible.

7.1.3.1 Equilibrio entre sesgo y varianza: infraajuste y sobreajuste

Queremos aprender más allá de los datos de entrenamiento (hacer inferencia sobre nuevas observaciones). En AE hay que tener especial cuidado con el sobreajuste: el modelo se ajusta demasiado bien a los datos de entrenamiento pero falla con datos nunca vistos.

Como ejemplo ilustrativo empleamos regresión polinómica, considerando el grado del polinomio como hiperparámetro que determina la complejidad del modelo. Simulamos una muestra y ajustamos modelos con distinta complejidad:

```
# Simulación datos
n <- 30
x <- seq(0, 1, length = n)
mu <- 2 + 4*(5*x - 1)*(4*x - 2)*(x - 0.8)^2 # grado 4
sd <- 0.5
set.seed(1)
y <- mu + rnorm(n, 0, sd)
plot(x, y)
lines(x, mu, lwd = 2)
# Ajuste de los modelos
fit1 <- lm(y ~ x)
lines(x, fitted(fit1))
fit2 <- lm(y ~ poly(x, 4))
lines(x, fitted(fit2), lty = 2)
fit3 <- lm(y ~ poly(x, 20))
lines(x, fitted(fit3), lty = 3)
legend("topright", lty = c(1, 1, 2, 3), lwd = c(2, 1, 1, 1),
      legend = c("Verdadero", "Ajuste con grado 1",
                  "Ajuste con grado 4", "Ajuste con grado 20"))
```

Al aumentar la complejidad se consigue un mejor ajuste a los datos de entrenamiento (Figura 7.2), a costa de un incremento de la variabilidad de las predicciones, lo que puede empeorar el comportamiento del modelo con datos distintos de los observados. Si calculamos medidas de bondad de ajuste (MSE , R^2) se obtienen mejores resultados al aumentar la complejidad (Tabla 7.1):

Tabla 7.1: Medidas de bondad de ajuste de los modelos polinómicos (muestra de entrenamiento).

| | MSE | R^2 | R^2_{adj} |
|------|-------|-------|-------------|
| fit1 | 1.22 | 0.20 | 0.17 |
| fit2 | 0.19 | 0.87 | 0.85 |
| fit3 | 0.07 | 0.95 | 0.84 |

Pero si generamos nuevas respuestas del mismo proceso (predecimos), la precisión del modelo más complejo empeora considerablemente (Figura 7.3):

```
y.new <- mu + rnorm(n, 0, sd)
plot(x, y)
points(x, y.new, pch = 2)
lines(x, mu, lwd = 2)
lines(x, fitted(fit1))
lines(x, fitted(fit2), lty = 2)
lines(x, fitted(fit3), lty = 3)
leyenda <- c("Verdadero", "Muestra", "Ajuste con grado 1",
             "Ajuste con grado 4", "Ajuste con grado 20", "Nuevas observaciones")
```

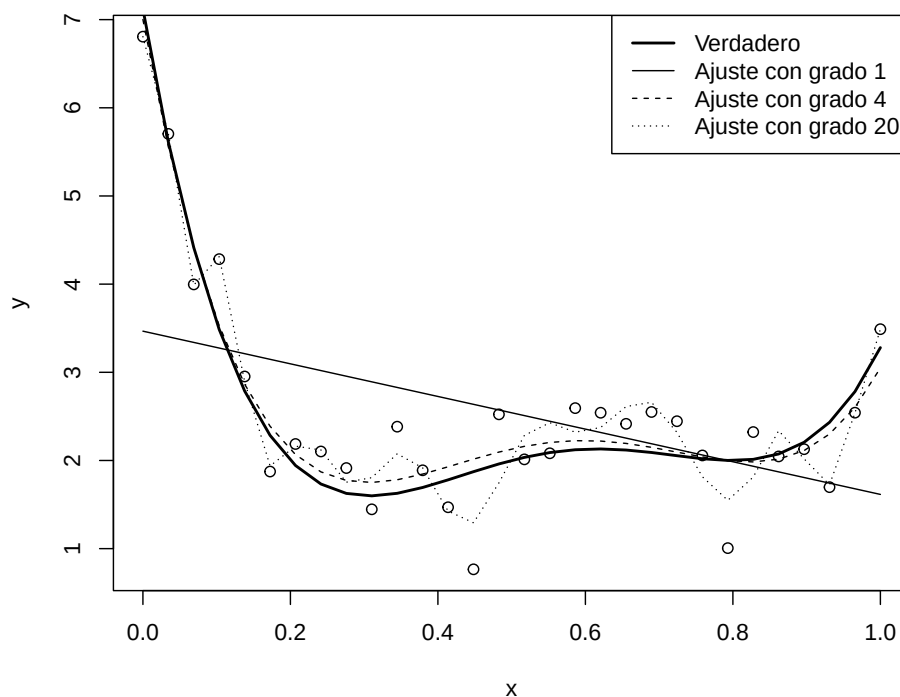


Figura 7.2: Muestra (simulada) y ajustes polinómicos con distinta complejidad.

```
legend("topright", legend = leyenda, lty = c(1, NA, 1, 2, 3, NA),
      lwd = c(2, NA, 1, 1, 1, NA), pch = c(NA, 1, NA, NA, NA, 2))
```

```
MSEP <- sapply(list(fit1 = fit1, fit2 = fit2, fit3 = fit3),
               function(x) mean((y.new - fitted(x))^2))
MSEP
```

```
##      fit1      fit2      fit3
## 1.4983208 0.1711238 0.2621064
```

Repitiendo la simulación 100 veces variando el grado del polinomio de 1 a 20 y evaluando la precisión tanto en la muestra de ajuste como en un nuevo conjunto de datos, obtenemos la Figura 7.4: los errores de entrenamiento disminuyen al aumentar la complejidad, pero los errores de predicción en nuevas observaciones inicialmente disminuyen, alcanzan un mínimo (línea vertical discontinua, modelo de grado 4) y después aumentan. A la izquierda de esa línea habría infraajuste (mayor sesgo, menor varianza) y a la derecha, sobreajuste (menor sesgo, mayor varianza).

```
nsim <- 100
set.seed(1)
grado.max <- 20
grados <- seq_len(grado.max)
# Simulación, ajustes y errores cuadráticos
mse <- mse.new <- matrix(nrow = grado.max, ncol = nsim)
for(i in seq_len(nsim)) {
  y <- mu + rnorm(n, 0, sd)
  y.new <- mu + rnorm(n, 0, sd)
```

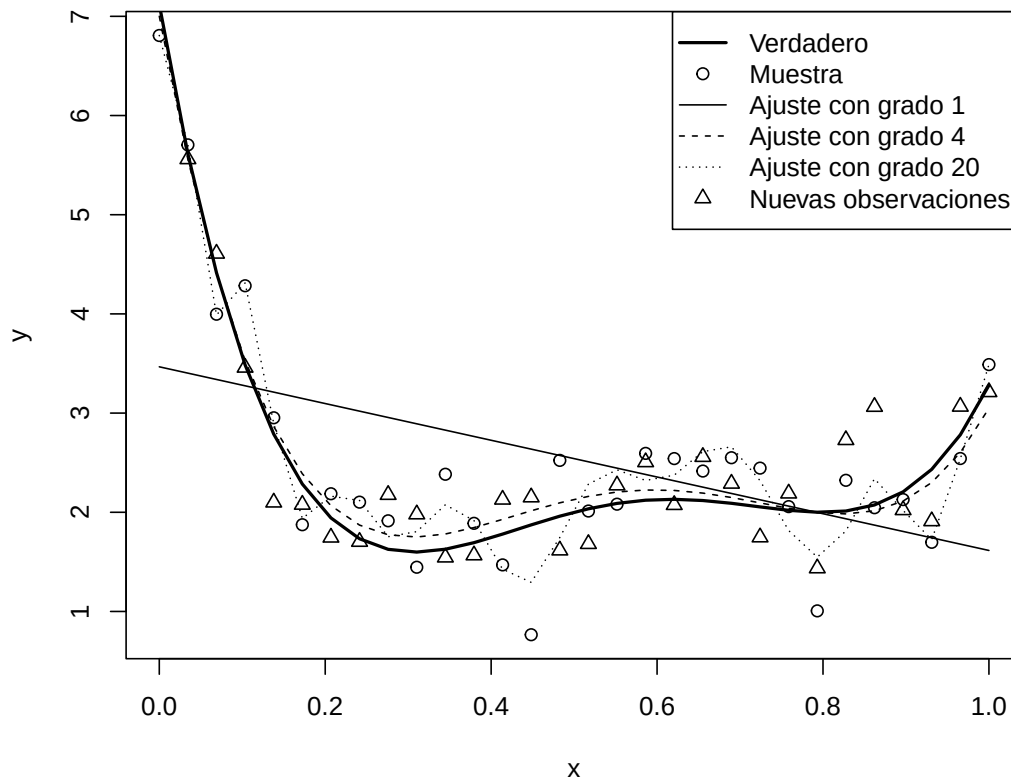


Figura 7.3: Muestra con ajustes polinómicos con distinta complejidad y nuevas observaciones.

```
for (grado in grados) {
  fit <- lm(y ~ poly(x, grado))
  mse[grado, i] <- mean(residuals(fit)^2)
  mse.new[grado, i] <- mean((y.new - fitted(fit))^2)
}
}
# Representación errores simulaciones
matplot(grados, mse, type = "l", col = "lightgray", lty = 1, ylim = c(0, 2),
  xlab = "Grado del polinomio (complejidad)", ylab = "Error cuadrático medio")
matlines(grados, mse.new, type = "l", lty = 2, col = "lightgray")
# Errores globales
precision <- rowMeans(mse)
precision.new <- rowMeans(mse.new)
lines(grados, precision, lwd = 2)
lines(grados, precision.new, lty = 2, lwd = 2)
abline(h = sd^2, lty = 3); abline(v = 4, lty = 3)
legend("topright", legend = c("Muestras", "Nuevas observaciones"), lty = c(1, 2))
```

En general, al aumentar la complejidad disminuye el sesgo y aumenta la varianza (y viceversa): es el compromiso entre sesgo y varianza (*bias-variance tradeoff*, Figura 7.5). La recomendación es seleccionar los hiperparámetros tratando de equilibrar ambos.

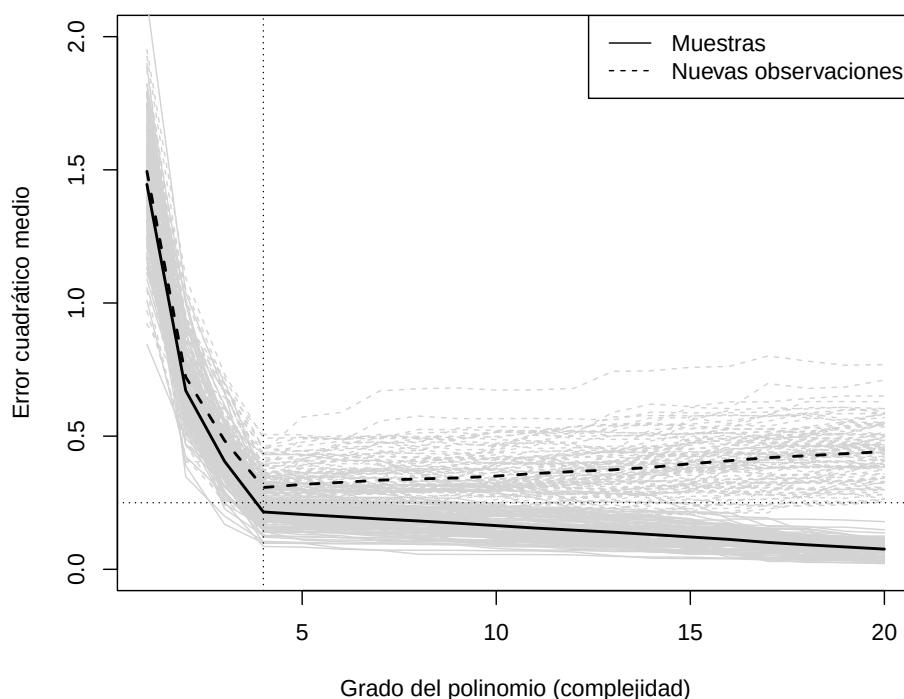


Figura 7.4: Precisiones (errores cuadráticos medios) de ajustes polinómicos en las muestras empleadas en el ajuste y en nuevas observaciones.

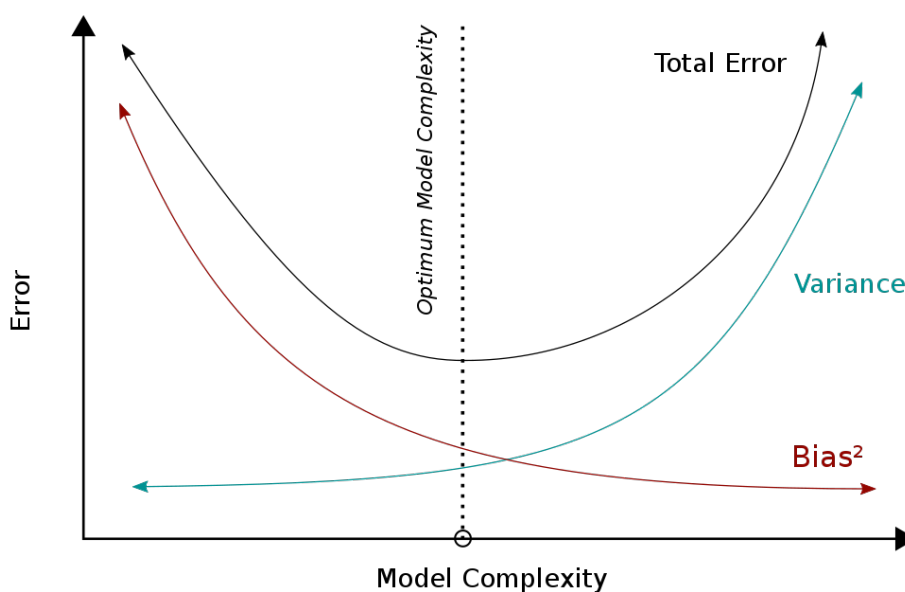


Figura 7.5: Equilibrio entre sesgo y varianza (Fuente: Wikimedia Commons).

7.1.3.2 Datos de entrenamiento y datos de test

Si el número de observaciones no es muy grande, se puede entrenar el modelo con todos los datos y emplear técnicas de remuestreo para evaluar la precisión (validación cruzada o *bootstrap*). Si el número

de observaciones es grande, se suele particionar la base de datos en 2 (o 3) conjuntos disjuntos: **entrenamiento** (para construir los modelos) y **test** (para evaluar el rendimiento; los errores en esta muestra aproximan lo que ocurriría con nuevas observaciones). Típicamente se selecciona al azar un 80% para entrenamiento y un 20% para test.

Como ejemplo consideraremos el conjunto de datos **Boston** del paquete **MASS**, con la valoración de viviendas (**medv**) y el porcentaje de población con “menor estatus” (**lstat**) en los suburbios de Boston:

```
data(Boston, package = "MASS")
set.seed(2)
nobs <- nrow(Boston)
itrain <- sample(nobs, round(0.8 * nobs))
train <- Boston[itrain, ]
test <- Boston[-itrain, ]
```

Análisis descriptivo básico de la muestra de entrenamiento:

```
str(train)
```

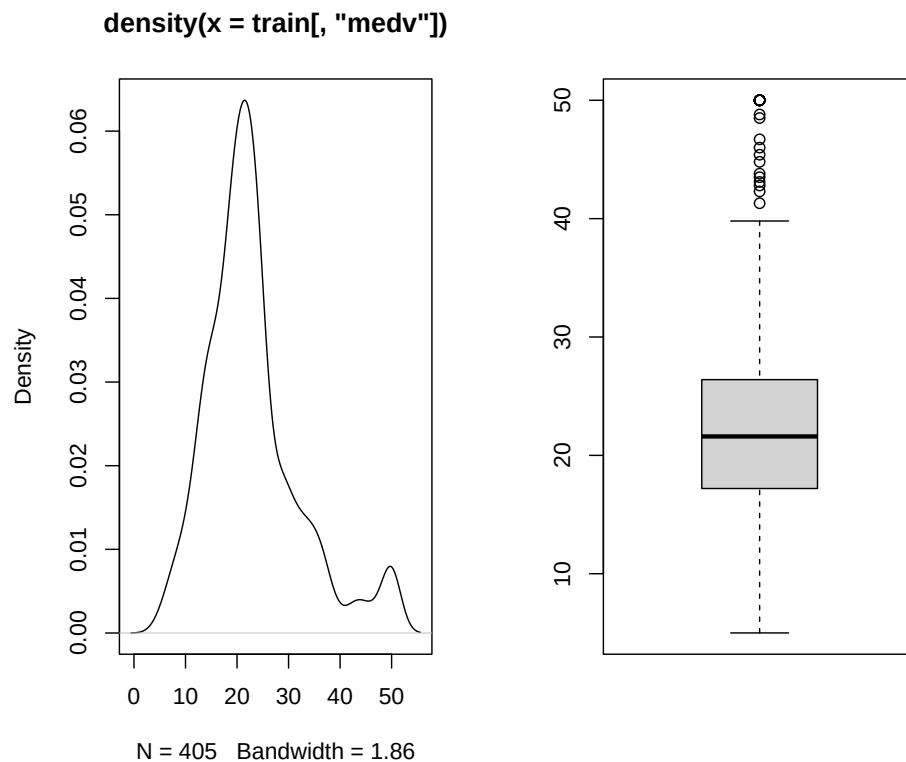
```
## 'data.frame': 405 obs. of 14 variables:
## $ crim : num 0.0615 6.6549 0.0467 0.5341 18.0846 ...
## $ zn : num 0 0 80 20 0 0 20 80 0 95 ...
## $ indus : num 5.19 18.1 1.52 3.97 18.1 18.1 6.96 2.01 18.1 2.68 ...
## $ chas : int 0 0 0 0 0 0 0 0 0 0 ...
## $ nox : num 0.515 0.713 0.404 0.647 0.679 ...
## $ rm : num 5.97 6.32 7.11 7.52 6.43 ...
## $ age : num 58.5 83 36.6 89.4 100 82.5 58.7 29.7 65.4 33.2 ...
## $ dis : num 4.81 2.73 7.31 2.14 1.83 ...
## $ rad : int 5 24 2 5 24 24 3 4 24 4 ...
## $ tax : num 224 666 329 264 666 666 223 280 666 224 ...
## $ ptratio: num 20.2 20.2 12.6 13 20.2 20.2 18.6 17 20.2 14.7 ...
## $ black : num 396.9 396.9 354.3 388.4 27.2 ...
## $ lstat : num 9.29 13.99 8.61 7.26 29.05 ...
## $ medv : num 18.7 19.5 30.3 43.1 7.2 23.2 24.4 24.5 21.4 48.5 ...
```

```
summary(train)
```

```
##      crim      zn      indus      chas
## Min.   : 0.00632   Min.   : 0.00   Min.   : 0.46   Min.   :0.00000
## 1st Qu.: 0.07151   1st Qu.: 0.00   1st Qu.: 4.95   1st Qu.:0.00000
## Median : 0.21977   Median : 0.00   Median : 8.56   Median :0.00000
## Mean   : 3.36920   Mean   : 11.69   Mean   :10.91   Mean   :0.06914
## 3rd Qu.: 3.47428   3rd Qu.: 18.00   3rd Qu.:18.10   3rd Qu.:0.00000
## Max.   :88.97620   Max.   :100.00   Max.   :27.74   Max.   :1.00000
##      nox      rm      age      dis
## Min.   :0.3890   Min.   :3.561   Min.   : 2.90   Min.   : 1.137
## 1st Qu.:0.4480   1st Qu.:5.895   1st Qu.: 42.30   1st Qu.: 2.122
## Median :0.5240   Median :6.223   Median : 74.30   Median : 3.317
## Mean   :0.5517   Mean   :6.309   Mean   : 67.25   Mean   : 3.835
## 3rd Qu.:0.6240   3rd Qu.:6.635   3rd Qu.: 93.90   3rd Qu.: 5.245
## Max.   :0.8710   Max.   :8.780   Max.   :100.00   Max.   :12.127
##      rad      tax      ptratio      black
## Min.   : 1.000   Min.   :187.0   Min.   :12.60   Min.   : 0.32
## 1st Qu.: 4.000   1st Qu.:276.0   1st Qu.:17.00   1st Qu.:377.07
## Median : 5.000   Median :313.0   Median :18.70   Median :392.23
## Mean   : 9.306   Mean   :400.9   Mean   :18.37   Mean   :358.58
## 3rd Qu.:24.000   3rd Qu.:666.0   3rd Qu.:20.20   3rd Qu.:396.90
```

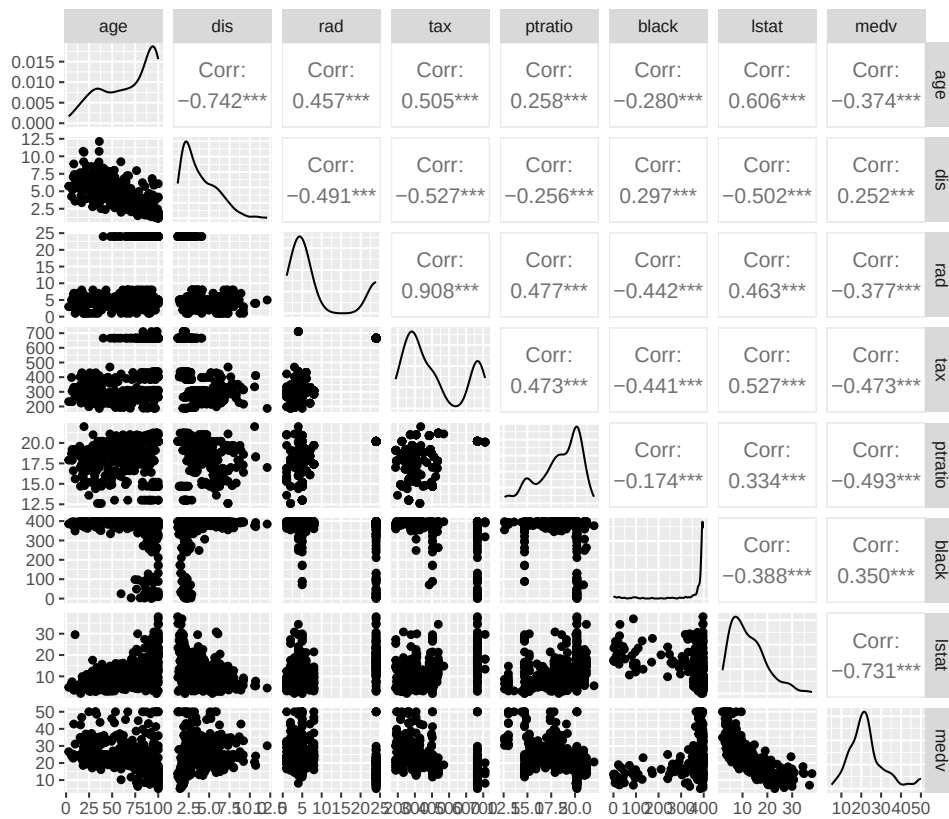
```
## Max. :24.000 Max. :711.0 Max. :22.00 Max. :396.90
## lstat medv
## Min. : 1.92 Min. : 5.00
## 1st Qu.: 6.78 1st Qu.:17.20
## Median :10.74 Median :21.60
## Mean :12.39 Mean :22.92
## 3rd Qu.:16.44 3rd Qu.:26.40
## Max. :37.97 Max. :50.00
```

```
par(mfrow = c(1, 2))
plot(density(train[, "medv"]))
boxplot(train$medv)
```



Para explorar las relaciones entre variables numéricas podemos representar la matriz de correlaciones con `GGally::ggpairs()`:

```
library(GGally)
# solo se visualizan las variables de la posición (columna) 7 a 14
ggpairs(train, columns = 7:14)
```



```
# Alternativas clásicas:
pairs(train[, 7:14])
mcor <- cor(train)
round(mcor, 2)
library(corrplot)
corrplot(mcor)
```

Los datos de *test* deben utilizarse **exclusivamente** para evaluar el rendimiento final de los modelos, nunca para seleccionar hiperparámetros. Para esto último hay varias estrategias habituales:

- **Partición entrenamiento-validación-test:** la muestra se divide en tres subconjuntos (por ejemplo 70%-15%-15%). Los hiperparámetros se seleccionan con la muestra de validación y el modelo final se evalúa una única vez sobre el test.
- **Validación cruzada (*cross-validation*):** los datos de entrenamiento se reutilizan mediante remuestreo. En la validación cruzada *k-fold*, los datos se dividen en *k* bloques disjuntos; en cada iteración uno actúa como validación y los *k-1* restantes como entrenamiento, repitiendo el proceso *k* veces.
- **Validación cruzada dejando una observación fuera (*leave-one-out*, LOOCV):** caso particular de *k-fold* con tantos bloques como observaciones. Aprovecha al máximo los datos, pero puede ser costosa computacionalmente y dar estimaciones con elevada varianza.
- **Bootstrap:** se generan remuestras con reemplazamiento; el modelo se ajusta en cada una y se evalúa sobre las observaciones no seleccionadas (*out-of-bag*).

En el caso de series temporales no es adecuado emplear particiones aleatorias, ya que se rompería la dependencia temporal: se emplean esquemas que respetan el orden cronológico (bloques temporales, o *rolling forecasting/time series cross-validation*, reentrenando sucesivamente con observaciones pasadas

para predecir valores futuros; ver Capítulo 5 y, para más detalle, la Sección 5.10 de *Forecasting: Principles and Practice* de Hyndman y Athanasopoulos).

7.1.3.3 Selección de hiperparámetros mediante validación cruzada

Continuando con el ejemplo anterior, empleemos regresión polinómica para explicar la valoración de las viviendas a partir del estatus de los residentes (Figura 7.6), considerando de nuevo el grado del polinomio como hiperparámetro:

```
formula <- medv ~ lstat
plot(formula, data = train)
```

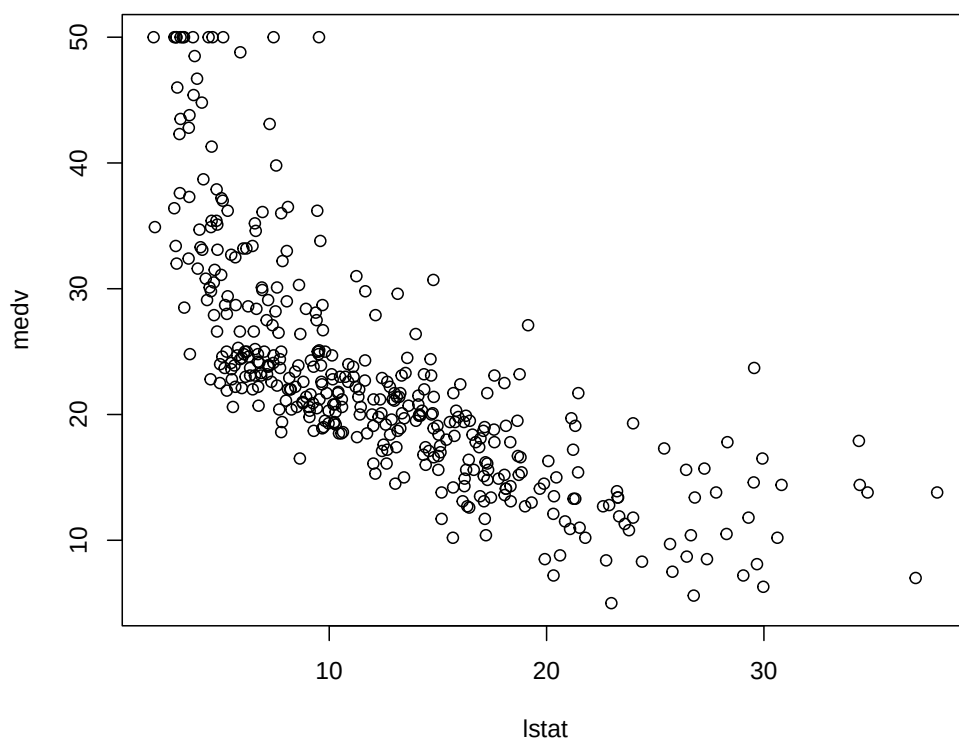


Figura 7.6: Valoración de las viviendas (medv) frente al porcentaje de población con menor estatus (lstat).

```
modelo0 <- lm(formula, train)
modelo0
```

```
##
## Call:
## lm(formula = formula, data = train)
##
## Coefficients:
## (Intercept)      lstat
##    34.7024    -0.9512
```

```
summary(modelo0)
```

```
##
## Call:
## lm(formula = formula, data = train)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -9.974 -4.133 -1.353  2.081 24.363
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 34.70241    0.62997   55.09  <2e-16 ***
## lstat       -0.95125    0.04419  -21.53  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.271 on 403 degrees of freedom
## Multiple R-squared:  0.5349, Adjusted R-squared:  0.5337
## F-statistic: 463.5 on 1 and 403 DF,  p-value: < 2.2e-16
```

El ajuste lineal simple no es del todo adecuado (Figura 7.7), lo que motiva probar con polinomios de mayor grado:

```
par(mfrow = c(2, 2))
plot(modelo0)
```

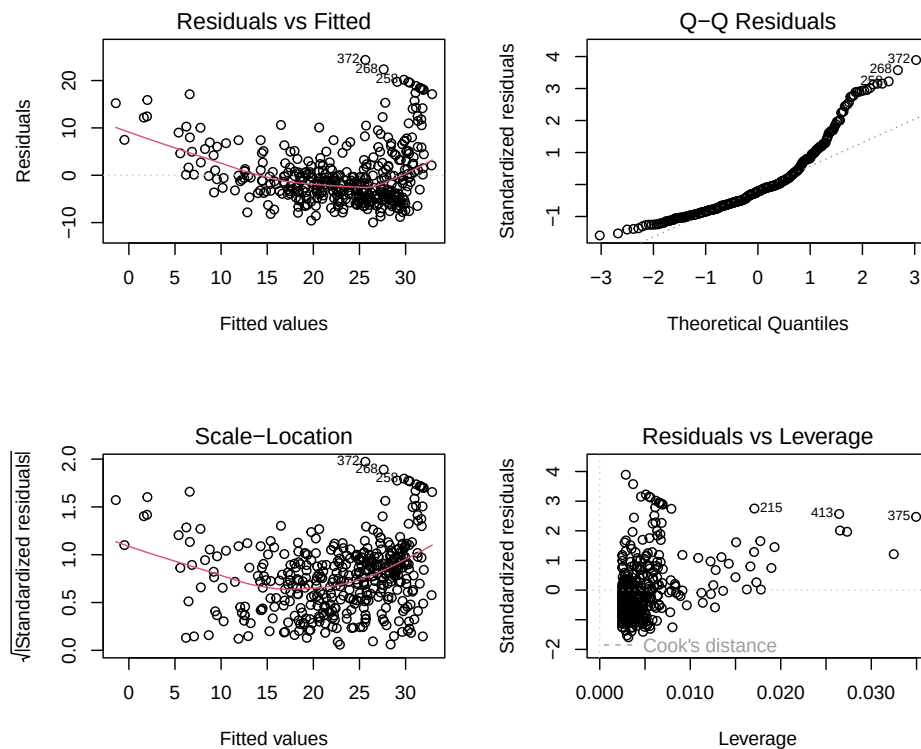


Figura 7.7: Gráficos de diagnóstico del modelo lineal simple.

```
par(mfrow = c(1, 1))
```

En el caso de regresión lineal múltiple (y otros predictores lineales) se pueden obtener fácilmente las predicciones eliminando una observación a partir del ajuste con todos los datos (ver `?rstandard`), evitando así ajustar el modelo tantas veces como observaciones:

```
cv.lm <- function(formula, datos) {
  modelo <- lm(formula, datos)
  return(rstandard(modelo, type = "predictive"))
}
res.cv2 <- cv.lm(medv ~ poly(lstat, 2), train)
res.cv4 <- cv.lm(medv ~ poly(lstat, 4), train)
res.cv5 <- cv.lm(medv ~ poly(lstat, 5), train) # Óptimo con 5 grados
res.cv6 <- cv.lm(medv ~ poly(lstat, 6), train)
c(mean(res.cv2^2), mean(res.cv4^2), mean(res.cv5^2), mean(res.cv6^2))
```

```
## [1] 30.88877 29.06375 28.20607 28.38612
```

Calculamos el error cuadrático medio de validación cruzada para cada grado del polinomio y seleccionamos el que lo minimiza (Figura 7.8):

```
grado.max <- 10
grados <- seq_len(grado.max)
cv.mse <- cv.mse.sd <- numeric(grado.max)
for (grado in grados) {
  cv.res <- cv.lm(medv ~ poly(lstat, grado), train)
  se <- cv.res^2
  cv.mse[grado] <- mean(se)
  cv.mse.sd[grado] <- sd(se) / sqrt(length(se))
}
plot(grados, cv.mse, ylim = c(25, 45), xlab = "Grado del polinomio")
imin.mse <- which.min(cv.mse)
grado.min <- grados[imin.mse]
points(grado.min, cv.mse[imin.mse], pch = 16)
```

```
grado.min
```

```
## [1] 5
```

En lugar de emplear el valor óptimo del hiperparámetro, Breiman et al. (1984) propusieron la regla de “un error estándar” (*one-standard-error rule*): dado que las estimaciones de precisión presentan variabilidad, se selecciona el modelo más simple cuya precisión esté dentro de un error estándar de la del modelo óptimo (Figura 7.9):

```
plot(grados, cv.mse, ylim = c(25, 45), xlab = "Grado del polinomio")
segments(grados, cv.mse - cv.mse.sd, grados, cv.mse + cv.mse.sd)
upper.cv.mse <- cv.mse[imin.mse] + cv.mse.sd[imin.mse]
abline(h = upper.cv.mse, lty = 2)
imin.1se <- min(which(cv.mse <= upper.cv.mse))
grado.1se <- grados[imin.1se]
points(grado.1se, cv.mse[imin.1se], pch = 16)
```

```
grado.1se
```

```
## [1] 2
```

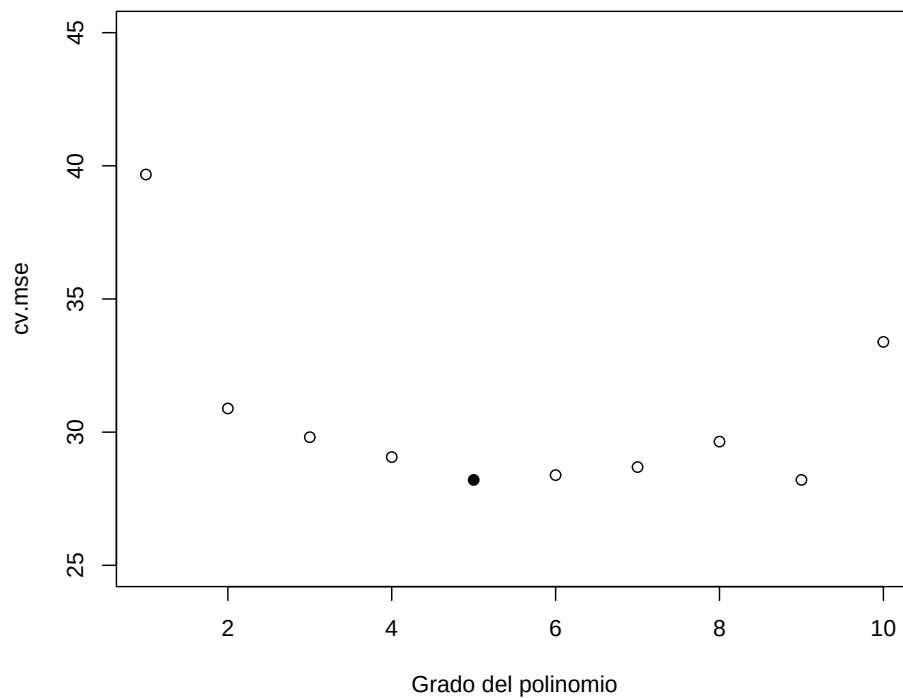


Figura 7.8: Error cuadrático medio de validación cruzada según el grado del polinomio (complejidad).

```
plot(medv ~ lstat, data = train)
fit.min <- lm(medv ~ poly(lstat, grado.min), train)
fit.1se <- lm(medv ~ poly(lstat, grado.1se), train)
newdata <- data.frame(lstat = seq(0, 40, len = 100))
lines(newdata$lstat, predict(fit.min, newdata = newdata), lwd = 3)
lines(newdata$lstat, predict(fit.1se, newdata = newdata), lty = 2, col = 2, lwd = 3)
legend("topright", legend = c(paste("Grado óptimo:", grado.min),
                               paste("oneSE rule:", grado.1se)), lty = c(1, 2))
```

Ejercicio 7.1

Particiona la muestra **Boston** en datos de entrenamiento (70%), validación (15%) y test (15%), para entrenar los modelos polinómicos, seleccionar el grado óptimo (el hiperparámetro) y evaluar las predicciones del modelo final. Puede ser de utilidad:

```
df <- Boston
set.seed(1)
nobs <- nrow(df)
itrain <- sample(nobs, 0.7 * nobs)
inotrain <- setdiff(seq_len(nobs), itrain)
ivalidate <- sample(inotrain, 0.15 * nobs)
itest <- setdiff(inotrain, ivalidate)
train <- df[itrain, ]
validate <- df[ivalidate, ]
test <- df[itest, ]
```

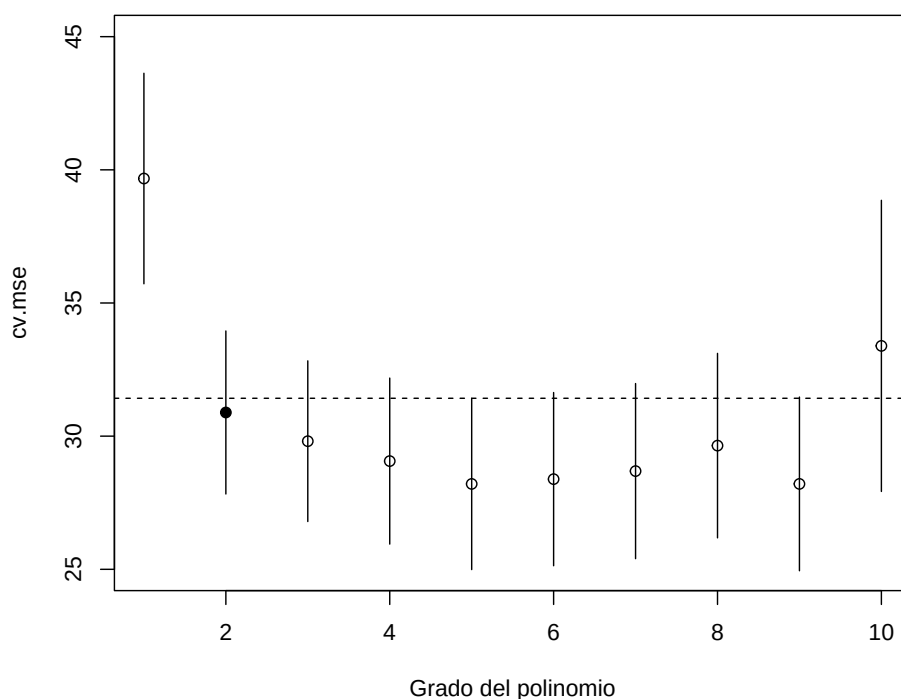


Figura 7.9: Selección del grado del polinomio mediante validación cruzada: valor óptimo y criterio de un error estándar (punto sólido).

Ejercicio 7.2

Una alternativa a la partición clásica en entrenamiento y validación es emplear **bootstrap** para estimar el error de predicción: se genera una remuestra bootstrap a partir del entrenamiento, se ajusta el modelo sobre ella y se evalúa el error en las observaciones no seleccionadas (*out-of-bag*, OOB). Trabajando de nuevo con **Boston** (*medv* frente a *lstat*), realiza el ajuste empleando una única muestra bootstrap y evalúa las predicciones en la muestra de test. Puede ser de utilidad:

```
data(Boston, package = "MASS")
set.seed(1)
nobs <- nrow(Boston)
itrain <- sample(nobs, size = floor(0.8 * nobs))
train <- Boston[itrain, ]
test <- Boston[-itrain, ]
# Remuestra bootstrap (con reemplazamiento) de los índices de entrenamiento
set.seed(1)
ntrain <- nrow(train)
itrain_boot <- sample(seq_len(ntrain), replace = TRUE)
train_boot <- train[itrain_boot, ]
# Observaciones "out of bag" (no seleccionadas en la remuestra)
oob <- train[-itrain_boot, ]
```

7.1.3.4 Evaluación de un método de regresión

Para estudiar la precisión de las predicciones se evalúa el modelo en el conjunto de test y se comparan las predicciones frente a los valores reales. En un gráfico de dispersión de observaciones frente a predicciones

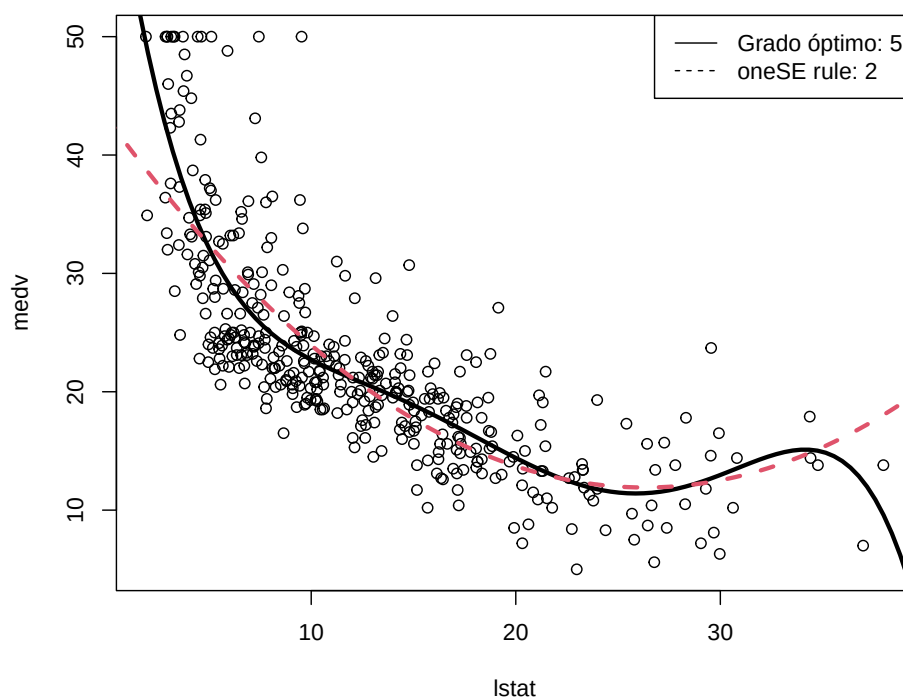


Figura 7.10: Ajuste de los modelos finales: valor óptimo (línea continua) y criterio de un error estándar (línea discontinua).

(Figura 7.11), los puntos deberían estar en torno a la recta $y = x$:

```
obs <- test$medv
pred <- predict(fit.min, newdata = test)
plot(pred, obs, xlab = "Predicción", ylab = "Observado")
abline(a = 0, b = 1)
res <- lm(obs ~ pred)
abline(res, lty = 2)
```

También es habitual calcular medidas de error, por ejemplo con `caret::postResample()`:

```
caret::postResample(pred, obs)
```

```
##      RMSE Rsquared      MAE
## 4.8529138 0.6234068 3.6652591
```

```
sqrt(mean((obs - pred)^2))
```

```
## [1] 4.852914
```

```
mean(abs(obs - pred))
```

```
## [1] 3.665259
```

Además de las medidas de error habituales, `postResample()` calcula un *pseudo R-cuadrado* basado en el cuadrado de la correlación entre predicciones y observaciones, que no tiene en cuenta el sesgo (obtendríamos el mismo valor si desplazamos las predicciones sumando una constante). Una alternativa preferible

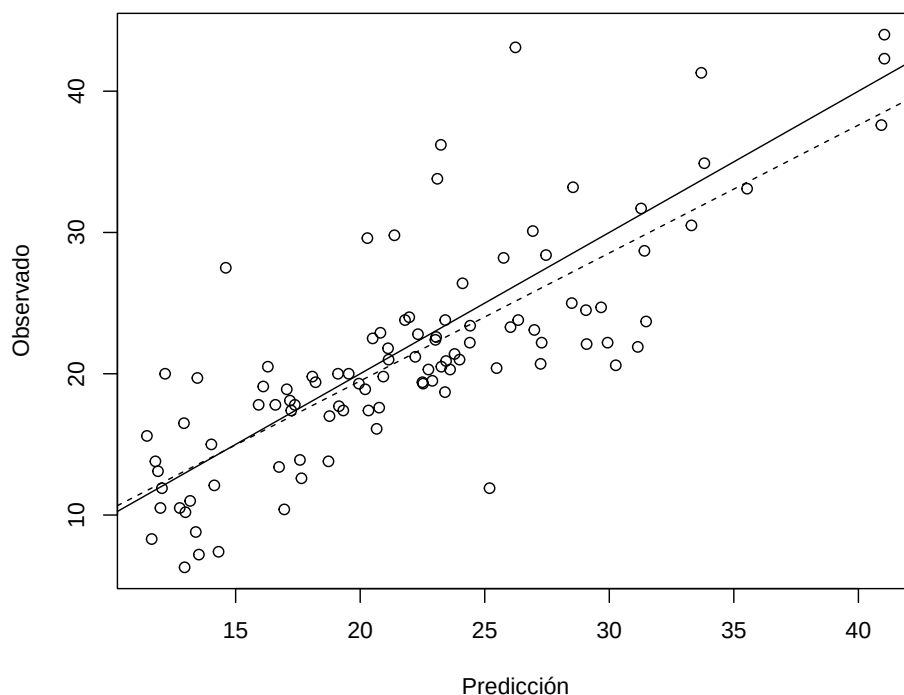


Figura 7.11: Observaciones frente a predicciones (identidad, línea continua, y ajuste lineal, línea discontinua).

sería:

$$\tilde{R}^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

(equivalente al coeficiente de determinación ajustado, pero sin depender de las hipótesis estructurales del modelo), implementada junto con otras medidas de error en la función `accuracy()` del paquete `mpae` (el mismo paquete de los autores del libro de referencia de esta sección):

```
library(mpae)
accuracy <- function(pred, obs, na.rm = FALSE,
                      tol = sqrt(.Machine$double.eps)) {
  err <- obs - pred      # Errores
  if (na.rm) {
    is.a <- !is.na(err)
    err <- err[is.a]
    obs <- obs[is.a]
  }
  perr <- 100 * err / pmax(obs, tol) # Errores porcentuales
  return(c(
    me = mean(err),           # Error medio
    rmse = sqrt(mean(err^2)), # Raíz del error cuadrático medio
    mae = mean(abs(err)),     # Error absoluto medio
    mpe = mean(perr),        # Error porcentual medio
    mape = mean(abs(perr)),   # Error porcentual absoluto medio
    r.squared = 1 - sum(err^2) / sum((obs - mean(obs))^2) # Pseudo R-cuadrado
  ))
}
```

```

}
accu.min <- accuracy(pred, obs)
accu.min

##          me          rmse          mae          mpe          mape  r.squared
## -0.6932010  4.8529138  3.6652591 -8.7126633 19.9985572  0.6086314

accu.1se <- accuracy(predict(fit.1se, newdata = test), obs)
accu.1se

##          me          rmse          mae          mpe          mape  r.squared
## -0.863974  5.216321  4.077066 -9.109356 21.570488  0.547822

```

En este caso, el ajuste polinómico con el grado óptimo explicaría un 60.9 % de la variabilidad de la respuesta en nuevas observaciones (un 6.1 % más que el modelo seleccionado con el criterio de un error estándar).

7.1.4 Clasificación

Todo lo anterior se ilustró con un problema de regresión (*medv* es numérica). Cuando la respuesta es categórica hablamos de **clasificación**; veamos un ejemplo breve, tanto con dos clases (clasificación *binaria*) como con más de dos (*multiclase*), empleando el conjunto de datos *iris* (tres especies de lirio a partir de las medidas de sépalos y pétalos).

7.1.4.1 Ejemplo: clasificación binaria y multiclase con KNN

El método de los **k-vecinos más próximos** (*k-nearest neighbors*, KNN) es uno de los más simples e intuitivos: para clasificar una nueva observación, busca las *k* observaciones de entrenamiento más cercanas (según alguna distancia, típicamente la euclídea) y le asigna la clase mayoritaria entre ellas.

Para el caso binario nos quedamos únicamente con dos de las tres especies:

```

data(iris)
iris2 <- droplevels(subset(iris, Species != "setosa"))
set.seed(1)
nobs <- nrow(iris2)
itrain <- sample(nobs, round(0.8 * nobs))
train2 <- iris2[itrain, ]
test2 <- iris2[-itrain, ]

library(class)
pred2 <- knn(train = train2[, c("Petal.Length", "Petal.Width")],
             test = test2[, c("Petal.Length", "Petal.Width")],
             cl = train2$Species, k = 5)
table(Predicho = pred2, Real = test2$Species)

##          Real
## Predicho  versicolor virginica
## versicolor      11          0
## virginica       0           9

mean(pred2 == test2$Species) # precisión (proporción de aciertos)

## [1] 1

```

Lo interesante de KNN es que se extiende de forma **directa** al caso multiclase: la votación por mayoría entre los *k* vecinos funciona igual sea cual sea el número de clases, sin necesidad de ningún truco adicional.

Repitamos el ejemplo con las tres especies:

```
set.seed(1)
nobs <- nrow(iris)
itrain <- sample(nobs, round(0.8 * nobs))
train3 <- iris[itrain, ]
test3 <- iris[-itrain, ]

pred3 <- knn(train = train3[, c("Petal.Length", "Petal.Width")],
             test = test3[, c("Petal.Length", "Petal.Width")],
             cl = train3$Species, k = 5)
table(Predicho = pred3, Real = test3$Species)
```

```
##           Real
## Predicho  setosa versicolor virginica
## setosa      11         0         0
## versicolor  0         12         1
## virginica   0         0         6
```

```
mean(pred3 == test3$Species)
```

```
## [1] 0.9666667
```

No todos los métodos son tan flexibles: muchos clasificadores habituales (regresión logística, máquinas de soporte vectorial...) son *binarios por construcción* y necesitan una estrategia adicional para abordar más de dos clases. Las dos más empleadas son:

- **Uno contra uno** (*one-vs-one*, OVO): se entrena un clasificador binario para cada posible par de clases ($\binom{K}{2}$ modelos con K clases) y se decide por votación mayoritaria entre todos ellos.
- **Uno contra el resto** (*one-vs-all/one-vs-rest*, OVA/OVR): se entrena un clasificador binario por cada clase frente a todas las demás juntas (K modelos), y se asigna la clase cuyo modelo dé mayor confianza.

Por ejemplo, `e1071` resuelve el caso multiclase de las máquinas de soporte vectorial mediante la estrategia OVO por defecto:

```
library(e1071)
modelo_ovo <- svm(Species ~ Petal.Length + Petal.Width, data = train3, kernel = "linear")
pred_ovo <- predict(modelo_ovo, newdata = test3)
table(Predicho = pred_ovo, Real = test3$Species)
```

```
##           Real
## Predicho  setosa versicolor virginica
## setosa      11         0         0
## versicolor  0         12         1
## virginica   0         0         6
```

```
mean(pred_ovo == test3$Species)
```

```
## [1] 0.9666667
```

Como con la regresión, la evaluación (matriz de confusión, precisión, u otras medidas como sensibilidad/especificidad) debe hacerse siempre sobre datos de test no utilizados para entrenar, y los hiperparámetros (aquí, k o el tipo de *kernel*) se seleccionarían por validación cruzada (Sección 7.1.3.3) igual que en regresión.

7.1.4.2 La maldición de la dimensionalidad

Los métodos basados en distancias o vecindarios locales, como KNN, se ven especialmente afectados por la llamada **maldición de la dimensionalidad** (*curse of dimensionality*): a medida que aumenta el número de predictores p , los datos se vuelven cada vez más dispersos en ese espacio de mayor dimensión, y el concepto de “vecino cercano” deja de ser realmente local.

Una forma sencilla de verlo: para capturar una fracción fija r del volumen de datos mediante un hipercubo centrado en el punto de interés, el lado necesario de ese hipercubo es $r^{1/p}$. Cuanto mayor es p , más se acerca ese lado al rango completo de cada variable:

```
r <- 0.1 # fracción de datos que queremos capturar (10%)
p <- c(1, 2, 5, 10, 20, 50, 100)
lado <- r^(1 / p)
data.frame(p = p, lado_hipercubo = round(lado, 3))
```

| ## | p | lado_hipercubo |
|------|-----|----------------|
| ## 1 | 1 | 0.100 |
| ## 2 | 2 | 0.316 |
| ## 3 | 5 | 0.631 |
| ## 4 | 10 | 0.794 |
| ## 5 | 20 | 0.891 |
| ## 6 | 50 | 0.955 |
| ## 7 | 100 | 0.977 |

```
plot(p, lado, type = "b", ylim = c(0, 1), xlab = "Número de predictores (p)",
     ylab = "Lado del hipercubo necesario")
abline(h = 1, lty = 2)
```

Con $p = 1$ basta con recorrer el 10% del rango de la variable; con $p = 100$ haría falta cubrir prácticamente el rango completo (97.7%) en *cada* predictor para conseguir esos mismos “vecinos”. Es decir, con muchos predictores, los vecinos más próximos dejan de ser realmente próximos, salvo que se disponga de muestras enormes. Esto explica por qué, en dimensión alta, suelen preferirse métodos que no dependen tanto de la localidad (modelos lineales, regularización) o técnicas previas de selección/reducción de variables, que se tratan en el libro completo de referencia.

7.1.5 Análisis e interpretación de los modelos

Además de obtener buenas predicciones, en muchos problemas resulta importante **analizar e interpretar los modelos ajustados**, es decir, comprender qué variables influyen en la respuesta y de qué manera. Este aspecto ha cobrado especial relevancia dentro del AE y el ML, dando lugar al área conocida como *interpretable machine learning*.

Existe un compromiso claro entre **capacidad predictiva e interpretabilidad**: a mayor complejidad del modelo, suele ser menor la facilidad de interpretación. Por ello, cuando varios modelos presentan un rendimiento similar, suele preferirse el más simple. En los modelos estadísticos clásicos (lineales, aditivos) la interpretación se apoya directamente en la estructura del modelo, aunque la colinealidad o las interacciones pueden dificultarla. En modelos más complejos (“cajas negras”) se recurre a herramientas adicionales, como las **medidas de importancia de variables** o los **gráficos de efectos parciales**.

En esta asignatura se emplearán principalmente modelos con una estructura interpretable; las herramientas avanzadas de interpretación se introducirán solo cuando resulten necesarias en capítulos posteriores.

Para quien quiera profundizar en aprendizaje estadístico más allá de esta introducción, el libro completo

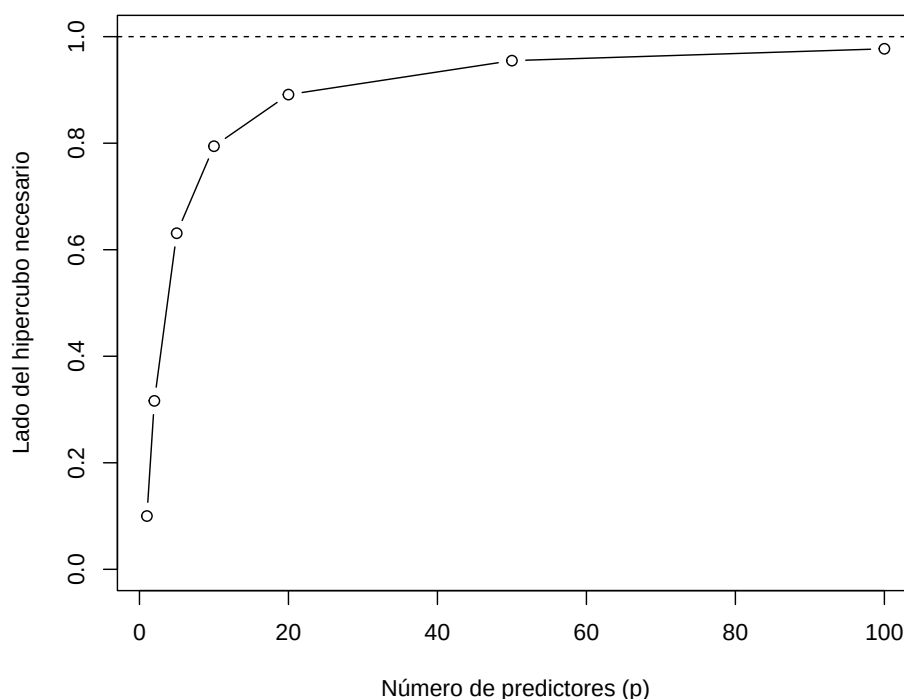


Figura 7.12: Lado del hipercubo necesario para capturar un 10% de los datos, según la dimensión.

de referencia dedica capítulos específicos a la regresión (selección de variables, regularización, regresión logística y multinomial), a la clasificación (más allá de KNN: árboles, SVM, evaluación específica con curvas ROC...), a la regresión no paramétrica (splines, modelos aditivos, regresión local) y a la maldición de la dimensionalidad, contenidos que quedan fuera del alcance de esta asignatura pero pueden ser de interés para quien continúe por esa línea.

7.2 Tecnologías Big Data (Hadoop/Spark y Visualización)

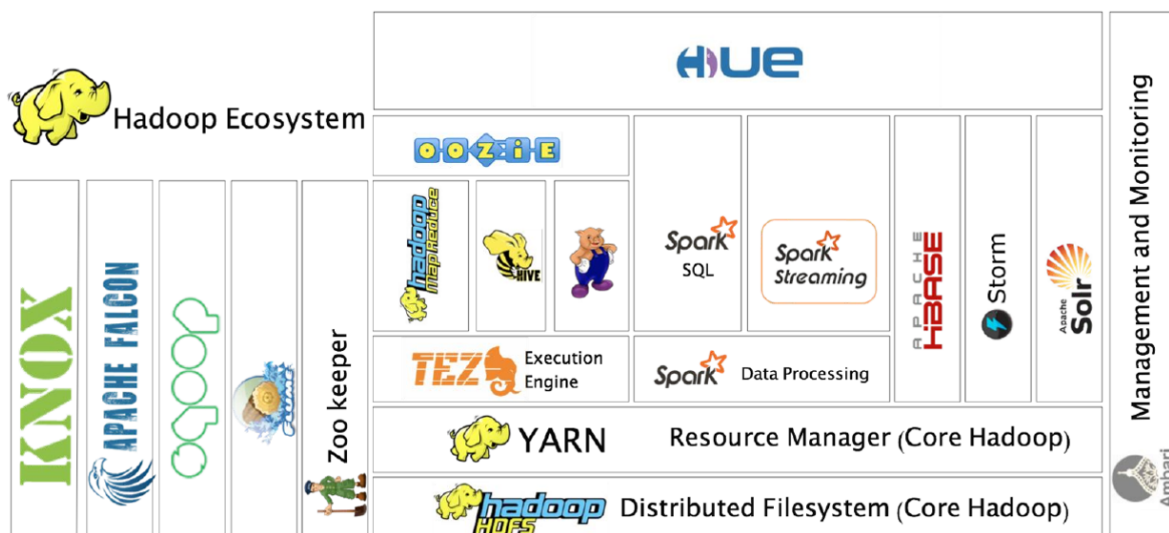
7.2.1 Tecnologías Hadoop, Spark, y Sparklyr

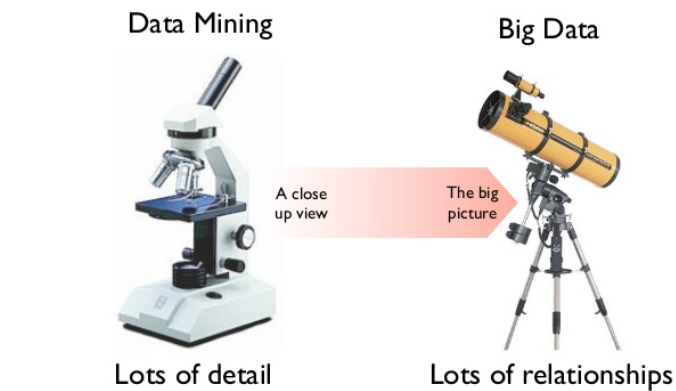
A continuación se introducen los conceptos básicos de las tecnologías Hadoop, Spark y Sparklyr:

- Hadoop: framework open-source desarrollado en Java principalmente que soporta aplicaciones distribuidas sobre miles de nodos y a escala Petabyte. Está inspirado en el diseño de las operaciones de MapReduce de Google y el Google File System (GFS). Entre sus principales componentes destaca HDFS Hadoop Distributed File System, sistema de archivos distribuido sobre múltiples nodos y accesible a nivel de aplicación. También destaca YARN como gestor de recursos, para ejecutar aplicaciones. Destacar que la versión original, Hadoop 1, estaba basada extensivamente en Map Reduce, Hadoop 2 colocó en su core a YARN y Hadoop 3 está orientado a la provisión de Plataforma como servicio y ejecución simultánea de múltiples cargas de trabajo distribuidas sobre recursos solicitados bajo demanda.
- Hive: es un sistema de almacenamiento y explotación de datos del estilo de un data warehouse open source diseñado para ser ejecutado en entornos Hadoop. Permite agrupar, consultar y analizar

datos almacenados en Hadoop File System y en Amazon S3 (almacenamiento de objetos en general) en esquema en estrella. Su lenguaje de consulta de datos, Hive Query Language o (HQL).

- Spark: framework de computación distribuida open-source para el procesamiento de datos masivos sobre Hadoop con un paralelismo implícito sobre su estructura de datos (Resilient Distributed Dataset o RDD), permite operar en paralelo sobre una colección de datos sin saber en qué servidores están disponibles dichos datos y de una forma tolerante a fallos. Es uno de los principales frameworks de programación de entornos Hadoop al estar optimizado su procesamiento sobre memoria (en lugar de sobre archivos en disco) para obtener velocidad, tanto en sus vertientes Spark streaming y Spark SQL, como Spark Machine Learning MLlib. Dispone de interfaces en Java, Scala, Python y R, siendo las interfaces de R Rspark y Sparklyr.
- SparkR: es un paquete, el primero que apareció, para conectar R con Spark. Intenta ser lo más parecida a la interfaz estándar de R de manipulación de datos.
- Sparklyr: es una librería para conectar R con Spark posterior a SparkR. Intenta ser lo más parecida a dplyr y embeber SQL en las consultas, soportando una mayor cantidad de paquetes. Por este motivo es el proyecto más activo actualmente, sustituyendo a SparkR.

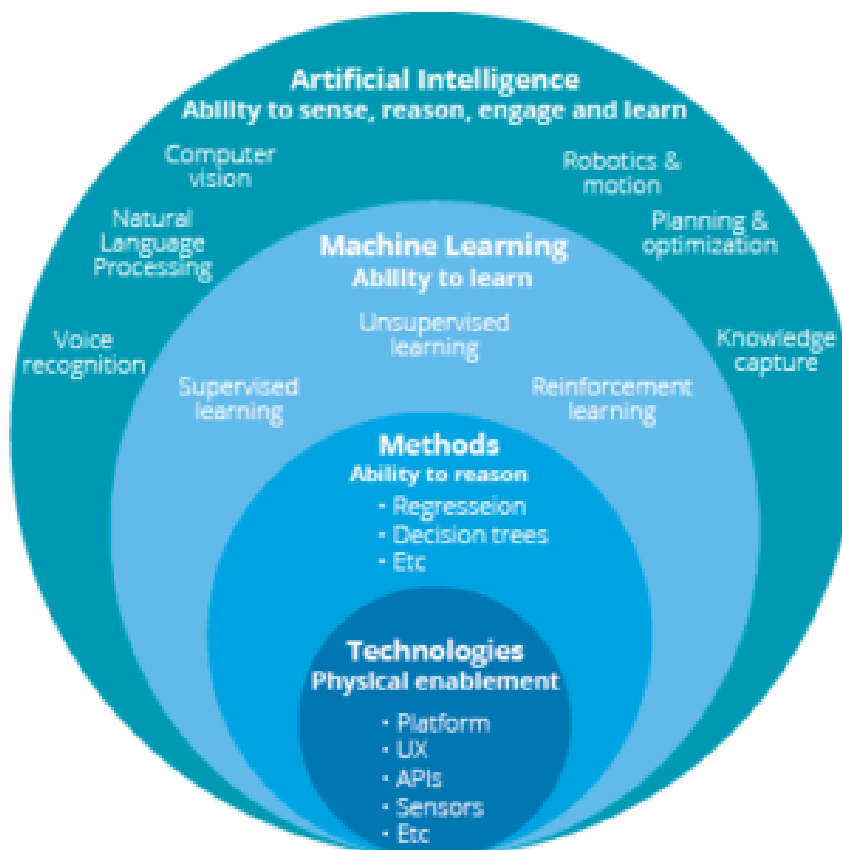


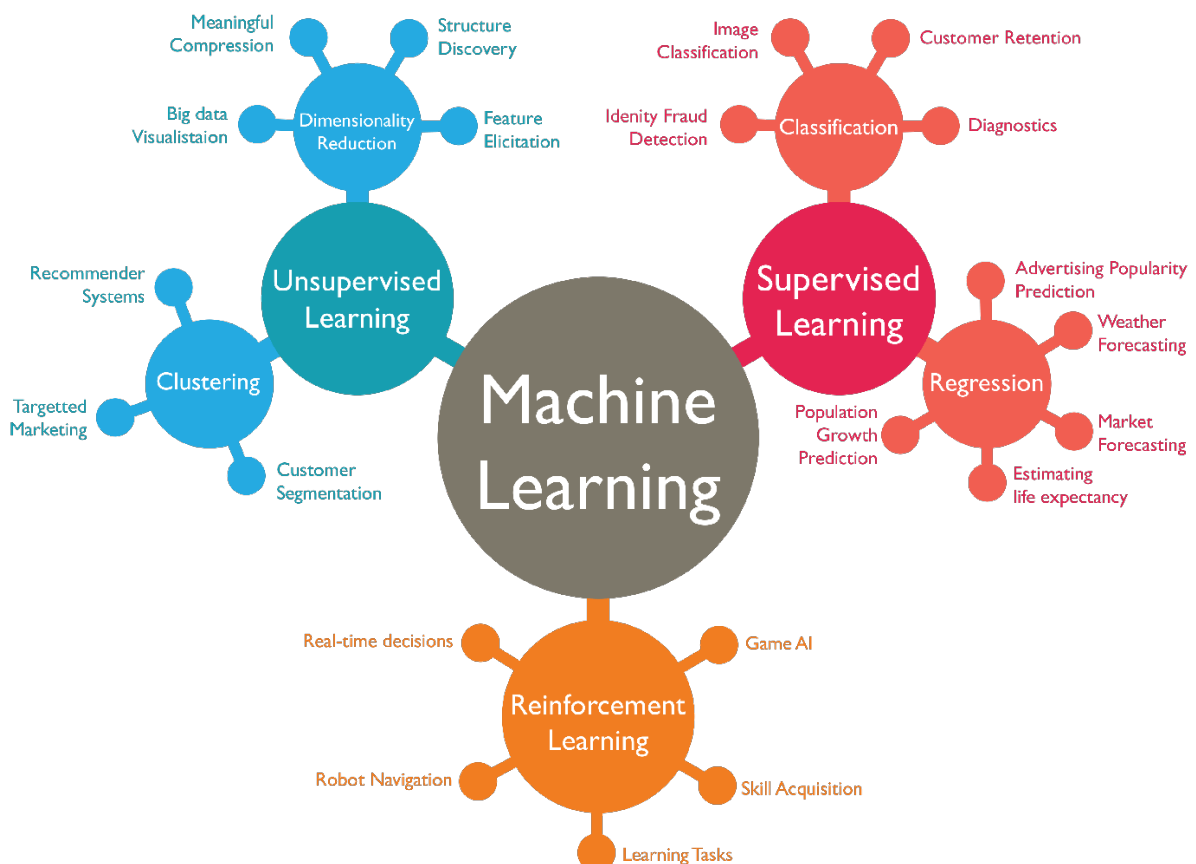
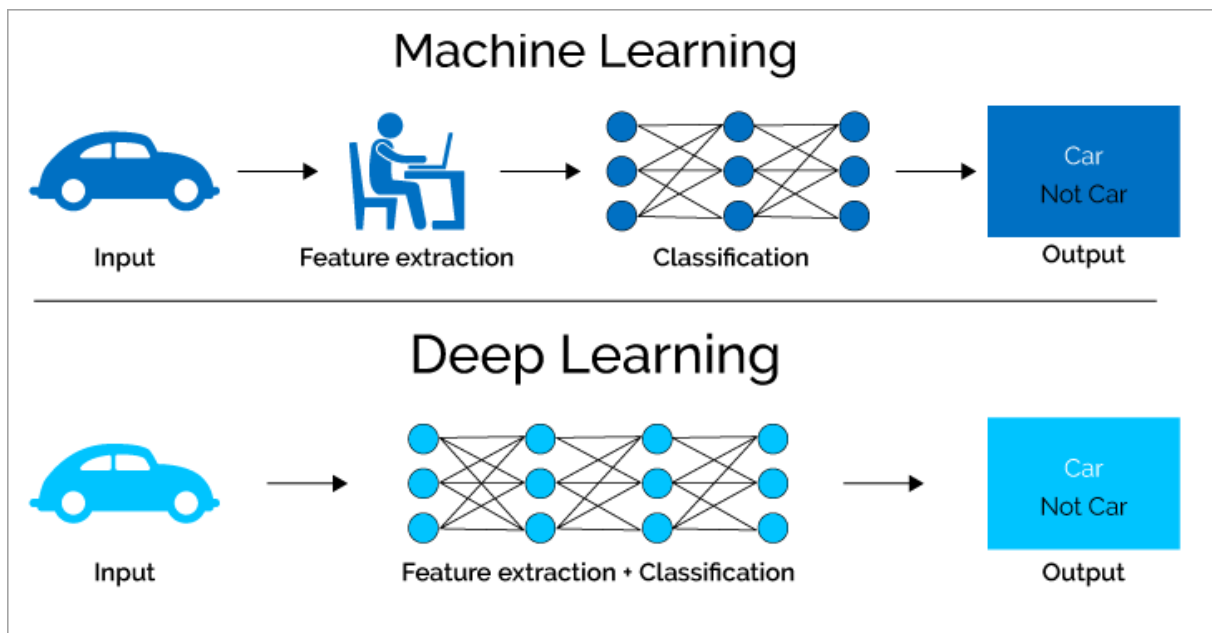


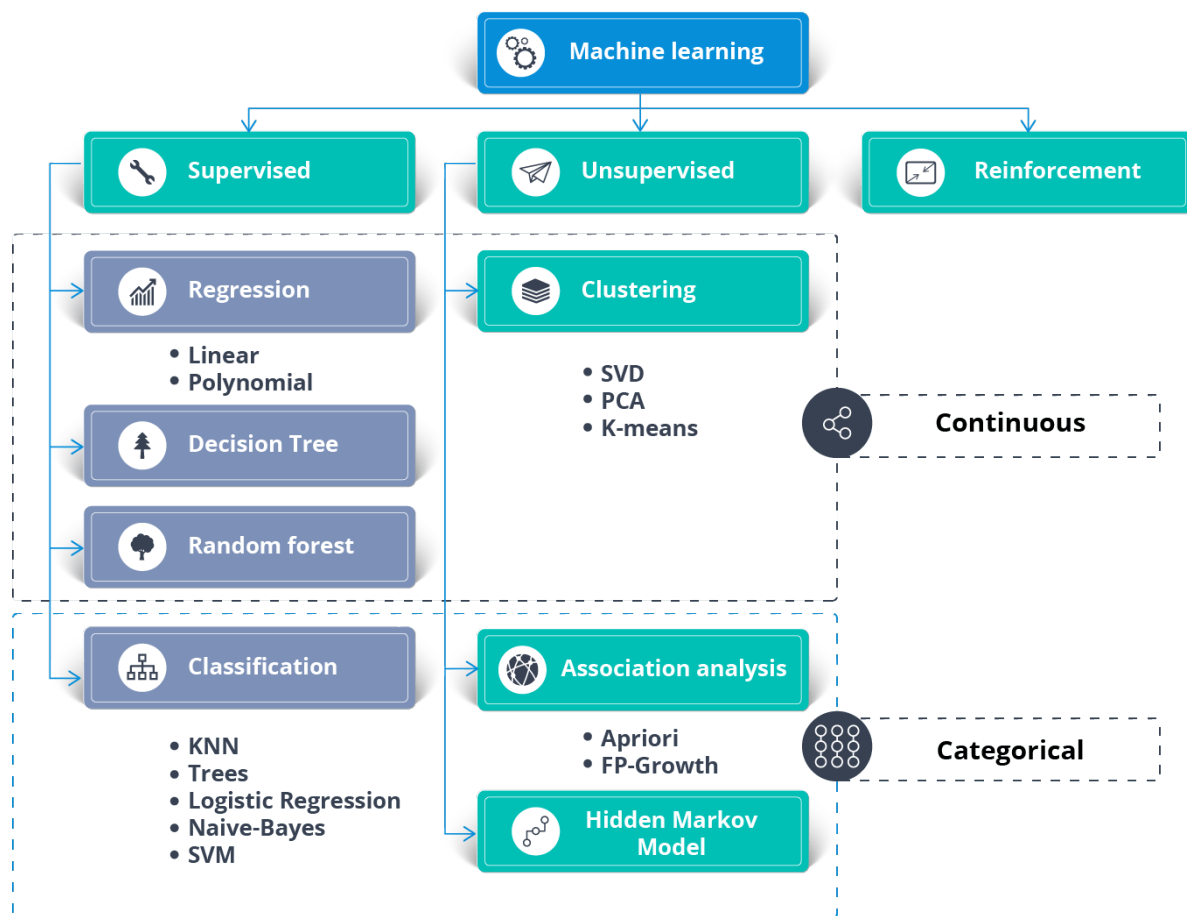
Thursday, 31 January 13

7.2.2 Big Data y Machine Learning

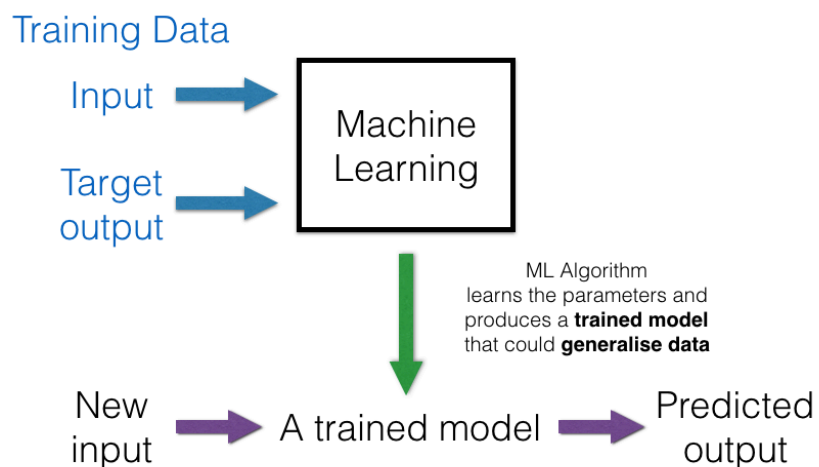
El Machine Learning o Aprendizaje Máquina es aquella parte de la inteligencia artificial con capacidad de aprender de los datos.







Y un ejemplo de cómo se trabaja en machine learning:

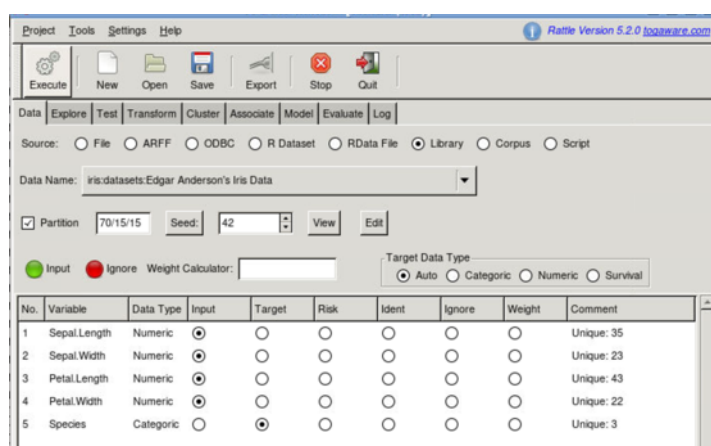


| | | Actual class (ground truth) | | |
|--------------------|---------------------------------------|---|---|--|
| Total (n)=100 | | Dog
(Positive) | Not a Dog
(Negative) | |
| Predicted
class | Dog
(Positive) | 15 (TP) | 20 (FP,
Type I Error) | Precision
=TP/(TP+FP)
=0.42 |
| | Not a Dog
(Negative) | 5 (FN,
Type II Error) | 60 (TN) | |
| | Accuracy
=(TP+TN)/Total
=0.75 | Sensitivity, Recall,
TPR =TP/(TP+FN)
= 0.75 | FPR =
FP/(FP+TN)
=0.25 | F1 Score
=2*(Precision*Recall)
/ (Precision+Recall)
=0.53 |
| | Error Rate
=(FP+FN)/Total
=0.25 | Miss Rate, FNR
=FN/(TP+FN)
=0.25 | Specificity, TNR
= TN/(FP+TN)
=0.75 | |

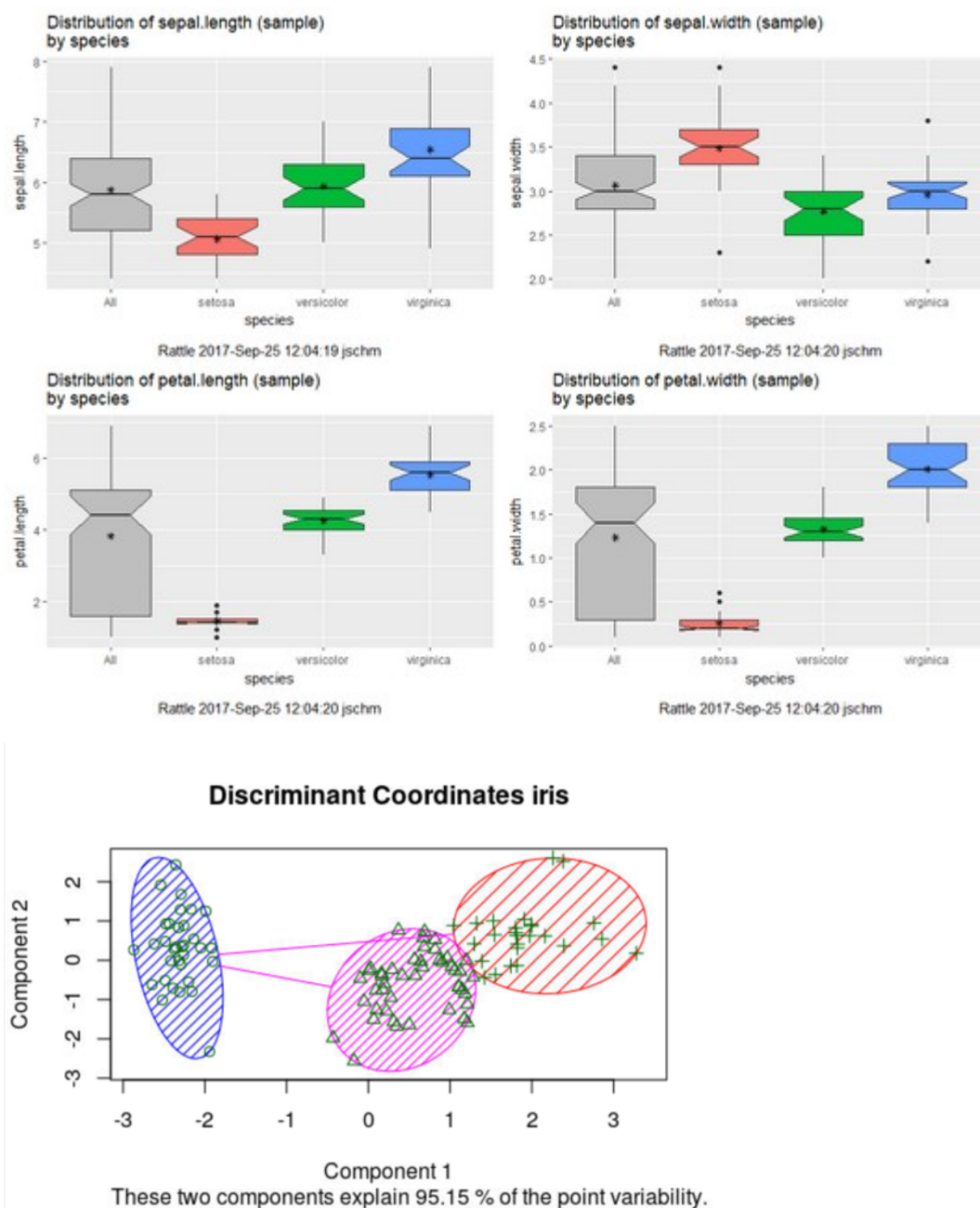
7.2.3 Rattle como alternativa a RapidMiner en R

Las instrucciones para instalar R está en el Apéndice 3 de este documento

Un tutorial adecuado para introducirse en Rattle es éste



Con el tutorial se pueden ver las capacidades de rattle de explorar los datos, como se puede apreciar a continuación.



7.2.4 Visualización y Generación de Cuadros de Mando

Se sigue un tutorial de la herramienta PowerBI, con datos de Excel y OData Feed

Como documentación de soporte se cuenta con la web de PowerBI y un tutorial adicional

7.3 Introducción al Análisis de Datos Masivos

En primer lugar se ha de considerar explorar los datos y realizar minería con ellos, y eso es posible hacerlo vía sparklyr.

Este apartado, eminentemente práctico, lo trabajaremos a través de la práctica 3 de TGD.

Bibliografía

- Bezanson, J., Edelman, A., Karpinski, S., and Shah, V. B. (2017). Julia: A fresh approach to numerical computing. *SIAM review*, 59(1):65–98.
- James, G., Witten, D., Hastie, T., Tibshirani, R., et al. (2013). *An introduction to statistical learning*, volume 112. Springer.
- James, G., Witten, D., Hastie, T., Tibshirani, R., and Taylor, J. (2023). *An introduction to statistical learning: With applications in python*. Springer Nature.
- Little, R. J. (1988). A test of missing completely at random for multivariate data with missing values. *Journal of the American statistical Association*, 83(404):1198–1202.
- Van Buuren, S. (2018). *Flexible imputation of missing data*. CRC press.
- Van Rossum, G. and Drake Jr, F. L. (1995). *Python reference manual*. Centrum voor Wiskunde en Informatica Amsterdam.
- Wickham, H., Çetinkaya-Rundel, M., and Grolemund, G. (2023). *R for data science*. ” O’Reilly Media, Inc.”.
- Wickham, H., François, R., Henry, L., Müller, K., and Vaughan, D. (2026). *dplyr: A Grammar of Data Manipulation*. R package version 1.2.1.